Week 1 - Preprocessing

Please run the cells of the notebook as you get to them while reading

In [89]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
```

1. Lesson on how to search for Python commands

Let's consider a few possible ways to learn about Python programming. Let's suppose you want to learn how to produce a short summary of the information in your DataFrame.

1. Your **instructor** could provide the information.

You could be provided with a lesson about functions like info() and describe(). If you have a pandas DataFrame called df, then you can summarize its contents using df.info() or df.describe(). df.info() provides a list of column names with their counts and data types. df.describe() will provide information such as the mean, min, max, standard deviation, and quantiles. Thus:

In [90]:
```
df = pd.DataFrame([[1, 4], [2, 5], [3, 6], [4, 7]], columns = ['A', 'B'])
df.describe()
```

Out[90]:

	A	B
count	4.000000	4.000000
mean	2.500000	5.500000
std	1.290994	1.290994
min	1.000000	4.000000
25%	1.750000	4.750000
50%	2.500000	5.500000
75%	3.250000	6.250000
max	4.000000	7.000000

In this describe() result, we see that the two columns A and B each have four elements. The means and other statistics are shown.

2. You could look up the information on **Google**.

If I Google the question "how do I briefly summarize the contents of a dataframe using Python," I receive the following link (among others), which discusses the describe() command mentioned above:

https://www.w3schools.com/python/pandas/ref_df_describe.asp

It also provide the complete usage information:

dataframe.describe(percentiles, include, exclude, datetime_is_numeric)

It explains that "percentiles" is set by default to [0.25, 0.5, 0.75] but we could change that. Let's try it! Since there are three intervals here rather than four, it might be more meaningful to ask about a 33rd and 67th percentile rather than 25, 50, and 75. We can use 1/3 for 0.33 and 2/3 for 0.67 to get the exact percentile values.

In [91]:
```
df = pd.DataFrame([[1, 4], [2, 5], [3, 6], [4, 7]], columns = ['A', 'B'])
df.describe(percentiles = [1/3, 2/3])
```

Out[91]:

	A	B
count	4.000000	4.000000
mean	2.500000	5.500000
std	1.290994	1.290994
min	1.000000	4.000000
33.3%	2.000000	5.000000
50%	2.500000	5.500000
66.7%	3.000000	6.000000
max	4.000000	7.000000

Apparently, the 50% value (the median) stays even though we did not specifically request it.

3. You could look up the official **documentation**.

Now that we know we want the pandas describe() function, try Googling: pandas documentation describe.

Here is the general documentation page for pandas:

https://pandas.pydata.org/docs/index.html

Here is the specific page for the describe() function:

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html

When I look at this, it appears to be showing the most recent (currently 2.2) version of pandas; this is shown in the upper right corner.

4. You could also ask **ChatGPT**.

Let's try it. ChatGPT, "how do I briefly summarize the contents of a dataframe using Python"

When I do this, ChatGPT mentions describe() among other options, but does not go into detail. However, I could ask it. ChatGPT, "tell me more about describe() in Python for summarizing dataframes."

Then, I get a good explanation of describe(), although it does not mention the percentiles option. One advantage of using Google or the documentation in addition of ChatGPT is that these sources may provide interesting information that does not directly answer our question. Thus, we might not have known about the various arguments, such as percentiles, if we only used ChatGPT. A second issue is that ChatGPT sometimes hallucinates (it makes up information). In general, by examining multiple sources - Google, documentation, and ChatGPT - we can get more information.

2. Weekly graph question

In Storytelling With Data, on page 1: examine the pie chart graph in the upper left corner of the graphs. Please write a short explanation of the pros and cons of this graph. What do you think of the choice of pie chart as a format? The color scheme? The legend? The title? How would you draw it differently if you were creating this graph?

The data could be represented better using a bar chart versus the pie chart. The pie chart was likely chosen because the data displayed is percentages adding up to 100%, which can be shown easily as the full pie chart. However, the pie chart makes it difficult to compare the quantities. Using a horizontal bar chart with a y axis showing the percentage or absolute count for each category would make it easier for comparison. Additionally, the title "Survey Results" is not effective since it misses context about what the survey was, and what is being measured in the chart. A better title would be "How do you feel about science?". The color scheme and legend are both done well. The colors contrast well and the legend labels make sense with an updated title. The one drawback of the labels is that they take up a lot of room in the chart. The follow-up charts in page 5 of Storytelling With Data have an improved legend which spans the top of the chart instead.

3. Homework - Bank Customers

I will begin by creating a file for you to analyze. I will show you all of the steps I used to create it. Please run this code in order to create and save a file about bank customers.

The numbered problems are for you to solve.

In [92]:
```
num_customers = 100
np.random.seed(0)
```

In [93]:
```
df_bank = pd.DataFrame(columns = ["CustomerID"])
```

In [94]:
```
df_bank["CustomerID"] = [str(x) for x in np.arange(num_customers)]
```

In [95]:
```
start = datetime(1950, 1, 1)
end = datetime(2024, 1, 1)
numdays = (end - start).days
random_days = np.random.randint(0, numdays, size = num_customers)
df_bank["BirthDate"] = start + pd.to_timedelta(random_days, unit='D')
df_bank["BirthDate"] = df_bank["BirthDate"].dt.strftime('%Y-%m-%d')
```

In [96]:
```
def make_ssn_string(num):
    ssn_str = f'{num:09}'
    return ssn_str[0:3] + "-" + ssn_str[3:5] + "-" + ssn_str[5:9]
ssn_vector_func = np.vectorize(make_ssn_string)
df_bank["SSN"] = ssn_vector_func(np.random.randint(0, 999999999, size = num_custome
```

In [97]:
```
df_bank["AccountID"] = np.random.randint(0, num_customers, size = num_customers)
```

In [98]:
```
random_days = np.random.randint(0, 365 * 80, size = num_customers)
df_bank["AccountOpened"] = (pd.to_datetime(df_bank["BirthDate"]) + pd.to_timedelta(
```

In [99]:
```
df_bank.loc[0, "BirthDate"] = "1980"
df_bank.loc[1, "BirthDate"] = "no date"
```

In [100…
```
df_bank.loc[2, "AccountID"] = np.nan
```

In [101…
```
df_bank["AccountType"] = np.random.choice(["checking", "savings", "cd"], size = num
```

Load the bank_customers.csv file. (There is no practical reason to save it, then load it - we're just demonstrating how this would be done.) I am calling the loaded df by a new name, df_bank_loaded, to make clear why it's not the same variable as the old df. Of course, in actuality the two contain the exact same data! But it's good to get in the habit of naming things carefully.

In [102…
```
df_bank.loc[num_customers - 1] = df.loc[0]
df_bank.to_csv("bank_customers.csv", index=False)
```

In [103…
```
df_bank_loaded = pd.read_csv("bank_customers.csv")
```

1. Use describe() and info() to analyze the data. Also, look at the first few rows.

In [104…
```
df_bank_loaded.describe()
```

Out[104…

	CustomerID	AccountID
count	99.000000	98.000000
mean	49.000000	46.551020
std	28.722813	27.679358
min	0.000000	0.000000
25%	24.500000	25.500000
50%	49.000000	42.000000
75%	73.500000	71.000000
max	98.000000	97.000000

In [105…

```
df_bank_loaded.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   CustomerID     99 non-null     float64
 1   BirthDate      99 non-null     object 
 2   SSN            99 non-null     object 
 3   AccountID      98 non-null     float64
 4   AccountOpened  99 non-null     object 
 5   AccountType    99 non-null     object 
dtypes: float64(2), object(4)
memory usage: 4.8+ KB
```

In [106…

```
df_bank_loaded
```

Out[106…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
0	0.0	1980	530-47-1866	4.0	1959-12-06	checking
1	1.0	no date	682-76-9175	67.0	2039-02-20	cd
2	2.0	1976-12-15	377-98-9839	NaN	2019-03-25	checking
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd
...	...	...	...	...	...	...
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd
97	97.0	1960-09-23	773-33-2797	75.0	1993-09-16	cd
98	98.0	1975-08-17	846-64-7054	34.0	2032-12-25	savings
99	NaN	NaN	NaN	NaN	NaN	NaN

100 rows × 6 columns

Suggested Google Search or ChatGPT prompt: "how do I use the describe function in python"

Example Google result: https://www.w3schools.com/python/pandas/ref_df_describe.asp

In [107…

```
# The first few rows
df_bank_loaded.iloc[0:5]
```

Out[107…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
0	0.0	1980	530-47-1866	4.0	1959-12-06	checking
1	1.0	no date	682-76-9175	67.0	2039-02-20	cd
2	2.0	1976-12-15	377-98-9839	NaN	2019-03-25	checking
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd

If you used describe() and info(), you now know that BirthDate and AccountOpened are strings. But we want them to be dates. Let's convert them to dates (or Timestamps in pandas). When we try this, we get a ValueError.

In [108…

```
try:
    df_bank_loaded["BirthDate"] = pd.to_datetime(df_bank_loaded["BirthDate"], forma
    print("It worked!")
except ValueError as e:
    print(f"ValueError for BirthDate: {e}")
```

```
ValueError for BirthDate: time data "1980" doesn't match format "%Y-%m-%d", at posit
ion 0. You might want to try:
    - passing `format` if your strings have a consistent format;
    - passing `format='ISO8601'` if your strings are all ISO8601 but not necessarily
in exactly the same format;
    - passing `format='mixed'`, and the format will be inferred for each element ind
ividually. You might want to use `dayfirst` alongside this.
```

In [109…

```
try:
    df_bank_loaded["AccountOpened"] = pd.to_datetime(df_bank_loaded["AccountOpened"
    print("It worked!")
except ValueError as e:
    print(f"ValueError for AccountOpened: {e}")
```

```
It worked!
```

The simple way to fix this is to remove the rows that have bad dates for BirthDate. I Googled:

"How to remove rows from a dataframe that have poorly formatted dates using python"

https://stackoverflow.com/questions/21556744/pandas-remove-rows-whose-date-does-not-follow-specified-format

This recommends that I verify that the date is a string of length 10, because YYYY-MM-DD has that length:

df1[df1.BirthDate.str.len() !=10]

In [110…

```
len(df_bank_loaded[df_bank_loaded.BirthDate.str.len() == 10])
```

Out[110…

```
97
```

In [111…

```
df_bank_loaded[df_bank_loaded.BirthDate.str.len() != 10].iloc[0:5]
```

Out[111…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
0	0.0	1980	530-47-1866	4.0	1959-12-06	checking
1	1.0	no date	682-76-9175	67.0	2039-02-20	cd
99	NaN	NaN	NaN	NaN	NaT	NaN

Now we can make this permanent, creating a new DataFrame df_bank_datefix. I am making a copy in order to ensure that df_bank_datefix is a new DataFrame rather than being a slice of the old one.

In [112…

```
df_bank_datefix = df_bank_loaded[df_bank_loaded.BirthDate.str.len() == 10].copy()
```

Test again:

In [113…

```
try:
    df_bank_datefix["BirthDate"] = pd.to_datetime(df_bank_datefix["BirthDate"], for
    print("It worked!")
```

```
except ValueError as e:
    print(f"ValueError: {e}")
```

```
It worked!
```

2. To check that it worked, use a summary function that will tell you if the BirthDate field is now a datetime type

In [114…

```
df_bank_datefix.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 97 entries, 2 to 98
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CustomerID     97 non-null     float64
 1   BirthDate      97 non-null     datetime64[ns]
 2   SSN            97 non-null     object
 3   AccountID      96 non-null     float64
 4   AccountOpened  97 non-null     datetime64[ns]
 5   AccountType    97 non-null     object
dtypes: datetime64[ns](2), float64(2), object(2)
memory usage: 5.3+ KB
```

3. Check whether there are any null values in the DataFrame. If so, remove those rows or (if you prefer) fill in the value with an appropriate number.

First try at a Google search or ChatGPT prompt: "how do I find out if there are any null values in a pandas DataFrame?"

This page gives an answer. Unfortunately, it took my request too literally: it tells me only if there are any, and not which rows have them. On reflection, that's not really what I want - I think I asked the wrong question. I want to see the rows, not just *whether* there are any.

https://stackoverflow.com/questions/29530232/how-to-check-if-any-value-is-nan-in-a-pandas-dataframe

ChatGPT likewise doesn't give the answer I want - because I asked the wrong question.

Next try at a Google search or ChatGPT prompt: "how do I check which rows have null values in a pandas DataFrame?"

This page gives an answer:

https://stackoverflow.com/questions/36226083/how-to-find-which-columns-contain-any-nan-value-in-pandas-dataframe

ChatGPT also gives a good answer. I recommend looking at both of them!

Now try it on your own:

Suggested Google search or ChatGPT prompt: "how do I remove rows with null values in a pandas DataFrame?"

Suggested Google search or ChatGPT prompt: "how do I fill in null values in a pandas DataFrame?"

In [115…

```
df_bank_datefix.isna().sum()
```

Out[115…

```
CustomerID       0
BirthDate        0
SSN              0
AccountID        1
AccountOpened    0
AccountType      0
dtype: int64
```

In [116…

```
df_bank_datefix[df_bank_datefix.isna().any(axis=1)]
```

Out[116…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
2	2.0	1976-12-15	377-98-9839	NaN	2019-03-25	checking

In [117…

```
df_bank_datefix_non_null = df_bank_datefix.dropna()
print(f'Length before dropping nulls: {len(df_bank_datefix)}')
print(f'Length after dropping nulls: {len(df_bank_datefix_non_null)}')
df_bank_datefix_non_null
```

```
Length before dropping nulls: 97
Length after dropping nulls: 96
```

Out[117…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings
...	...	...	...	...	...	...
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd
97	97.0	1960-09-23	773-33-2797	75.0	1993-09-16	cd
98	98.0	1975-08-17	846-64-7054	34.0	2032-12-25	savings

96 rows × 6 columns

4. Find out if there are any duplicate rows (two rows exactly the same). List their row numbers. Then remove the duplicates

In [118…
```
df_bank_datefix_non_null[df_bank_datefix_non_null.duplicated(keep=False)]
```

Out[118…

CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType

There are no strictly duplicate rows, but we can confirm by checking if any duplicate SSNs

In [119…
```
df_bank[df_bank.duplicated(subset="SSN", keep=False)]
```

Out[119…

CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType

Suggested Google search or ChatGPT prompt: "how can I find out if there are any duplicate rows in a DataFrame using Python"

Again, Google provides me with a page that addresses the question:

https://saturncloud.io/blog/how-to-find-all-duplicate-rows-in-a-pandas-dataframe/

To remove the duplicates, do this search: "how can I remove the duplicate rows in a DataFrame using Python"

This leads me to the following documentation.

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop_duplicates.html

In [120…
```
# There were no duplicates so this does not do anything
df_bank_datefix_non_null = df_bank_datefix_non_null.drop_duplicates()
df_bank_datefix_non_null
```

Out[120…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings
...	...	...	...	...	...	...
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd
97	97.0	1960-09-23	773-33-2797	75.0	1993-09-16	cd
98	98.0	1975-08-17	846-64-7054	34.0	2032-12-25	savings

96 rows × 6 columns

5. Check whether the customers all have unique AccountIDs. If not, provide the first example of a non-unique AccountId.

In [121…

```
df_bank_datefix_non_null[df_bank_datefix_non_null.duplicated(subset="AccountID", ke
```

Out[121…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
41	41.0	1996-08-30	560-38-9694	3.0	2073-11-13	savings
74	74.0	2023-01-19	086-36-1272	3.0	2093-12-24	cd

Suggested Google search or ChatGPT prompt: "how can I find the first non-unique item from a pandas Series in python"

By the way: why didn't I ask the question "how can I check whether the customers all have unique AccountIDs"?

The problem would be that Google and ChatGPT don't know what "customers" you are talking about. It's important to understand that the AccountIDs are a column of a DataFrame, and as such they are a Series. Therefore, we should use the correct vocabulary and ask about a Series. If you mess up and ask about a "list" instead of a Series, you *might* get an answer that still works. But it's better to get the vocabularly right.

It's important to add "in python" because this task could be performed in many languages.

ChatGPT gave me this suggestion: data[data.isin(data[data.duplicated()])].iloc[0] However, ChatGPT did not explain how this code worked and even claimed (falsely) that it was going to use the value_counts() function in the solution. So although the code is correct, I personally found ChatGPT's answer very confusing. You could, perhaps, ask ChatGPT to explain further how this code works.

ChatGPT, "How does this code work: data[data.isin(data[data.duplicated()])].iloc[0]"

On the other hand, Google leads me to the documentation for the duplicated() function:

https://pandas.pydata.org/docs/reference/api/pandas.Series.duplicated.html

Here, I can see that when I really need is data.duplicated(keep = False), where "data" should be the Series in question. However, this just gives me a Series of boolean values indicating which ones are duplicates. I have to somehow know that extracting the numerical values instead of a Series of booleans involves boolean indexing: data[data.duplicated(keep = False)].

So as usual, I'd suggest that a combination of Google, documentation, and ChatGPT will give you the best information.

6. Count how many distinct AccountIDs there are.

In [122…

```
len(df_bank_datefix_non_null['AccountID'].unique())
```

Out[122…

```
63
```

Suggested Google search or ChatGPT prompt: "how can I find out how many distinct items there are in a pandas Series using python"

This time Google provides me with a page that's specifically made to answer this question:

https://www.geeksforgeeks.org/how-to-count-distinct-values-of-a-pandas-dataframe-column/

7. Remove the duplicate AccountIDs so that each AccountID appears only once.

This will involve using data.duplicated() but this time without keep = False. We don't want to drop all duplicates; we want to leave one example of each value.

In [123…

```
df_bank_datefix_non_null_unique_accts = df_bank_datefix_non_null.drop_duplicates(su
df_bank_datefix_non_null_unique_accts
```

Out[123…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings
...	...	...	...	...	...	...
88	88.0	1962-02-07	327-09-3267	97.0	1989-10-03	cd
90	90.0	1969-06-18	395-31-7064	34.0	2010-12-28	checking
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd

63 rows × 6 columns

In [124…

```
# Confirming no dupes left
df_bank_datefix_non_null_unique_accts[df_bank_datefix_non_null_unique_accts.duplica
```

Out[124…

CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType

8. What are the mean, median, and mode customer age in years? (Rounding down to the next lower age.) Are there any outliers? (Customers with very large or very small ages, compared with the other ages?)

In [125…

```
df_bank_with_age = df_bank_datefix_non_null.copy()
df_bank_with_age
```

Out[125…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings
...	...	...	...	...	...	...
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd
97	97.0	1960-09-23	773-33-2797	75.0	1993-09-16	cd
98	98.0	1975-08-17	846-64-7054	34.0	2032-12-25	savings

96 rows × 6 columns

In [126…

```
# https://stackoverflow.com/questions/26788854/pandas-get-the-age-from-a-date-examp
from datetime import date
def calculate_age(born):
    today = date.today()
    return today.year - born.year - ((today.month, today.day) < (born.month, born.d

df_bank_with_age['Age'] = df_bank_with_age['BirthDate'].apply(calculate_age)
```

In [127…

```
df_bank_with_age
```

Out[127…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType	Age
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd	22
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd	40
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking	17
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings	50
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings	9
...	...	...	...	...	...	...	...
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings	56
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking	50
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd	36
97	97.0	1960-09-23	773-33-2797	75.0	1993-09-16	cd	65
98	98.0	1975-08-17	846-64-7054	34.0	2032-12-25	savings	50

96 rows × 7 columns

In [128…

```
df_bank_with_age['Age'].mean(), df_bank_with_age['Age'].median(), df_bank_with_age[
```

Out[128…

```
(np.float64(35.010416666666664),
 np.float64(34.0),
 0    29
 Name: Age, dtype: int64)
```

Suggested Google search or ChatGPT prompt: "how can I find out the mean, median, and mode of a pandas Series"

9. One-hot encode the AccountType column. This means creating a new "checking," "savings", and "cd" columns so that you can run machine learning algorithms.

In [129…

```
df1 = df_bank_datefix_non_null_unique_accts.copy()
df2 = df_bank_datefix_non_null_unique_accts.copy()

one_hot = pd.get_dummies(df1["AccountType"], dtype=int)
df2 = df2.join(one_hot)
df2.iloc[0:5]
```

Out[129…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType	cd	checking
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd	1	0
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd	1	0
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking	0	1
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings	0	0
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings	0	0

Now, change the cd, checking, and savings columns into integers.

I modified the original code to do this by using `dtype=int` argument

10. Are there any other data values that do not seem right? If not, give an example?

In [130…

```
df2.sort_values(by="BirthDate")
```

Out[130…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType	cd	checking
39	39.0	1952-01-26	810-29-3625	67.0	1983-11-04	checking	0	1
40	40.0	1952-03-08	291-98-5506	82.0	1953-05-05	cd	1	0
33	33.0	1955-02-15	986-06-7499	93.0	1967-11-11	cd	1	0
28	28.0	1955-12-04	863-97-2245	40.0	1996-02-08	checking	0	1
35	35.0	1956-11-01	470-06-0415	95.0	2031-10-28	checking	0	1
...	...	...	...	...	...	...	...	...
37	37.0	2017-07-23	991-68-0187	88.0	2094-12-17	checking	0	1
54	54.0	2017-09-13	403-47-1401	37.0	2066-06-21	cd	1	0
59	59.0	2019-06-26	119-05-0326	89.0	2065-03-29	savings	0	0
62	62.0	2022-08-23	719-02-2091	51.0	2090-07-02	cd	1	0
61	61.0	2023-03-31	040-18-6728	53.0	2068-01-17	savings	0	0

63 rows × 9 columns

One issue with the data types is that AccountID is treated as a float, for example 4.0. We should be treating them as integers instead since they are always whole numbers

For the values themselves, we can see there are some issues with the AccountOpened feature. We see values in the future like "2093-12-24", which is impossible. Additionally, there are customers with recent BirthDates like "2023-12-26". It is unlikely that a customer would have an account this early in life unless their family opened it for them. We would want to evaluate the business rules to validate.

I don't think Google or ChatGPT alone will help you here. To answer the question, look at the columns and think about what relationships they should have with each other. For example, it seems reasonable to expect that BirthDate would be no earlier than 120 years ago (it's unlikely that a customer would be this old.) Now we can ask Google:

"How can I find out how long ago a pandas date is"

Google provides this helpful link, although it is not exactly the solution - you'll have to work with it a bit:

https://stackoverflow.com/questions/26072087/pandas-number-of-days-elapsed-since-a-certain-date

If you check, I think you'll find that all dates are more recent than 120 years ago. What about the AccountOpened columns? I see some obviously wrong dates there just by looking at the first few rows.

Along those same lines, are there any birth dates that are too recent? Do we think that any two year olds will have opened bank accounts? How common do you think this is in real life? How common is it in our data set? Can you detect the two year olds opening bank accounts using just one column, or do you need two columns?

11. Use Matplotlib and/or Seaborn to analyse the ages at which customers open their account. Is there a connection between the year they are born vs. the age at which they open the account? Graph this in whatever way you think is best.

I asked Google and ChatGPT: "How can I plot dates vs. dates in Matplotlib". This gave me a hard time at first - I had to tell ChatGPT it was giving me the wrong information because it tried to plot dates vs. numbers. Eventually, I found out that you plot dates vs. dates in the same way you'd plot numbers vs. numbers.

Think in terms of Storytelling With Data to plot these as best you can. Once you've seen the result, try to think of the best way to plot the data so as to show the user what you want them to see. Title the graph so as to display the lesson that you want the user to take away. Here are some options for the axes:

1. A scatter or line plot: On the x-axis, the date they are born. On the y-axis, the date they open the account.
2. A scatter or line plot: On the x-axis, the date they are born. On the y-axis, the age in years at which they open the account.
3. A scatter or line plot: On the x-axis, they year (integer) they are born. On the y-axis, the age in years at which they open the account.
4. A histogram: on the x-axis, the age at which they open the account.

Here is an example:

In [131…

```
df2
```

Out[131…

	CustomerID	BirthDate	SSN	AccountID	AccountOpened	AccountType	cd	checking
3	3.0	2003-10-18	474-05-7613	86.0	2004-03-02	cd	1	0
4	4.0	1985-12-06	750-55-5509	77.0	2059-05-05	cd	1	0
5	5.0	2008-02-29	671-43-0485	75.0	2025-05-03	checking	0	1
6	6.0	1975-04-05	288-62-9296	56.0	2048-09-21	savings	0	0
7	7.0	2016-06-18	593-49-1249	16.0	2087-07-24	savings	0	0
...	...	...	...	...	...	...	...	...
88	88.0	1962-02-07	327-09-3267	97.0	1989-10-03	cd	1	0
90	90.0	1969-06-18	395-31-7064	34.0	2010-12-28	checking	0	1
94	94.0	1969-03-14	958-12-8308	5.0	2037-04-03	savings	0	0
95	95.0	1975-09-23	607-97-1651	36.0	2011-07-24	checking	0	1
96	96.0	1989-03-09	870-80-0364	0.0	2054-11-01	cd	1	0

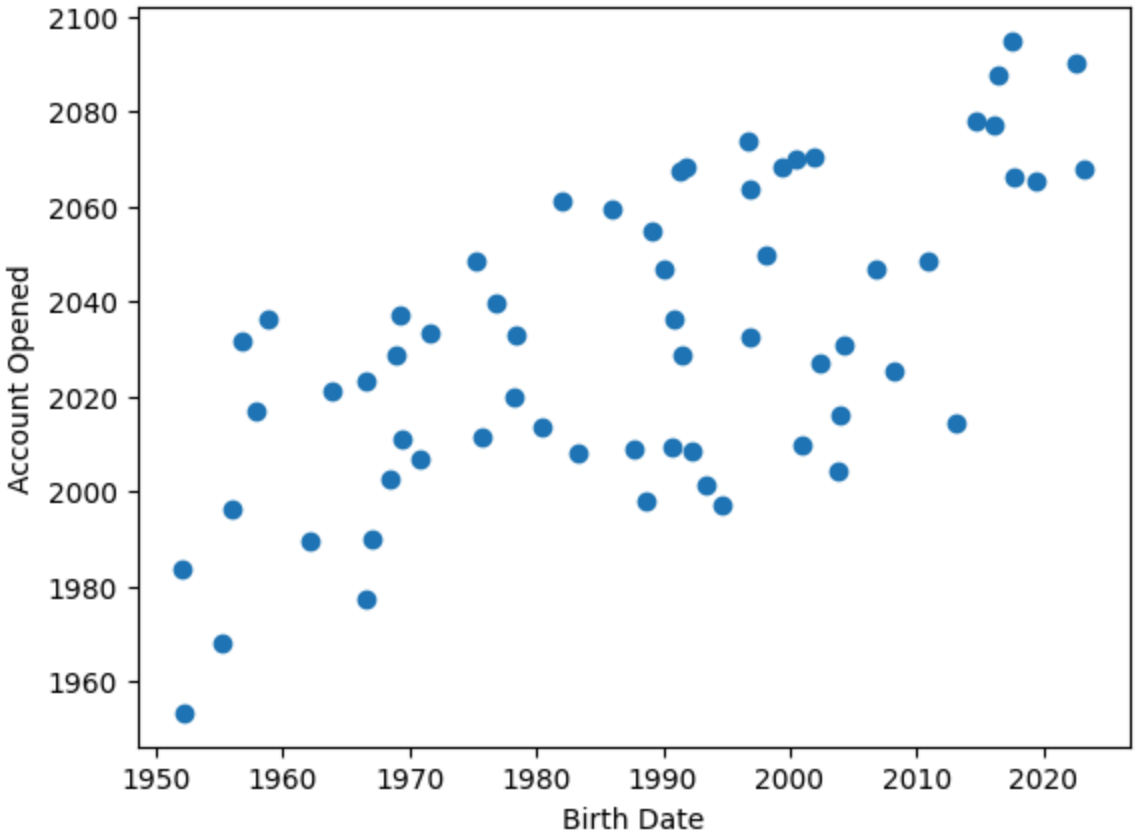
63 rows × 9 columns

In [132…

```
import matplotlib.pyplot as plt

ax = plt.gca() # get an "Axes" object to draw on; gca stands for "get current Axes"
ax.scatter(df2["BirthDate"], df2["AccountOpened"]) # create a scatter plot based on
ax.set_ylabel("Account Opened") # label the y axis
ax.set_xlabel("Birth Date") # label the x axis
```

Out[132…

```
Text(0.5, 0, 'Birth Date')
```

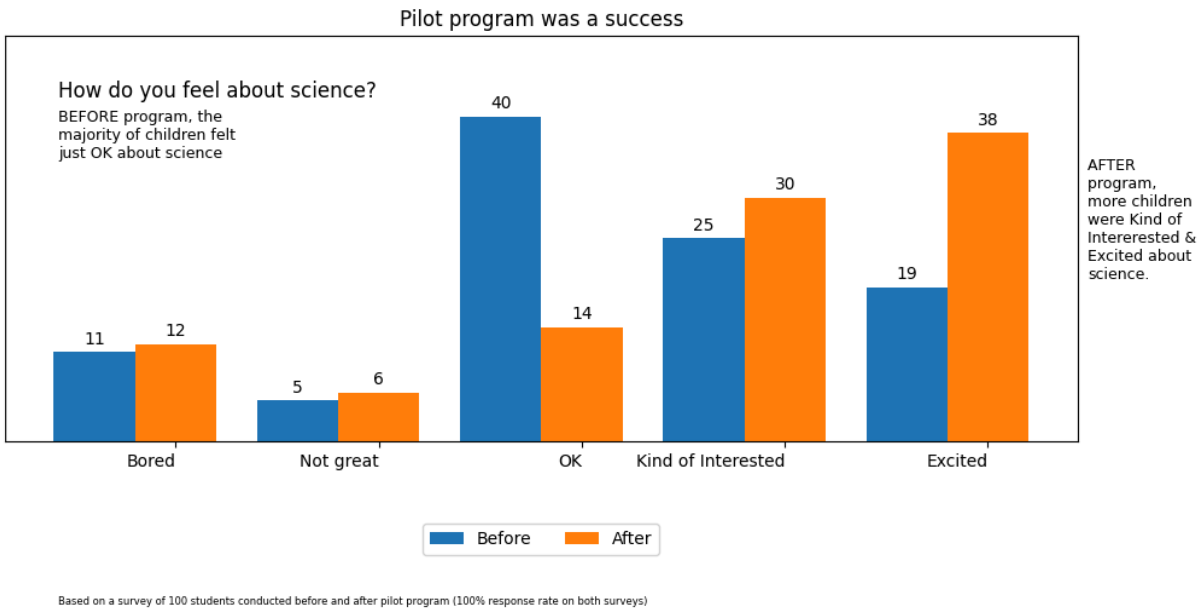


4. Storytelling With Data graph

Choose any graph in the Introduction of Storytelling With Data. Using matplotlib to reproduce it in a rough way. I don't expect you to spend an enormous amount of time on this; I understand that you likely will not have time to re-create every feature of the graph. However, if you're excited about learning to use matplotlib, this is a good way to do that. You don't have to duplicate the exact values on the graph; just the same rough shape will be enough. If you don't feel comfortable using matplotlib yet, do the best you can and write down what you tried or what Google searches you did to find the answers.

In [58]:
```
import matplotlib.pyplot as plt
import numpy as np

# Figure 0.5 (page 5)
import matplotlib.pyplot as plt
categories = ("Bored", "Not great", "OK", "Kind of Interested", "Excited")
pilot_program_data = {
    'Before': (11, 5, 40, 25, 19),
    'After': (12, 6, 14, 30, 38)
}

x = np.arange(len(categories))  # the label locations
width = 0.4  # the width of the bars
multiplier = 0
```

```
fig, ax = plt.subplots(layout='constrained', figsize=(10, 5))

for attribute, measurement in pilot_program_data.items():
    offset = width * multiplier
    rects = ax.bar(x + offset, measurement, width, label=attribute)
    ax.bar_label(rects, padding=3)
    multiplier += 1

ax.set_title('Pilot program was a success')
ax.set_xticks(x + width, categories, ha='right')
ax.legend(loc='lower center', ncols=2, bbox_to_anchor=(0.5, -0.3))
ax.set_ylim(0, 50)
ax.yaxis.set_visible(False)
ax.text(0.05, 0.85, "How do you feel about science?", horizontalalignment='left', f
ax.text(0.05, 0.7, "BEFORE program, the \nmajority of children felt \njust OK about
ax.text(1.01, 0.4, "AFTER \nprogram, \nmore children \nwere Kind of \nIntererested
ax.text(0.05, -0.4, "Based on a survey of 100 students conducted before and after p
None
```



Week 2 - Preprocessing, part 2

1. Lesson: None

2. Weekly graph question

The Storytelling With Data book mentions planning on a "Who, What, and How" for your data story. Write down a possible Who, What, and How for your data, using the ideas in the book.

By utilizing data science, we can employ models to detect financial crime and fraud detection. Financial services companies such as credit card and loan issuers can utilize these models to ensure customers are not charged for items they did not purchase, and loans are not requested for invalid use cases.

Who: Business stakeholders

What: Employing a data science and engineering team and deploying detection models will identify and prevent X amount of fraudulent charges and loan applications

How: Historical credit card transaction data, loan application data, and customer data will be utilized to train models. These models will be deployed to detect future financial crime and fraud on future transactions and applications with an alerting system and manual or automated intervention workflows.

3. Homework - work with your own data

In [1]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
```

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do Use Google, documentation, and ChatGPT to help you:

- Summarize the datasets using info() and describe()

- Are there any duplicate rows?
- Are there any duplicate values in a given column (when this would be inappropriate?)
- What are the mean, median, and mode of each column?
- Are there any missing or null values?
  - Do you want to fill in the missing value with a mean value? A value of your choice? Remove that row?
- Identify any other inconsistent data (e.g. someone seems to be taking an action before they are born.)
- Encode any categorical variables (e.g. with one-hot encoding.)

Conclusions:

- Are the data usable? If not, find some new data!
- Do you need to modify or correct the data in some way?
- Is there any class imbalance? (Categories that have many more items than other categories).

Loading the Data

In [2]:
```
# https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
df = pd.read_csv(f"C:/Users/caleb/Downloads/archive/creditcard.csv")
```

In [3]:
```
df
```

Out[3]:

	Time	V1	V2	V3	V4	V5	V6	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.2395
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.0788
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.7914
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.2376
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.5929
...	...	...	...	...	...	...	...	
284802	172786.0	-11.881118	10.071785	-9.834783	-2.066656	-5.364473	-2.606837	-4.9182
284803	172787.0	-0.732789	-0.055080	2.035030	-0.738589	0.868229	1.058415	0.0243
284804	172788.0	1.919565	-0.301254	-3.249640	-0.557828	2.630515	3.031260	-0.2968
284805	172788.0	-0.240440	0.530483	0.702510	0.689799	-0.377961	0.623708	-0.6861
284806	172792.0	-0.533413	-0.189733	0.703337	-0.506271	-0.012546	-0.649617	1.5770

284807 rows × 31 columns

Summarizing the Data

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

There are no missing values. The data types also look correct for every feature except "Time", which should be treated as an integer or datetime instead.

In [5]:
```
df.describe()
```

Out[5]:

	Time	V1	V2	V3	V4	
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.8480
mean	94813.859575	1.168375e-15	3.416908e-16	-1.379537e-15	2.074095e-15	9.6040
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.3802
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.1374
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.9159
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.4335
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.1192
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.4801

8 rows × 31 columns

As discussed in the documentation, the features V1-V28 are the result of PCA transformation of the original data to protect client confidentiality. Time, Amount, and Class are the other features, which all have different scales. Class is the intended target variable.

Identifying Duplicates

In [6]:

```
# Identify any identical rows
duplicates = df[df.duplicated(keep=False)]
duplicates
```

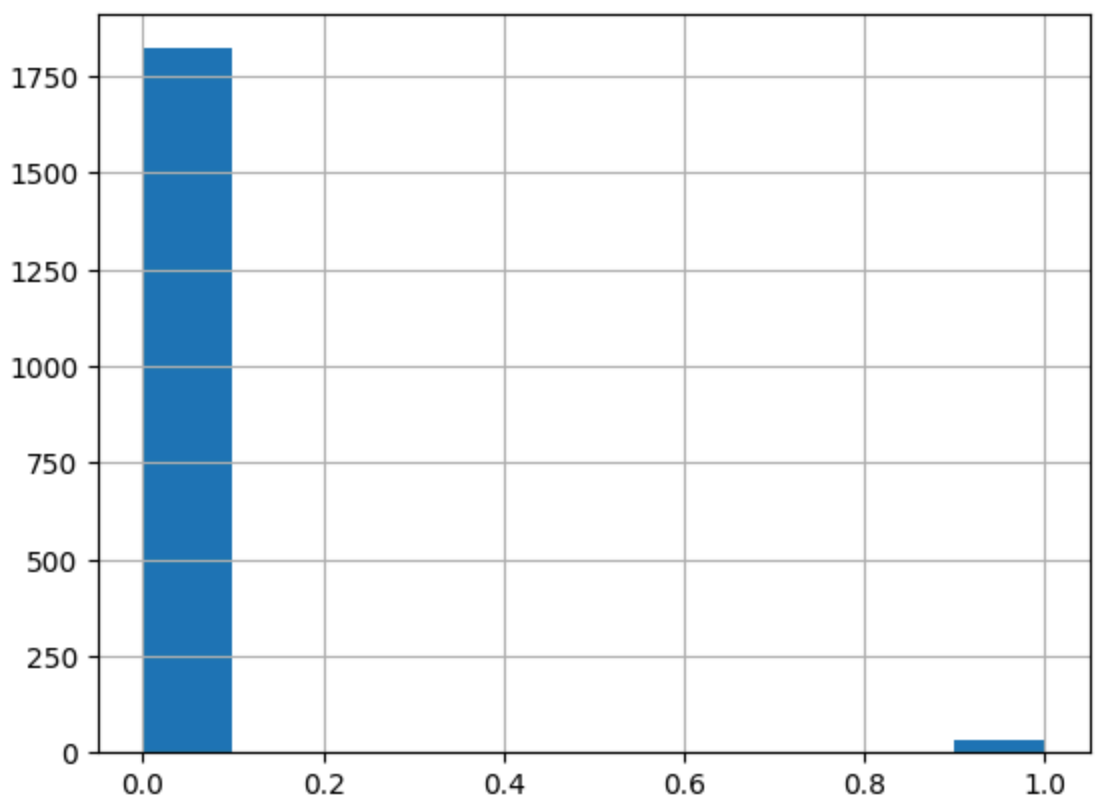
Out[6]:

	Time	V1	V2	V3	V4	V5	V6	V7
32	26.0	-0.529912	0.873892	1.347247	0.145457	0.414209	0.100223	0.711206
33	26.0	-0.529912	0.873892	1.347247	0.145457	0.414209	0.100223	0.711206
34	26.0	-0.535388	0.865268	1.351076	0.147575	0.433680	0.086983	0.693039
35	26.0	-0.535388	0.865268	1.351076	0.147575	0.433680	0.086983	0.693039
112	74.0	1.038370	0.127486	0.184456	1.109950	0.441699	0.945283	-0.036715
...	...	...	...	...	...	...	...	...
283485	171627.0	-1.457978	1.378203	0.811515	-0.603760	-0.711883	-0.471672	-0.282535
284190	172233.0	-2.667936	3.160505	-3.355984	1.007845	-0.377397	-0.109730	-0.667233
284191	172233.0	-2.667936	3.160505	-3.355984	1.007845	-0.377397	-0.109730	-0.667233
284192	172233.0	-2.691642	3.123168	-3.339407	1.017018	-0.293095	-0.167054	-0.745886
284193	172233.0	-2.691642	3.123168	-3.339407	1.017018	-0.293095	-0.167054	-0.745886

1854 rows × 31 columns

In [7]:
```
duplicates['Class'].hist()
```

Out[7]: <Axes: >



In [8]:
```
len(duplicates[duplicates['Class'] == 1])
```

Out[8]: 32

In [9]:
```
len(df[df['Class'] == 1])
```

Out[9]: 492

In [10]:
```
32 / 492
```

Out[10]: 0.06504065040650407

If we remove the duplicates it would result in losing 6% of the positive class (fraudulent charges). I will take the assumption that it is important to keep all these transactions instead of removing them. It is worth further investigation as to why there could be duplicate rows (especially with identical timestamps)

Identifying Inconsistent Data

Amounts

In [11]:
```
df['Amount'].min(), df['Amount'].max()
```

Out[11]: (np.float64(0.0), np.float64(25691.16))

In [12]:
```
len(df[df['Amount'] == 0])
```

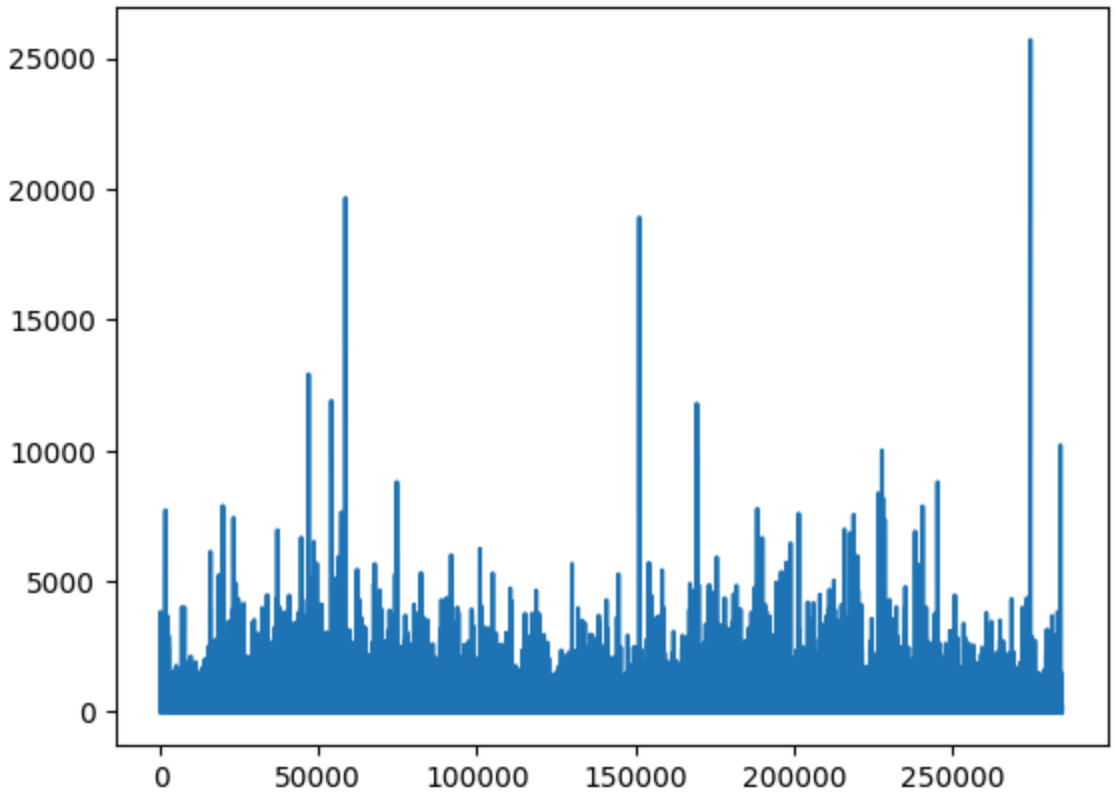
Out[12]: 1825

In [13]:
```
len(df[df['Amount'] == 25691.16])
```

Out[13]: 1

In [14]:
```
# Check whether any invalid amounts are present
df['Amount'].plot()
```

Out[14]: <Axes: >



The smallest amount is 0 and the max is 25,691.16. The largest Amount seems huge for a single credit card transaction, but since there are many it seems valid. The minimum of 0 seems unlikely as well, but it could be a charge to check a card is active. I am assuming they are valid as well.

Labels

In [15]:
```
df['Class'].unique()
```

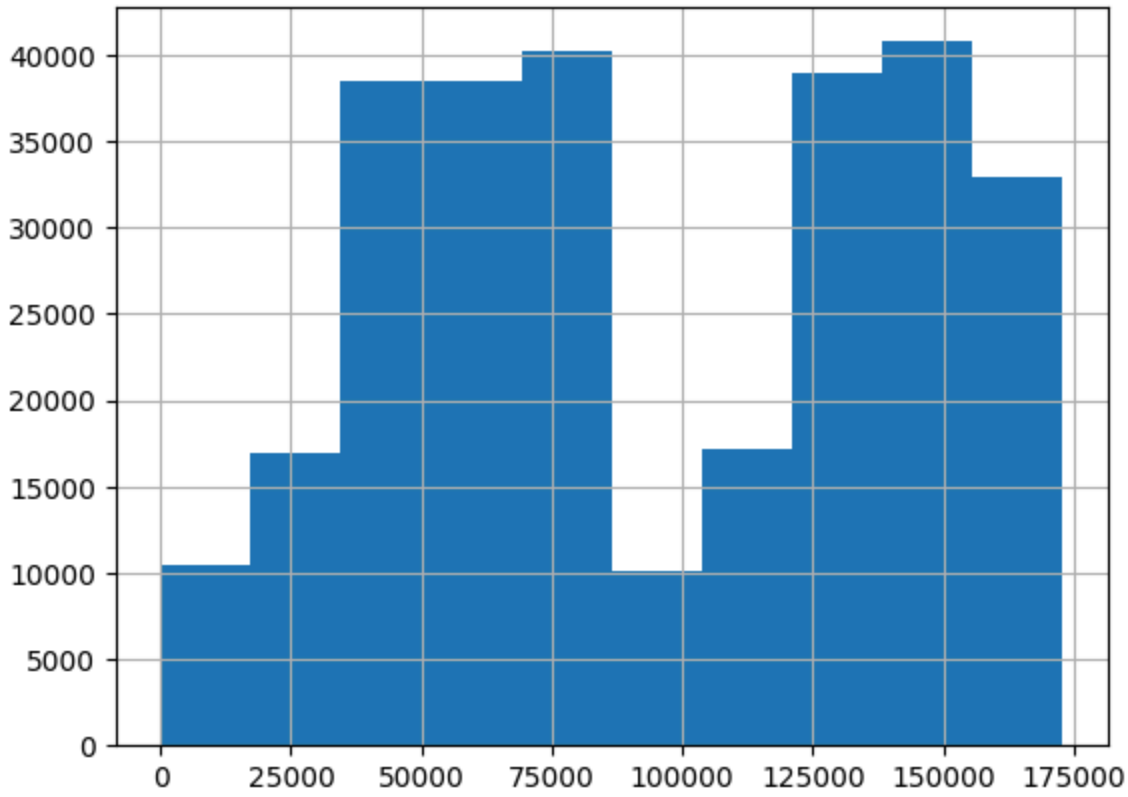
Out[15]:
```
array([0, 1])
```

The target looks good. It only has values 0 and 1.

Time

In [16]:
```
df['Time'].hist()
```

Out[16]:
```
<Axes: >
```

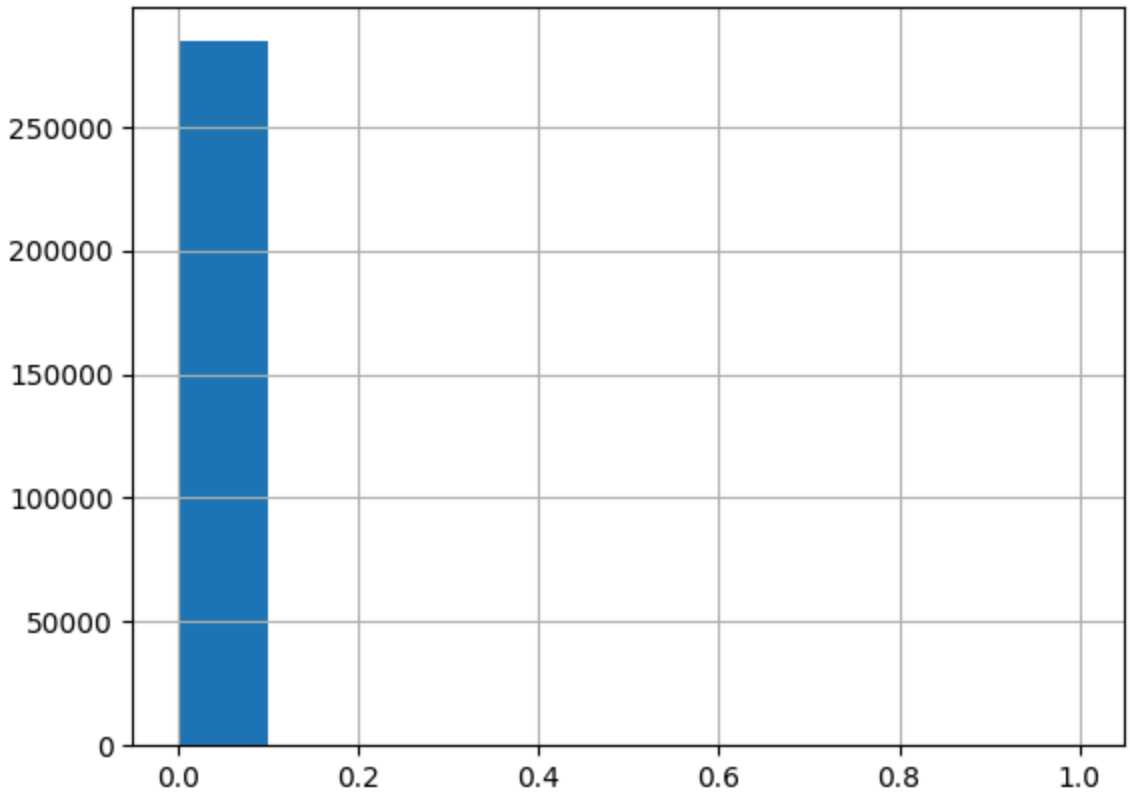


Interestingly, we can see that most of the instances are in the time range 30,000 - 85,000 and 120,000 - 175,000

Confirming Class Imbalance

In [17]:
```
df['Class'].hist()
```

Out[17]:
```
<Axes: >
```



In [18]:
```
len(df[df['Class'] == 1])
```

Out[18]: 492

It is clear there is a huge class imbalance for the target variable (as is typical for fraud datasets). We will have to consider this throughout the project.

Conclusions

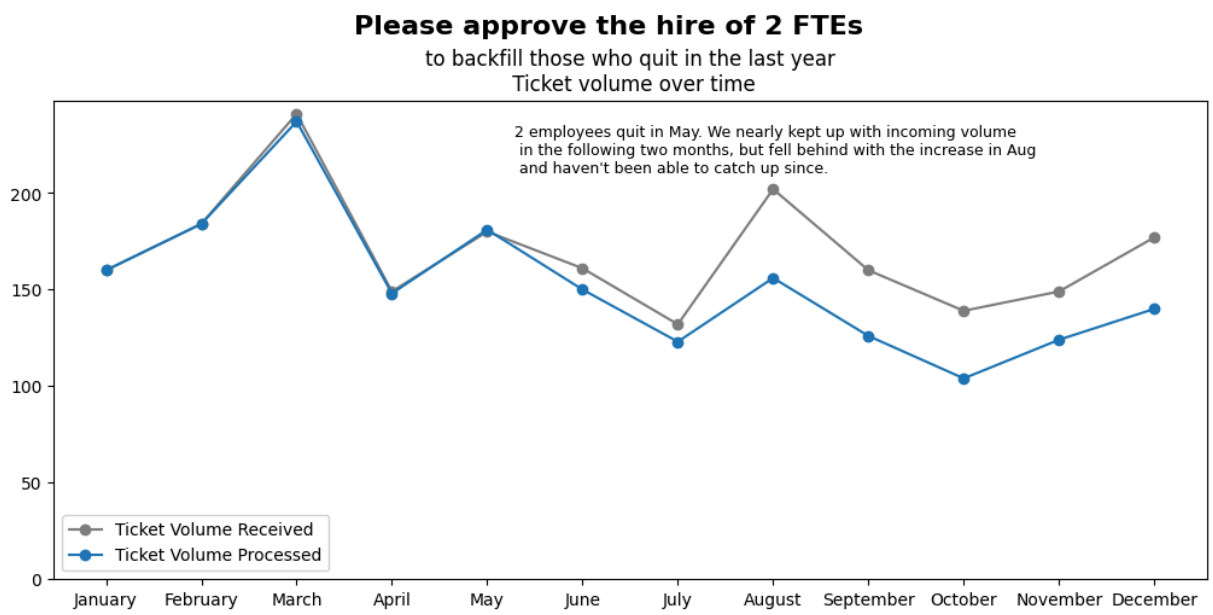
The data is mostly clean. There are duplicates in the dataset, which may need to be handled. Additionally, some of these duplicates have target value equal to 1. Since there is class imbalance for the target, we need to be careful with handling these duplicates. For the data, the transaction amounts do not have any strange or inconsistent data. Every value in this feature is numeric and positive. Additionally, all of the Time feature are positive, indicating the instances are recorded after the first record. We do see a few areas of the Time distribution which are dense, which could be interesting to explore further.

4. Storytelling With Data graph

Just like last week: choose any graph in the Introduction of Storytelling With Data (p. 1-17). Use matplotlib to reproduce it in a rough way. I don't expect you to spend an enormous amount of time on this; I understand that you likely will not have time to re-create every feature of the graph. However, if you're excited about learning to use matplotlib, this is a

good way to do that. You don't have to duplicate the exact values on the graph; just the same rough shape will be enough. If you don't feel comfortable using matplotlib yet, do the best you can and write down what you tried or what Google searches you did to find the answers.

In [61]:
```
# Figure 0.3
import matplotlib.pyplot as plt

ticket_data = {
    'Ticket Volume Received': (160, 184, 241, 149, 180, 161, 132, 202, 160, 139, 14
    'Ticket Volume Processed': (160, 184, 237, 148, 181, 150, 123, 156, 126, 104, 1
    'Month': ("January", "February", "March", "April", "May", "June", "July", "Augu
}
ticket_df = pd.DataFrame(ticket_data)
fig, ax = plt.subplots(layout='constrained', figsize=(10, 5))
plt.plot(ticket_df['Month'], ticket_df['Ticket Volume Received'], label="Ticket Vol
plt.plot(ticket_df['Month'], ticket_df['Ticket Volume Processed'], label="Ticket Vo
fig.suptitle('Please approve the hire of 2 FTEs', fontsize=16, fontweight='bold')
plt.title("to backfill those who quit in the last year\n Ticket volume over time")
ax.text(.4, .85, "2 employees quit in May. We nearly kept up with incoming volume \
plt.ylim(0)

plt.legend()
None
```



Week 3 - Univariate Analysis

Please run the cells of the notebook as you get to them while reading

In [2]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
```

1. Lesson: Bar charts and univariate graphs

Let's make a dataset (in this case, just a series) which is weighted to have more small values than large values. By squaring a random number between 0 and 1, we ensure that half (those whose initial value is below 0.5) are below 0.25, while the other half are between 0.25 and 1. This means that most of the values are small, and it's more interesting than analyzing a perfectly uniform dataset. This kind of trick - transforming one random variable to get another - can generate a variety of random datasets for you. We then multiply by 100 to get a number between 0 and 100.

In [3]:
```
np.random.seed(0)
lesson_series = np.round(np.random.random(size = 1000)**2 * 100, 2)
lesson_series[0:10] # check the first ten values.  Are they mostly on the small sid
```

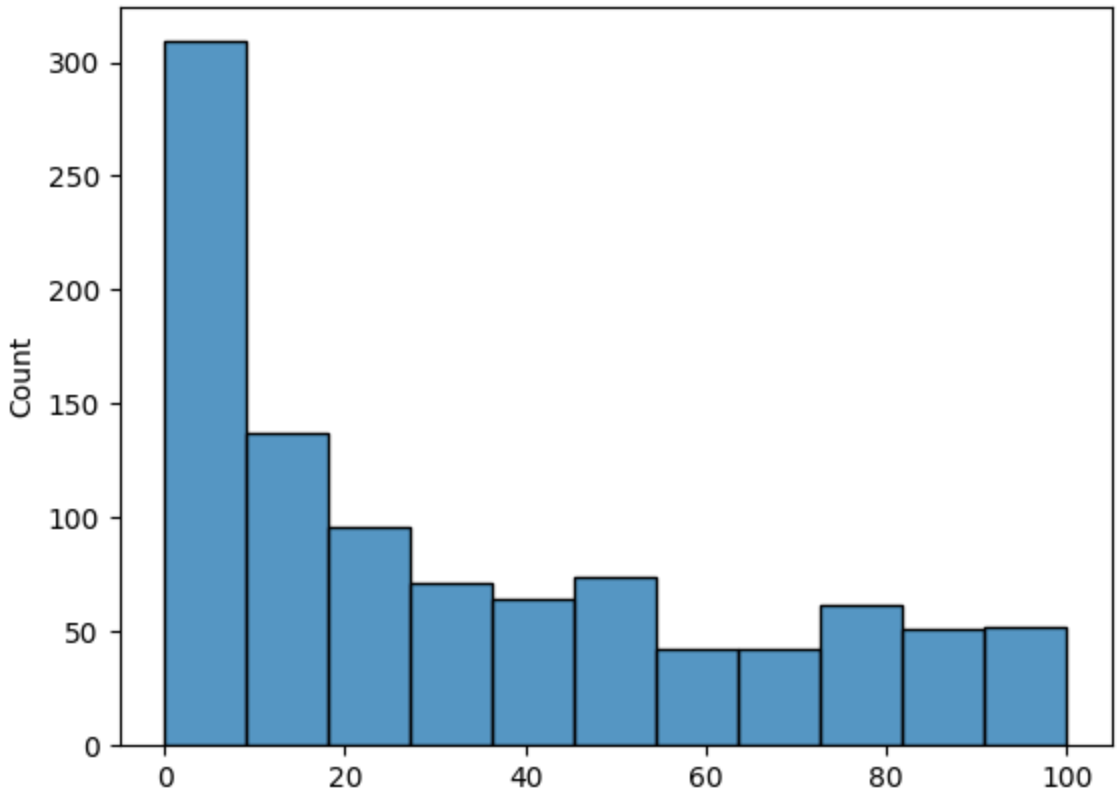
Out[3]:
```
array([30.12, 51.15, 36.33, 29.69, 17.95, 41.72, 19.15, 79.53, 92.86,
       14.7 ])
```

In [4]:
```
import seaborn as sns
```

In the plot below, you can see a histogram of the values in the series. For some reason, it decided to have exactly 11 bins (we allowed it to choose the number of bins.) Most values - about 300 of them - are between 0 and 9, and the next most likely bin is between 9 and 18. Since there are 1000 values, the total of the bars should be 1000.

In [5]:
```
sns.histplot(lesson_series)
```

Out[5]:
```
<Axes: ylabel='Count'>
```



There are 11 bins or bars, a number which by default is chosen by seaborn. We can reproduce this manually to (hopefully) see the same values as numbers. I'm not sure that this second histogram is guaranteed to be exactly the same, but it looks the same to me:

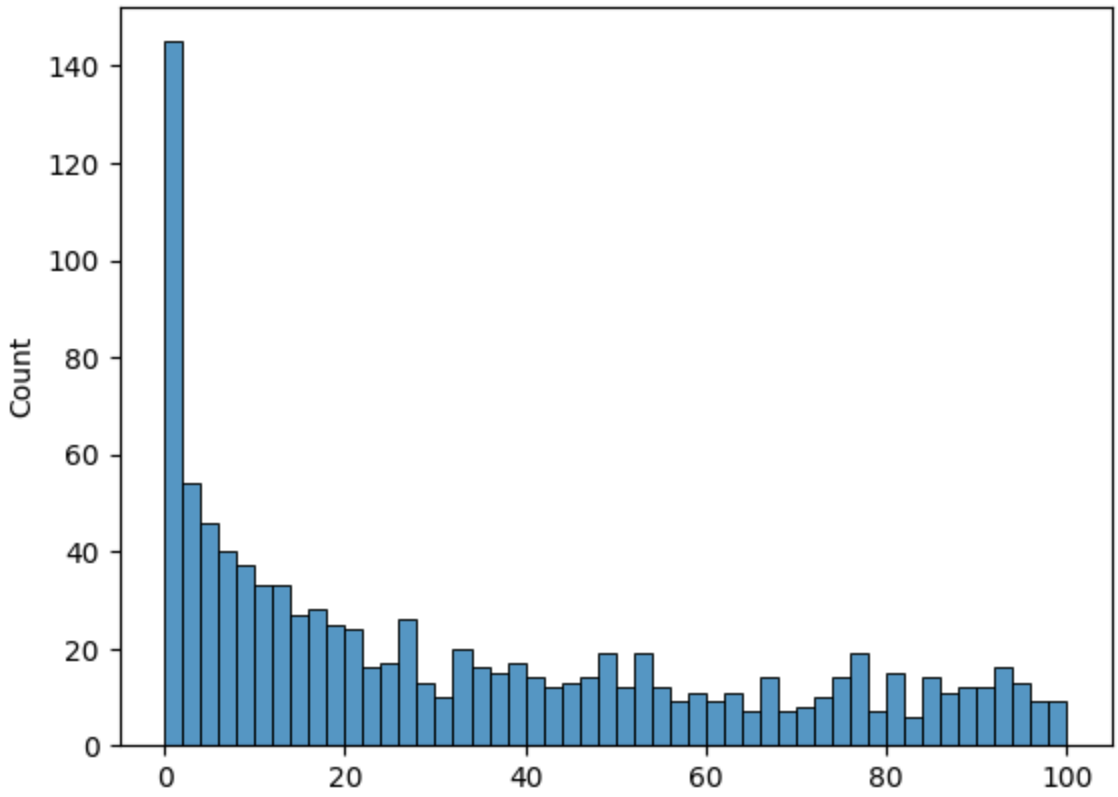
In [6]:
```
np.histogram(lesson_series, bins = 11)[0]
```

Out[6]:
```
array([309, 137,  96,  71,  64,  74,  42,  42,  62,  51,  52])
```

What happens if we override seaborn and choose the number of bins ourselves? We could choose a much larger number of bins:

In [7]:
```
sns.histplot(lesson_series, bins = 50)
```

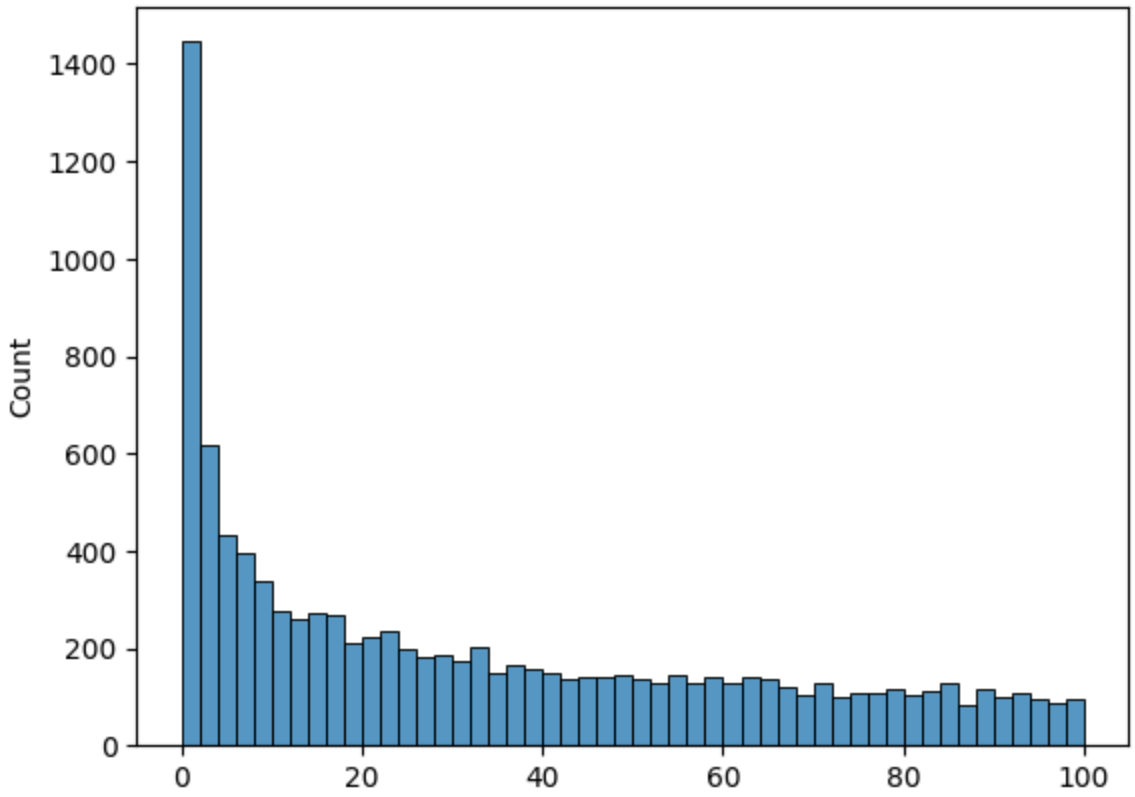
Out[7]:
```
<Axes: ylabel='Count'>
```



One disadvantage of this 50 bin picture is that the outliers are worse. That is, the graph wobbles up and down a bit more randomly. That's because there are fewer values in each bin, so there's more of a role for chance to take effect. If we had many more data points and/or fewer bins, we could get rid of this wobble.

In [8]:
```
lesson_series_2 = np.round(np.random.random(size = 10000)**2 * 100, 2)
sns.histplot(lesson_series_2, bins = 50)
```

Out[8]:
```
<Axes: ylabel='Count'>
```

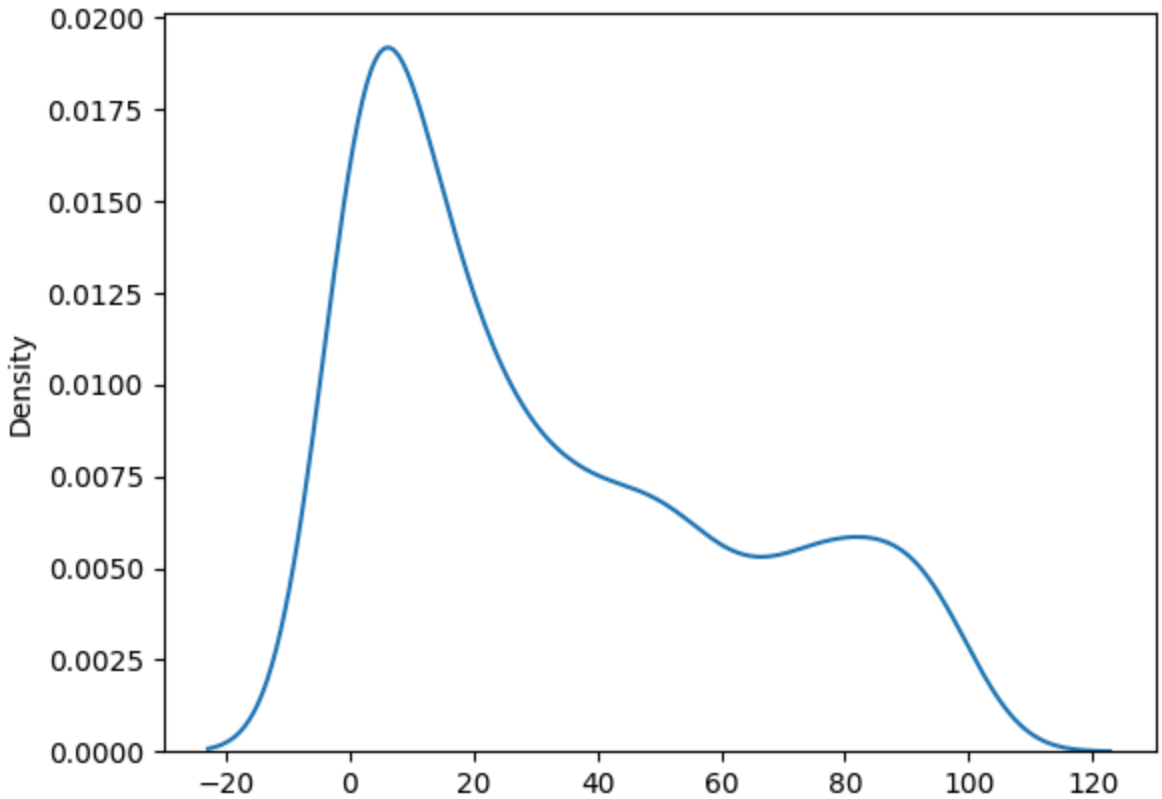


Here you can see that because the number of bins is the same as in the second graph above, but for more data, the histogram is a smoother graph. Why does more data make for a smoother graph? Something for you to think about. I said it's because a larger number of data points in each bin reduces the role of chance - but why is that?

Here is a KDE (Kernel Density Estimate) plot. It's just the same histogram, but drawn smoothly. The KDE plot doesn't have a "number of bins." It's always drawn the same way. In this case, because of the smoothness of the curve, it seems that x-values less then zero and above 100 are still plotted, even though there were no such values in the dataset. This seems like a drawback of the KDE plot, especially if the viewer is unprepared for this aspect of the plot.

In [9]:
```
sns.kdeplot(lesson_series)
```

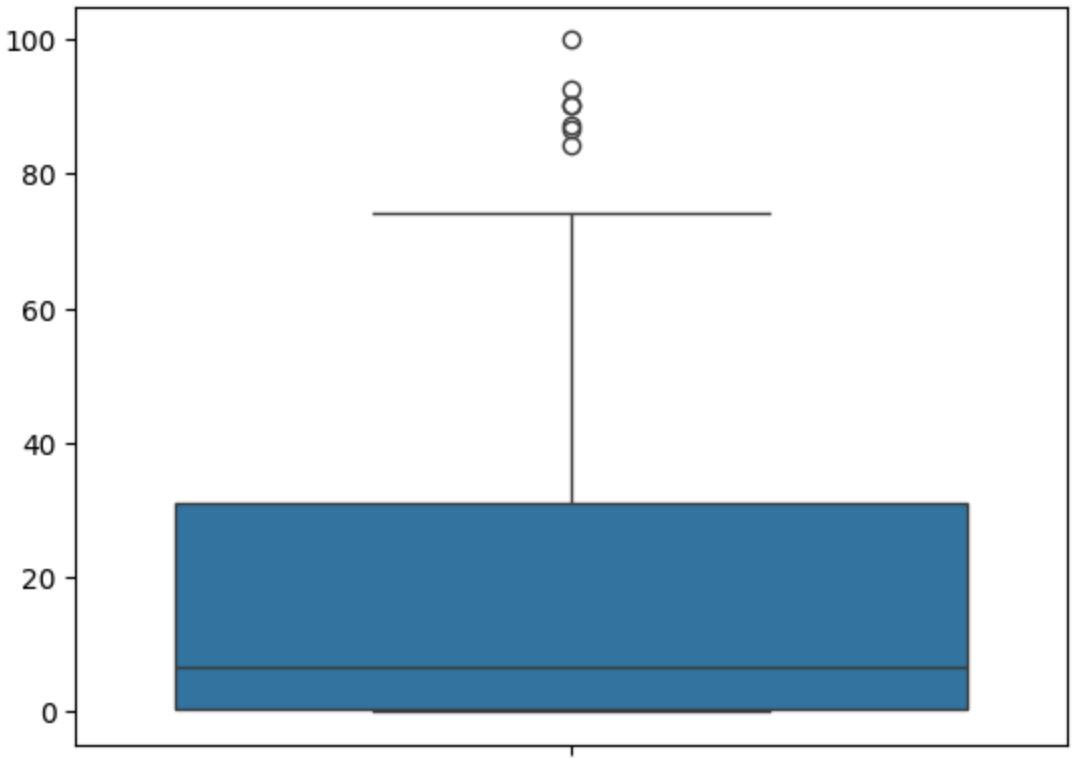
Out[9]:
```
<Axes: ylabel='Density'>
```



We could also draw a box plot. This time, to construct the data I used a fourth power rather than squaring, with only 100 data points, so that the points are even more concentrated toward the small numbers. It turns out that this will create a more interesting boxplot. The top and bottom edges of the box are the 75th and 25th percentile, respectively, and the top and bottom "whiskers" show a larger range which is a multiple of 1.5 times the the box height. (The bottom whisker cannot be see because it's pushed against the bottom of the graph.) The filled-in box shows that half of the values are between about 0 and 30 on the y-axis. Is that what you'd expect? The 25th and 75th percentile of the original uniform random variable are at 0.25 and 0.75. Taken to the fourth power and multiplied by 100 (remember, that's how we constructed our sample), that's 0.25**4 * 100 = 0.4 and 0.75**4 * 100 = 32. It's plausible that those are the height of the bottom and top of the box. We can see that a small number of samples are above the top whisker; they are shown as individual dots.

In [10]:
```
lesson_series_3 = np.round(np.random.random(size = 100)**4 * 100, 2)
sns.boxplot(lesson_series_3)
```

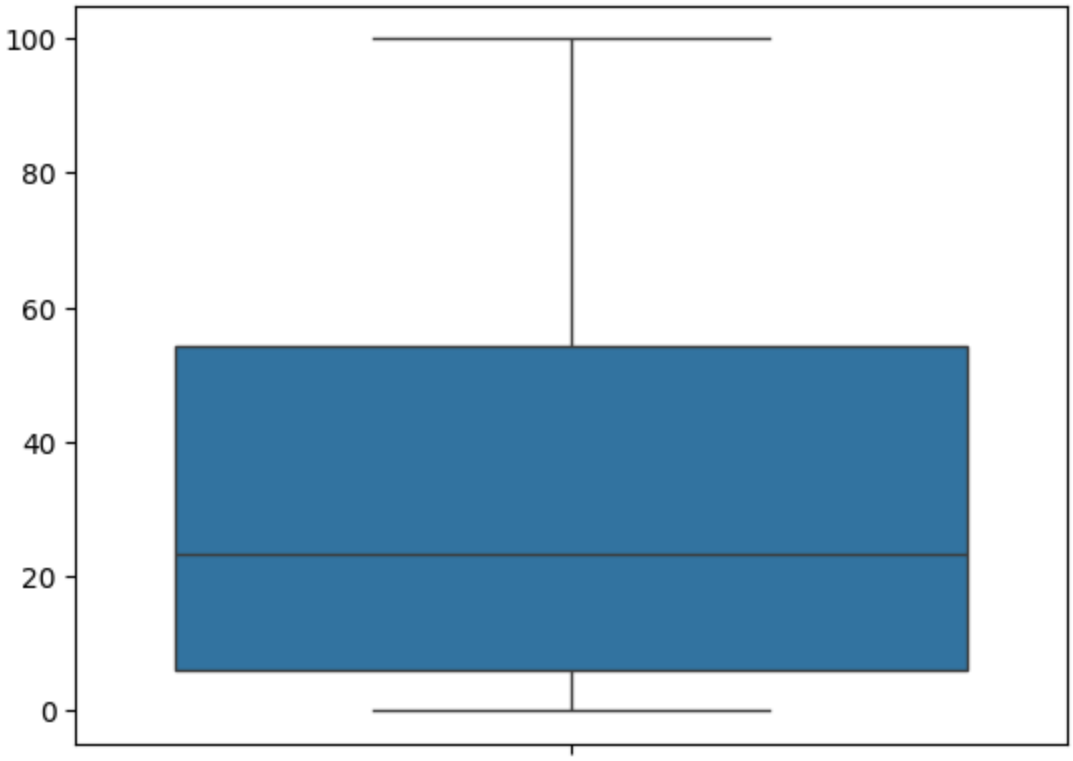
Out[10]: <Axes: >



If we go back to the original lesson_series with the squared values, there will be two whiskers, because it isn't so strongly weighted toward small values:

In [11]:
```
sns.boxplot(lesson_series)
```

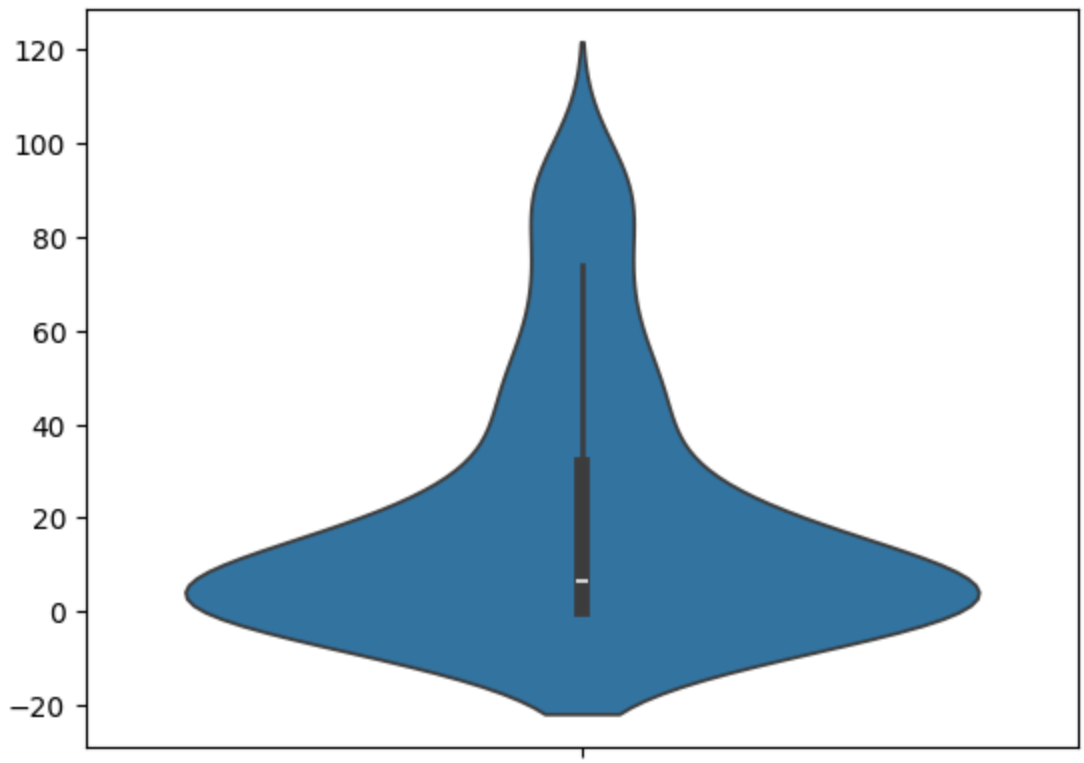
Out[11]: <Axes: >



Going back to the fourth power series, another histogram variant is the violin plot. This simply combines a kde plot (turned on its side and forming two side of the violin) with a boxplot:

In [12]:
```
sns.violinplot(lesson_series_3)
```

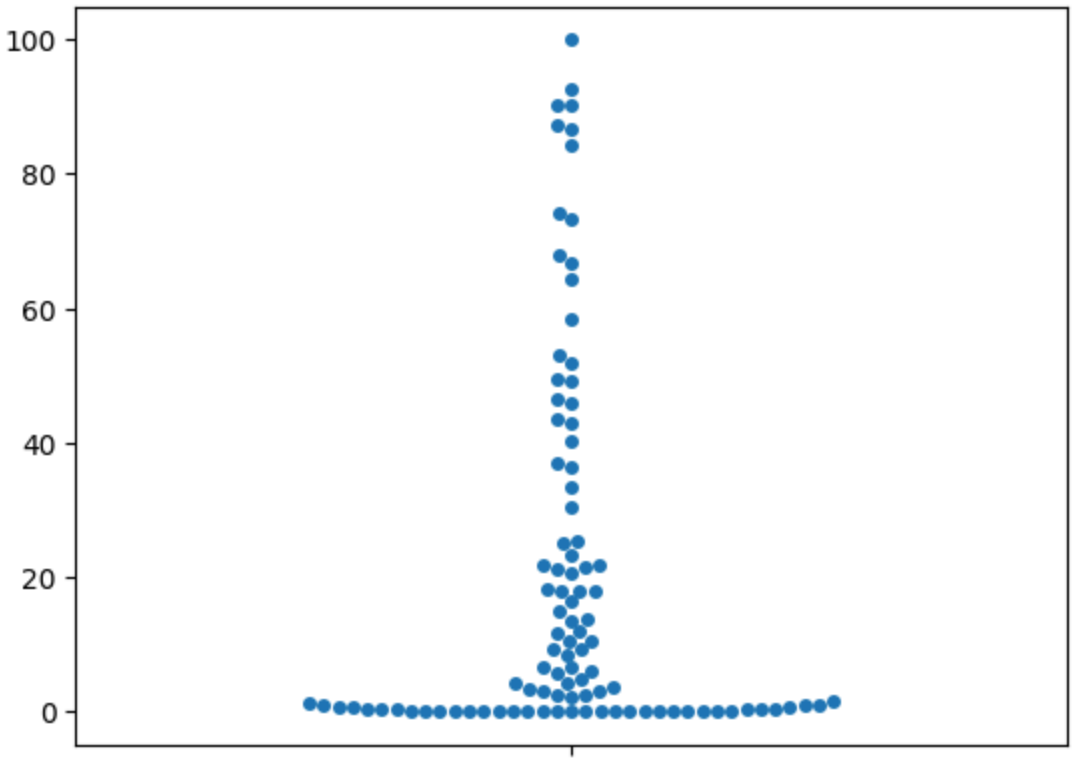
Out[12]: <Axes: >



Finally, a swarm plot shows the histogram (turned on its side and doubled, as with the violin plot) but showing each individual point.

In [13]:
```
sns.swarmplot(lesson_series_3)
```

Out[13]: <Axes: >



2. Weekly graph question

Below are a histogram and table representation of the same data. A species of bird is being analyzed, and each individual's body length in inches has been measured.

Please write a short explanation of the pros and cons of these two representations. Which would you choose? How would you modify the formatting, if at all, to make it more visually interesting, clear, or informative?

In [14]:
```
import numpy as np
import pandas as pd

np.random.seed(0)
num_data = 10000
data = np.random.normal(size = num_data) + 6
df = pd.DataFrame(data.T, columns = ["data"])
```

In [15]:
```
histnums = np.histogram(df["data"])
histcounts = histnums[0]
histmins = histnums[1][0:-1]
histmaxes = histnums[1][1:]
```

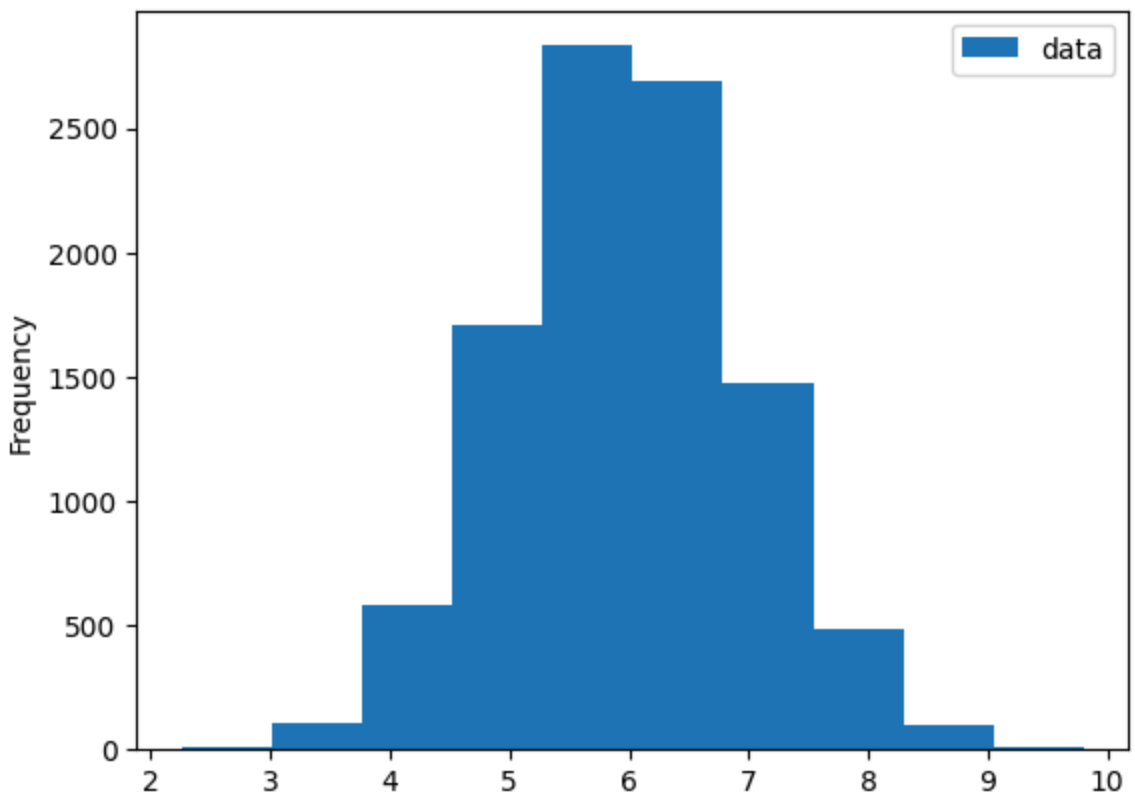
In [16]:
```
pd.DataFrame(np.array([histcounts, histmins, histmaxes]).T, columns = ["count", "mi
```

Out[16]:

	count	minval	maxval
0	10.0	2.259899	3.014075
1	110.0	3.014075	3.768252
2	579.0	3.768252	4.522428
3	1710.0	4.522428	5.276604
4	2833.0	5.276604	6.030780
5	2688.0	6.030780	6.784956
6	1479.0	6.784956	7.539132
7	487.0	7.539132	8.293308
8	97.0	8.293308	9.047484
9	7.0	9.047484	9.801660

In [17]:
```
df.plot.hist()
```

Out[17]: <Axes: ylabel='Frequency'>



The DataFrame and Plot Histogram each have benefits and disadvantages.

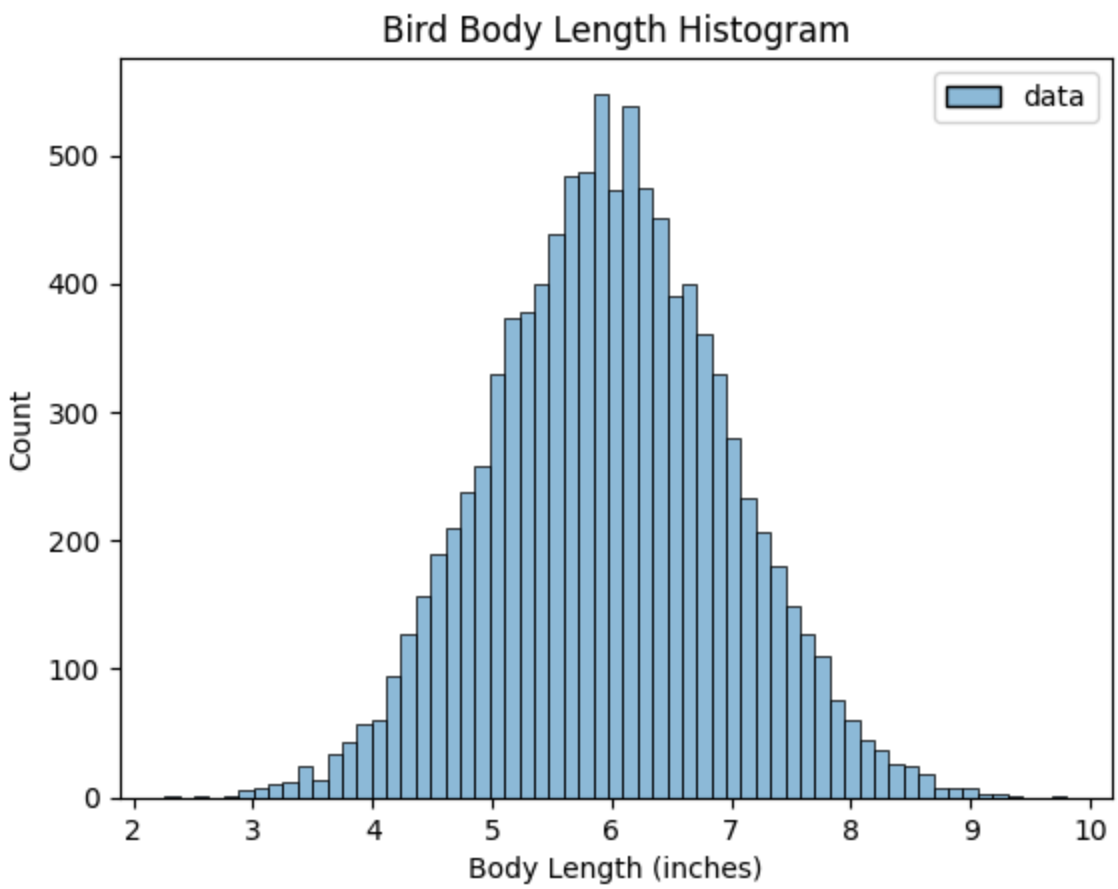
The DataFrame Histogram provides more granuarity into the data itself. You can clearly see what values each histogram bin contains, the number of bins, and the count of instances in each bin. However, the with the DataFrame it is harder to make inferences into how the

distribution is shaped. Your eyes need to dart back and forth between each row to make comparisons.

The Plot Histogram is much easier to read and visualize the distribution of the data. However, it does not contain the granuarity of what instances each bin contains to the decimal point.

I would generally utilize the histogram since it gives a more easily digestible representation of the data. A few improvements can be made to the plot, like adding a title, labeling the X axis, and making the legend more clear. Additionally, by increasing the number of bins we get a better idea of the distribution. I have made those changes below..

In [18]:
```
import matplotlib.pyplot as plt
sns.histplot(
    df
)
plt.title("Bird Body Length Histogram")
plt.xlabel("Body Length (inches)")
None
```



3. Homework - Amusement Park Rides

Now let's imagine we have some data about how many times different visitors to an amusement park used each ride, as well as how much money they spend at the amusement park. Each sample represents a single visit by a single visitor on a given date.

In [19]:
```
num_visits = 10000
np.random.seed(0)
```

In [20]:
```
df = pd.DataFrame(columns = ["VisitDate"])
```

In [21]:
```
start = datetime(2010, 1, 1)
end = datetime(2024, 1, 1)
numdays = (end - start).days
random_days = np.random.randint(0, numdays, size = num_visits)
s = start + pd.to_timedelta(random_days, unit='D')
s = s.sort_values()
df["VisitDate"] = s
```

In [22]:
```
df["IsAdult"] = np.random.choice([True, True, False], size = num_visits)
```

In [23]:
```
df["MartianRide"] = np.random.choice([0] * 8 + [1] * 3 + [2] * 3 + [3] * 1 + [10],
```

In [24]:
```
df["TeacupRide"] = np.random.choice([0] * 2 + [1] * 5 + [2] * 3 + [5] * 2, size = n
```

In [25]:
```
df["RiverRide"] = np.random.choice([0] * 8 + [1] * 3 + [2] * 2, size = num_visits)
```

In [26]:
```
df["MoneySpent"] = np.round(np.random.random(size = num_visits)**2 * 100, 2)
```

In [27]:
```
df.iloc[0:5]
```

Out[27]:

	VisitDate	IsAdult	MartianRide	TeacupRide	RiverRide	MoneySpent
0	2010-01-01	False	0	1	4	10.30
1	2010-01-01	True	3	0	0	38.77
2	2010-01-01	True	0	0	0	79.34
3	2010-01-01	True	0	0	1	87.84
4	2010-01-02	False	0	1	2	18.65

1. Find the mean, median, and mode for how many times visitors rode each ride. See Week 1 for Google advice on this.

In [28]:
```
def mode(x):
    return x.mode().iloc[0] if not x.mode().empty else None

ride_cols = ["MartianRide", "TeacupRide", "RiverRide"]

df[ride_cols].agg(['mean', 'median', mode])
```

Out[28]:

	MartianRide	TeacupRide	RiverRide
mean	0.9073	0.5862	1.2007
median	0.0000	0.0000	1.0000
mode	0.0000	0.0000	0.0000

2. Use groupby() to find the mean, median, and mode for how many times each ride was ridden on each given day.

Suggested Google search or ChatGPT prompt: I first tried: "How do I use groupby to find the mean over each day in my DataFrame?" but Google didn't help me.

Then I tried: "How do I find the mean over each date in a dataframe?" Sometimes, you have to try multiple searches.

This gives me a very helpful site, where someone is doing the same thing we are:

https://stackoverflow.com/questions/40788530/how-to-calculate-mean-of-some-rows-for-each-given-date-in-a-dataframe

In [29]:
```
df.groupby(by='VisitDate')[ride_cols].agg(['mean', 'median', mode])
```

Out[29]:

	MartianRide			TeacupRide			RiverRide		
	mean	**median**	**mode**	**mean**	**median**	**mode**	**mean**	**median**	**mode**
VisitDate									
2010-01-01	0.75	0.0	0	0.25	0.0	0	1.25	0.5	0
2010-01-02	0.00	0.0	0	1.00	1.0	1	2.00	2.0	2
2010-01-03	0.00	0.0	0	1.50	0.5	0	1.25	0.5	0
2010-01-04	0.00	0.0	0	0.00	0.0	0	1.00	1.0	1
2010-01-05	0.50	0.5	0	0.00	0.0	0	0.00	0.0	0
...	...	...	...	...	...	...	...	...	...
2023-12-27	0.50	0.5	0	0.50	0.5	0	1.00	1.0	0
2023-12-28	6.00	6.0	2	0.00	0.0	0	0.00	0.0	0
2023-12-29	0.00	0.0	0	1.00	1.0	1	3.00	3.0	3
2023-12-30	3.00	3.0	3	0.00	0.0	0	1.00	1.0	1
2023-12-31	0.00	0.0	0	0.00	0.0	0	1.00	1.0	1

4404 rows × 9 columns

3. Find the standard deviation and variance of the count for each ride.

Suggested Google search or ChatGPT prompt: "How do I find the standard deviation of a Series in pandas documentation?" and similar query for variance.

I included the word "documentation" because this task likely involves applying one single function for standard deviation and another for variance, so I expected that I could find a single documentation page for each that would cover my needs. I get these:

https://pandas.pydata.org/pandas-docs/dev/reference/api/pandas.Series.var.html

https://pandas.pydata.org/docs/reference/api/pandas.Series.std.html

In [30]:
```
df[ride_cols].agg(['std', 'var'])
```

Out[30]:

	MartianRide	TeacupRide	RiverRide
std	2.077339	1.232851	1.295757
var	4.315338	1.519922	1.678987

4. Find the 90th percentile count for each ride. That is, if the customers are ordered by their number of rides, and there are 100 customers, how many rides does the 90th person take? There is a function in pandas that easily does this calculation.

Suggested Google search or ChatGPT prompt: "How do I find a percentile value for a Series in pandas documentation?"

This time, I will let you find the page! Hint: it's not actually called the "percentile" function.

In [31]:
```
df[ride_cols].quantile(0.9)
```

Out[31]:
```
MartianRide    2.0
TeacupRide     2.0
RiverRide      3.0
Name: 0.9, dtype: float64
```

5. Plot a histogram of the ride count, using each day as data element as you did in #2. Use both the total ride count for each day as well as, separately, the mean ride count for each day.

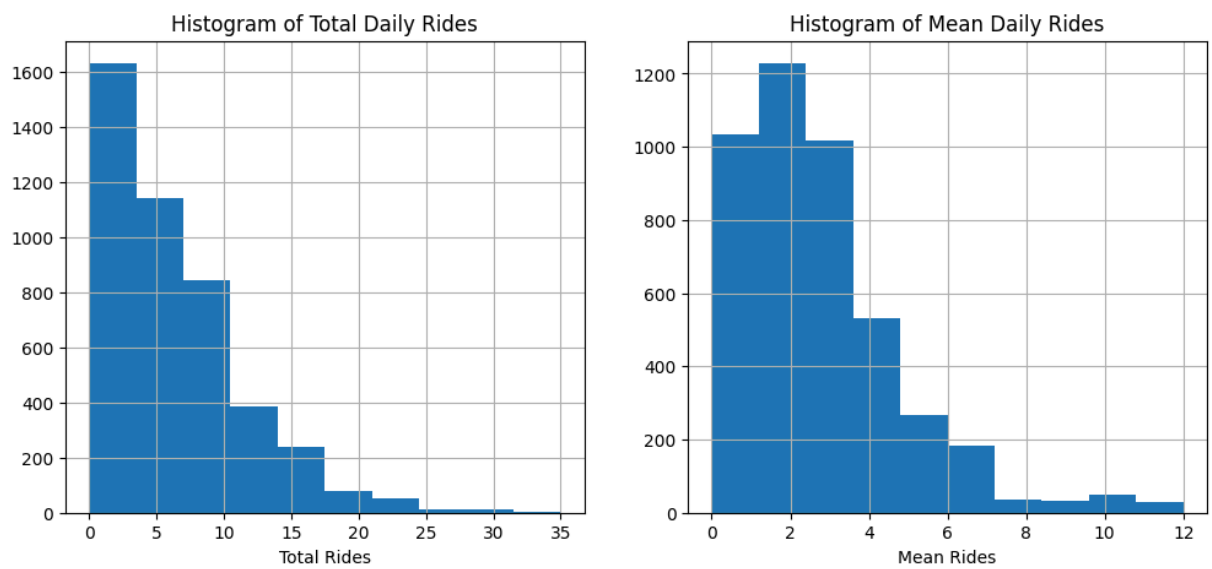
Suggested Google search or ChatGPT prompt: "How do I plot a histogram for a Series in pandas?"

I found this: https://pandas.pydata.org/docs/reference/api/pandas.Series.plot.hist.html

Note: the "by" keyword will not help you plot a count for each day; if you try to use it for this, it will try to draw a separate histogram for each day.

You've already learned how to find the mean ride count for each day. Can you use a similar idea to find the total ride count for each day?

In [32]:
```
df_copy = df.copy()
df_copy['TotalRideCount'] = df_copy['MartianRide'] + df_copy['TeacupRide'] + df_cop

total_ride_stats = df_copy.groupby('VisitDate')['TotalRideCount'].agg(['sum', 'mean

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

total_ride_stats['sum'].hist(ax=ax1)
ax1.set_title('Histogram of Total Daily Rides')
ax1.set_xlabel('Total Rides')

total_ride_stats['mean'].hist(ax=ax2)
ax2.set_title('Histogram of Mean Daily Rides')
ax2.set_xlabel('Mean Rides')

None
```



6. Make a bar chart showing the total number of Adult and Child participants.

Try to formulate your own Google search or ChatGPT prompt.

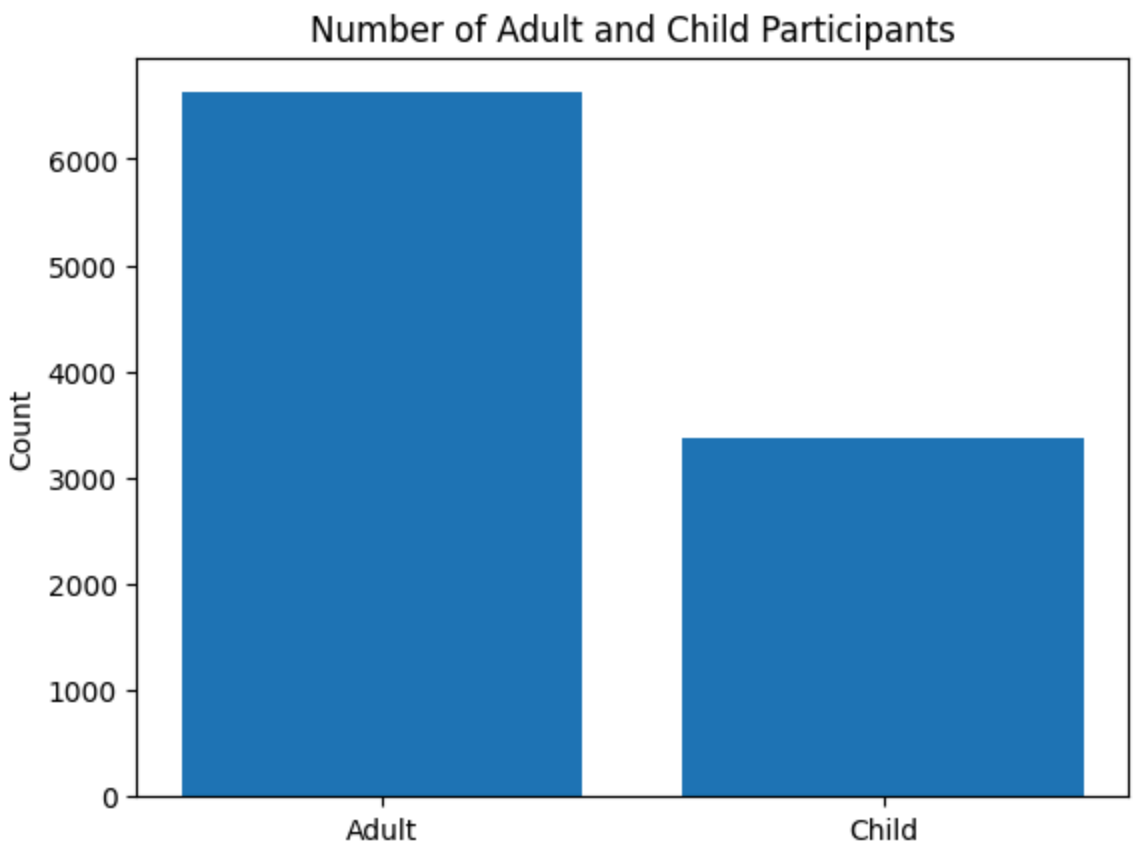
Here's one page I found:

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.bar.html

which shows some interesting examples for you to use. I recommend clicking on the first, basic example to see how to use the function.

You can also look up the pandas documentation, which might lead to an entirely different method.

In [33]:
```
num_adults = len(df[df['IsAdult'] == True])
num_children = len(df[df['IsAdult'] == False])
assert num_adults + num_children == len(df)
plt.bar(x=['Adult', 'Child'], height=[num_adults,num_children])
plt.title("Number of Adult and Child Participants")
plt.ylabel("Count")
None
```



7. Make a stacked bar chart showing how many Adult and Child visits took the River Ride, with the x-axis showing the number of rides and with two stacked bars (adult, child). That is, there could be a column for the number of Adults (and Children) who took 0 rides, 1 ride, 2 rides, and so on.

Try to formulate your own Google search or ChatGPT prompt.

For me, the pandas documentation was easier than the matplotlib documentation.

- If you use the pandas method, you might have to one-hot encode the IsAdult column. You might want names for the new columns other than True and False. Use df.join or pd.concat to attach the pd.get_dummies table you've created, and then groupby the RiverRide value.

- You will need to end up with a small table with two columns (IsAdult True and False) and an index (RiverRide) as well as a small number of rows (the values of RiverRide)
- That said, if you can come up with another approach, it's fine.

In [34]:
```
is_adult_one_hot = pd.get_dummies(df['IsAdult'], prefix='IsAdult', dtype=int)
is_adult_one_hot
```

Out[34]:

	IsAdult_False	IsAdult_True
0	1	0
1	0	1
2	0	1
3	0	1
4	1	0
...	...	...
9995	0	1
9996	0	1
9997	1	0
9998	0	1
9999	0	1

10000 rows × 2 columns

In [35]:
```
p7_df = df.copy().join(is_adult_one_hot).drop(columns=['IsAdult', 'MartianRide', 'T
p7_df
```

Out[35]:

	RiverRide	IsAdult_False	IsAdult_True
0	4	1	0
1	0	0	1
2	0	0	1
3	1	0	1
4	2	1	0
...	...	...	...
9995	0	0	1
9996	0	0	1
9997	3	1	0
9998	1	0	1
9999	1	0	1

10000 rows × 3 columns

In [36]:
```
adult_rides = p7_df[p7_df['IsAdult_True'] == 1]
adult_rides
```

Out[36]:

	RiverRide	IsAdult_False	IsAdult_True
1	0	0	1
2	0	0	1
3	1	0	1
6	0	0	1
7	0	0	1
...	...	...	...
9993	0	0	1
9995	0	0	1
9996	0	0	1
9998	1	0	1
9999	1	0	1

6625 rows × 3 columns

In [37]:
```
child_rides = p7_df[p7_df['IsAdult_False'] == 1]
child_rides
```

Out[37]:

	RiverRide	IsAdult_False	IsAdult_True
0	4	1	0
4	2	1	0
5	4	1	0
8	1	1	0
12	3	1	0
...	...	...	...
9971	3	1	0
9975	1	1	0
9981	1	1	0
9994	2	1	0
9997	3	1	0

3375 rows × 3 columns

In [38]:
```
child_rides_by_count = child_rides.groupby(by='RiverRide').sum()
child_rides_by_count
```

Out[38]:

RiverRide	IsAdult_False	IsAdult_True
1	841	0
2	844	0
3	798	0
4	892	0

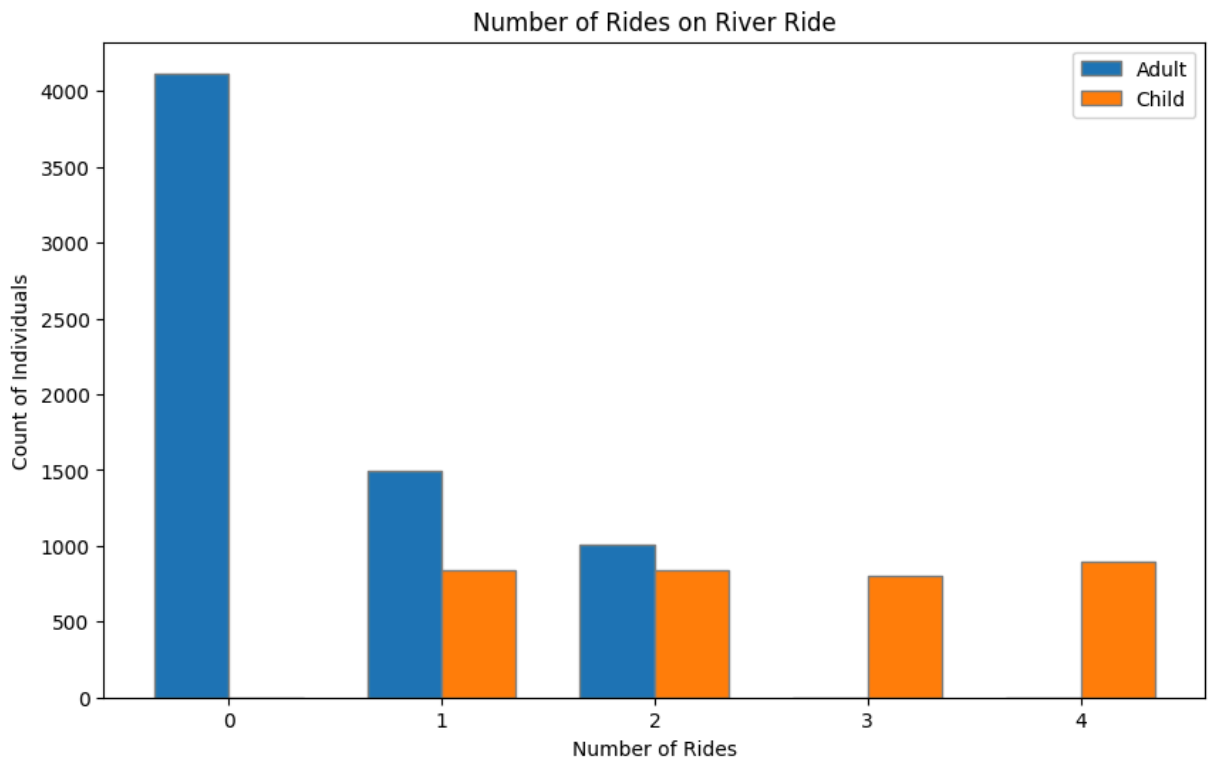
In [39]:
```
adult_rides_by_count = adult_rides.groupby(by='RiverRide').sum()
adult_rides_by_count
```

Out[39]:

RiverRide	IsAdult_False	IsAdult_True
0	0	4118
1	0	1498
2	0	1009

In [40]:
```
labels = ["0", "1", "2", "3", "4"]

# Adult Rides have values for ride count [0,1,2]. Child have values for [1,2,3,4].
```

```
adult_ride_counts = np.append(adult_rides_by_count['IsAdult_True'], [0,0])
child_ride_counts = np.append([0], child_rides_by_count['IsAdult_False'])

bar_width = 0.35
# Set position of bars on x-axis
r1 = np.arange(len(labels))
r2 = [x + bar_width for x in r1]

plt.figure(figsize=(10, 6))
plt.bar(r1, adult_ride_counts, width=bar_width, edgecolor='grey', label='Adult')
plt.bar(r2, child_ride_counts, width=bar_width, edgecolor='grey', label='Child')
plt.title("Number of Rides on River Ride")
plt.xlabel("Number of Rides")
plt.ylabel("Count of Individuals")
plt.xticks([r + bar_width/2 for r in r1], labels) # Center x-axis labels
plt.legend()
None
```



8. Other tasks

Use seaborn (import seaborn as sns) to create a kernel density estimation (kde) plot. (You can choose any aspect of the amusement park dataset to plot.) Here is a tutorial you can read about seaborn:

https://seaborn.pydata.org/tutorial/introduction.html

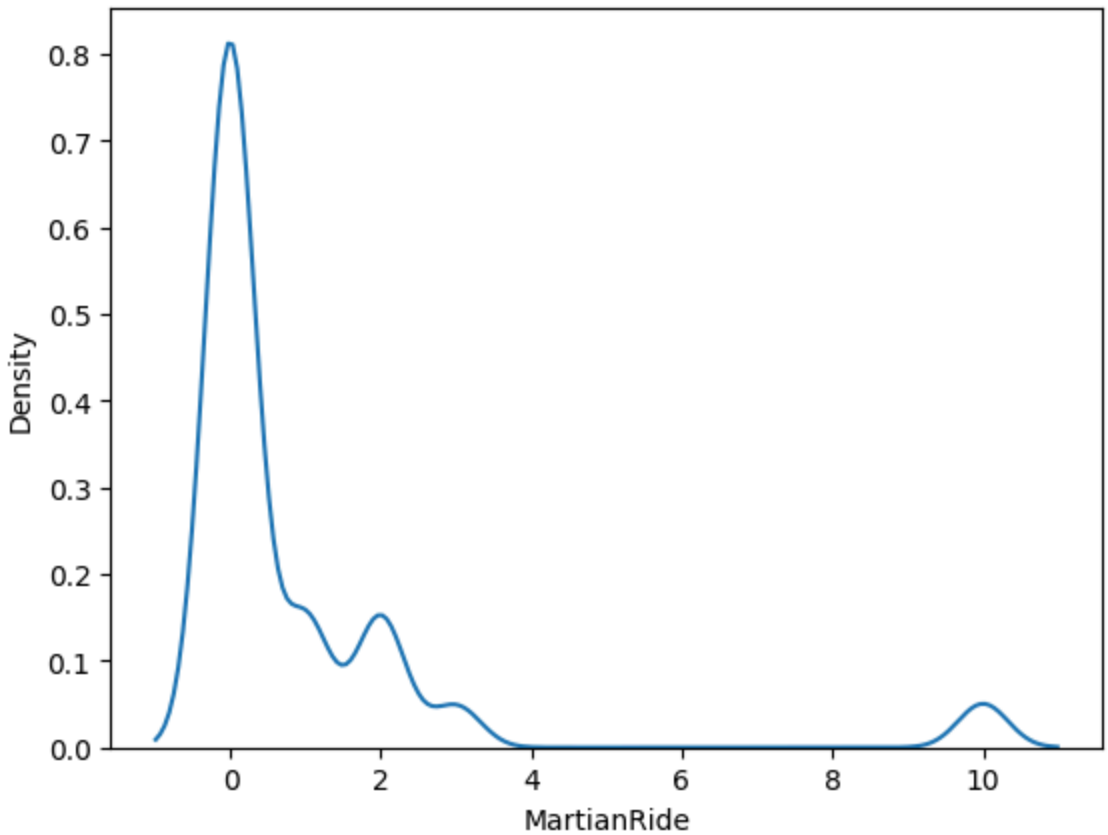
Seaborn often makes nicer looking graphs than pandas or matplotlib.

Now use seaborn to make a swarm plot, a violin plot, and a box plot.

Which plots are the best for showing this data?

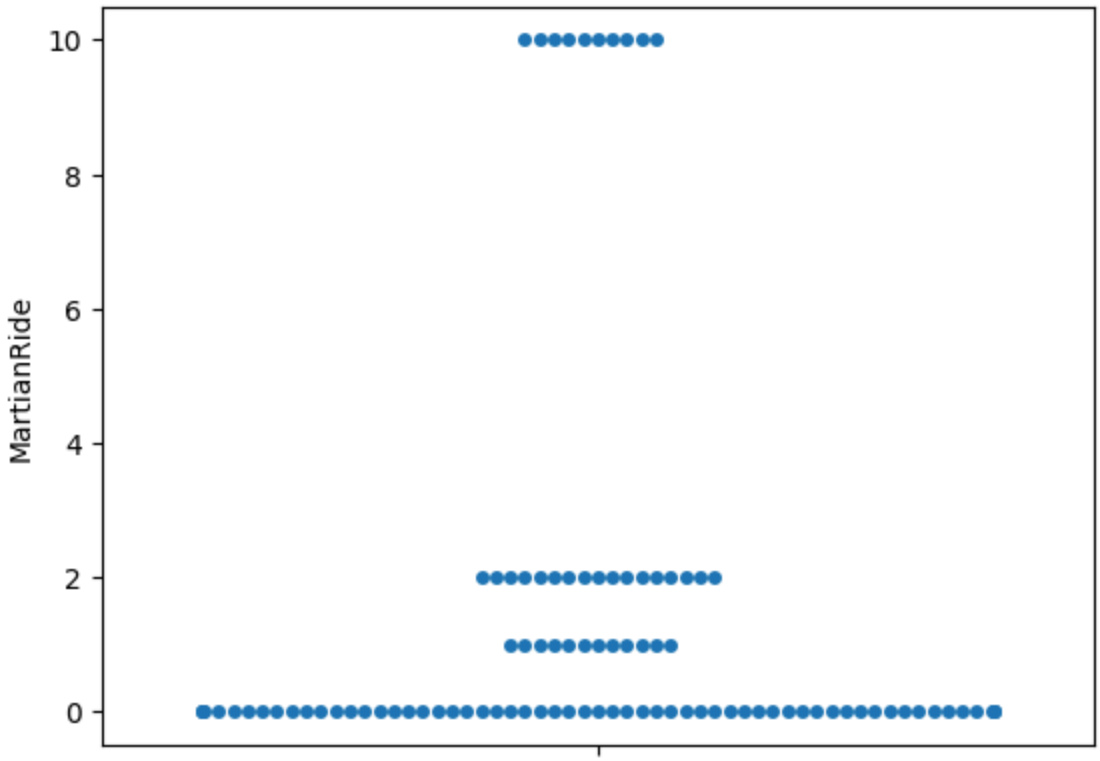
In [41]:
```
sns.kdeplot(df['MartianRide'])
```

Out[41]:
```
<Axes: xlabel='MartianRide', ylabel='Density'>
```

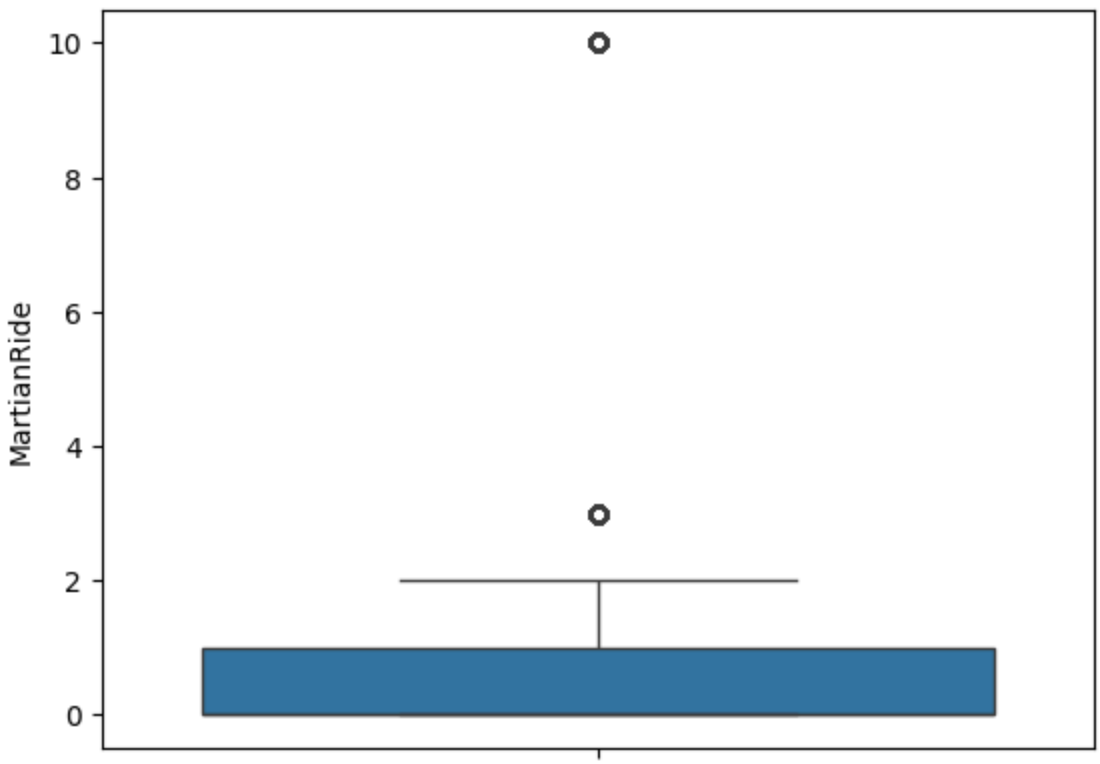


In [42]:
```
sns.swarmplot(df.sample(100)['MartianRide'])
None
```

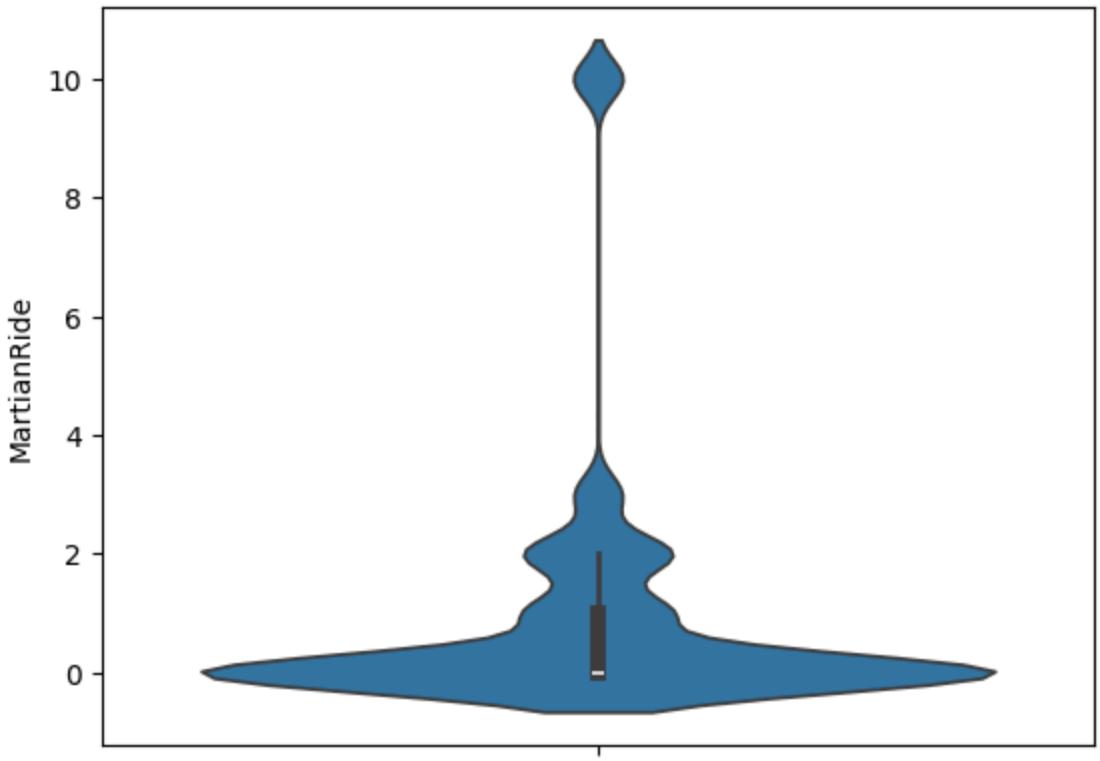
```
c:\Users\caleb\anaconda3\envs\iris-fisher\lib\site-packages\seaborn\categorical.py:3
399: UserWarning: 6.0% of the points cannot be placed; you may want to decrease the
size of the markers or use stripplot.
  warnings.warn(msg, UserWarning)
```



In [43]:
```
sns.boxplot(df['MartianRide'])
None
```



In [44]:
```
sns.violinplot(df['MartianRide'])
None
```



4. Storytelling With Data graph

Try to make a scatterplot using amusement park dataset that is similar to the one on page 45, where the two axes are the Teacup Ride count and the River Ride count. If you want to see individual dots and not a dense swarm of dots, you'll have to cut the dataset down to a small number of points (say, 100 points or so). You can remove the other points, for instance, and focus on the first 100 points.

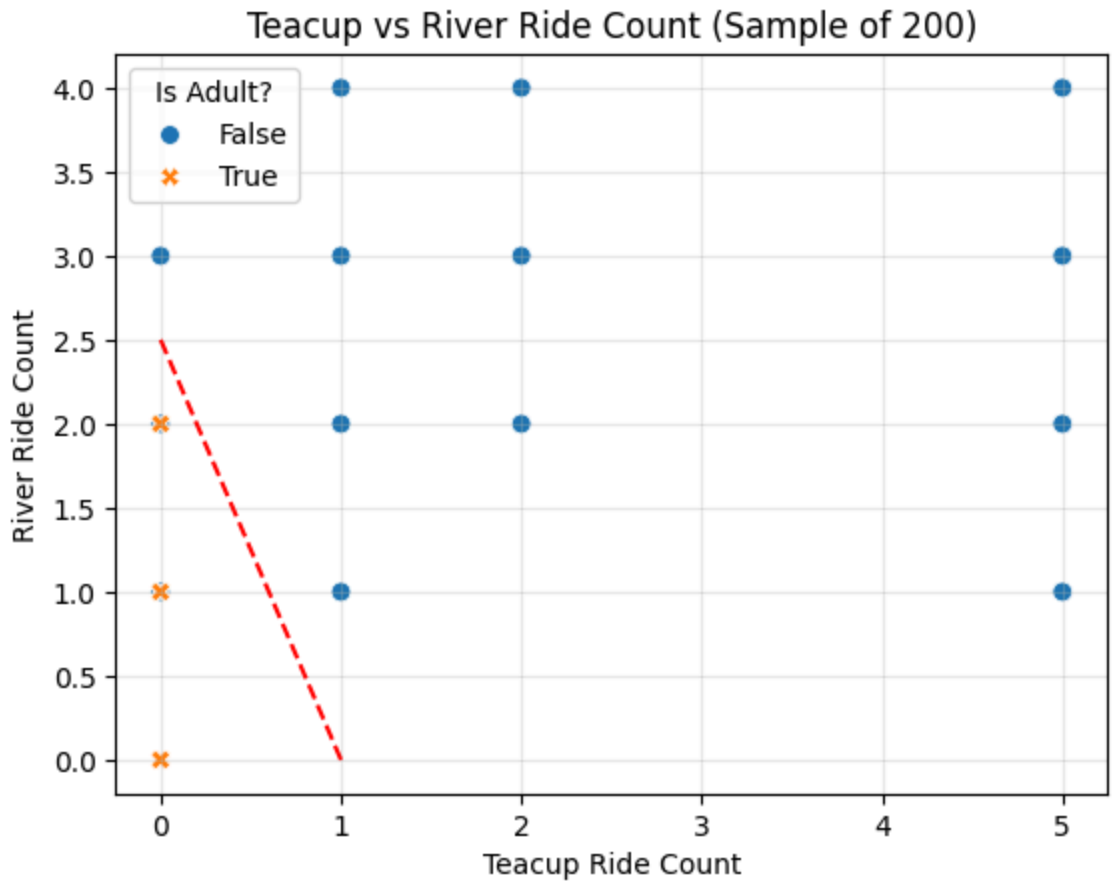
Here are some things you could do (you don't have to do all of them):

- Draw a dashed line that roughly separates the adult from child points.
- Draw the points in the adult vs. child region of the graph in different colors.
- Choose the right size and number of dots to make the graph look good.
- Write a word that appears on the dashed line (like AVG in the plot on page 45).

If there are any other graphs in the Storytelling With Data chapter that look interesting, and you want more practice, you can try to reproduce them too.

In [82]:
```
sample = df.sample(200)

sns.scatterplot(data=sample, x='TeacupRide', y='RiverRide', hue='IsAdult', style='I
plt.plot([0, 1], [2.5, 0], color='red', linestyle='--')
plt.title('Teacup vs River Ride Count (Sample of 200)')
plt.xlabel('Teacup Ride Count')
```

```
plt.ylabel('River Ride Count')
plt.legend(title='Is Adult?')
plt.grid(True, alpha=0.3)
```



Week 4 - Univariate Analysis, part 2

1. Lesson - None

2. Weekly graph question

Below are a histogram and boxplot representation of the same data. A pharmacy is keeping a record of the prices of the drugs that it sells, and an administrator wants to know how much the more expensive drugs tend to cost, in the context of the other prices.

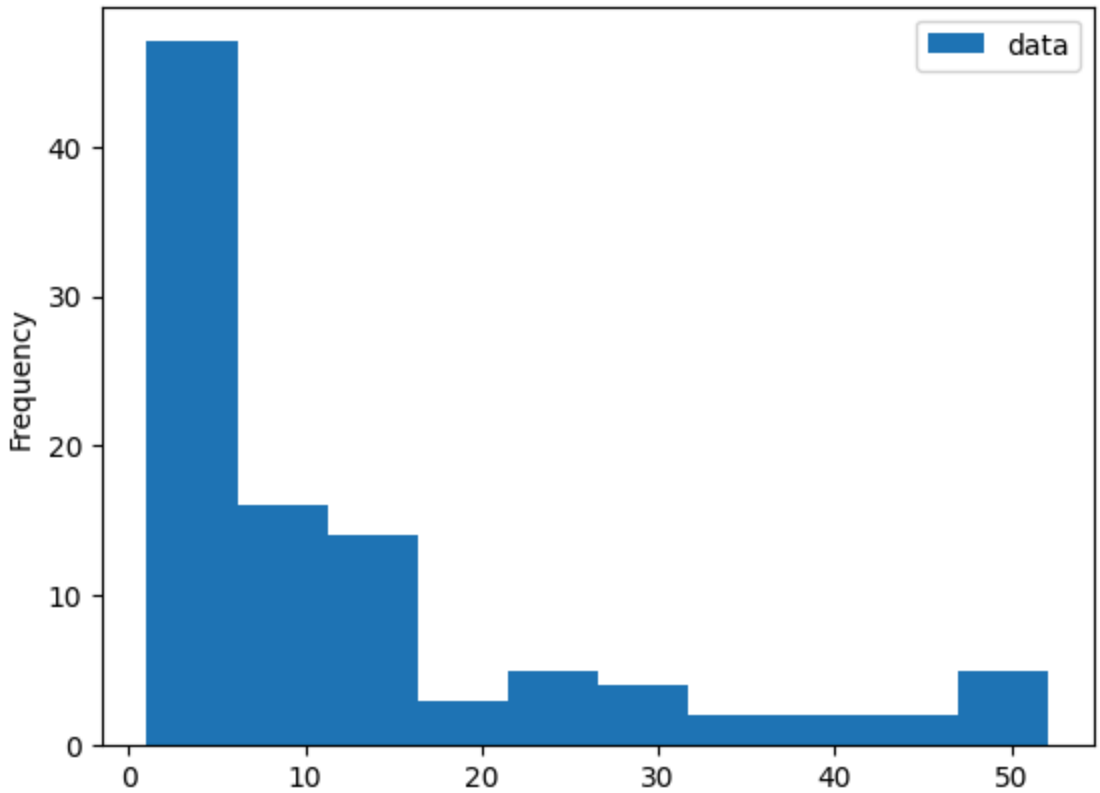
Please write a short explanation of the pros and cons of these two representations. Which would you choose? How would you modify the formatting, if at all, to make it more visually interesting, clear, or informative?

In [75]:
```
import numpy as np
import pandas as pd

np.random.seed(0)
num_data = 100
data = np.exp(np.random.uniform(size = num_data) * 4)
df = pd.DataFrame(data.T, columns = ["data"])
```

In [76]:
```
print("The 75th percentile is:", df.quantile(q = 0.75))
df.plot.hist()
```

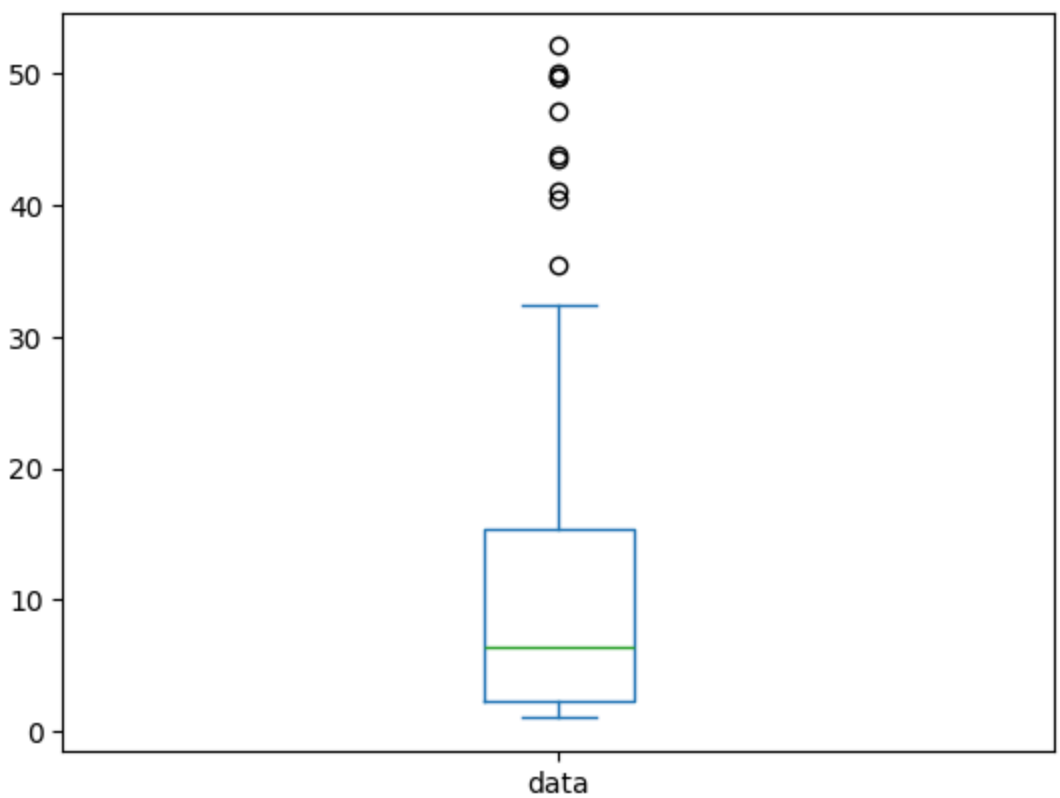
```
The 75th percentile is: data    15.457656
Name: 0.75, dtype: float64
```

Out[76]:
```
<Axes: ylabel='Frequency'>
```



In [77]:
```
df.plot.box()
```

Out[77]: <Axes: >



The histogram and box plot both make it possible to see the distribution of a feature

The histogram makes it easy to visualize the distribution. The reader can easily scan the bins to see which price ranges for the drug are most common, and what the low/high end prices are. The disadvantage of the histogram is you cannot easily identify metrics like the median or average price.

The boxplot also shows the distribution, but it additionally clearly displays what the outlier prices are. We can see the prices above 35 are outliers. It also shows the quartiles for the distributions, so we know the median is around 7, the upper quartile is around 15, and the lower quartile is around 3. It also shows the whiskers are around 2 and 35.

I would utilize the histogram for the administrator's inquiry. The histogram is easier to read for a business partner, and we can show given a the current price where it would fall on the distribution. I would add some additional detail like a title, X axis label, and if the user was interested in a metric like mean, we could add a line on the graph indicating where that falls.

3. Homework - working on your datasets

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do:

- Draw histograms and histogram variants for each feature or column. (Swarm plot, kde plot, violin plot).
- Draw grouped histograms. For instance, if you have tree heights for both maple and oak trees, you could draw histograms for both.
- Draw a bar plot to indicate total counts of each categorical variable in a given column.
- Find means, medians, and modes.

Conclusions:

- Explain what conclusions you would draw from this analysis: are the data what you expect? Are the data likely to be usable? If they are not useable, find some new data!
- What is the overall shape of the distribution? Is it normal, skewed, bimodal, uniform, etc.?
- Are there any outliers present? (Data points that are far from the others.)
- If there are multiple related histograms, how does the distribution change across different groups?

- What are the minimum and maximum values represented in each histogram?
- How do bin sizes affect the histogram? Does changing the bin width reveal different patterns in the data?
- Does the distribution appear normal, or does it have a different distribution?

In [78]:
```
import matplotlib.pyplot as plt
```

Read Data

In [79]:
```
# https://www.kaggle.com/datasets/danb91/covid-ppp-loan-data-with-fraud-examples
df = pd.read_csv("C:/Users/caleb/Downloads/ppploan/ppp_over_150k.csv")
df
```

Out[79]:

	LoanNumber	DateApproved	SBAOfficeCode	ProcessingMethod	BorrowerName
0	9547507704	2020-05-01	464	PPP	sumter coatings inc
1	9777677704	2020-05-01	464	PPP	pleasant places inc
2	5791407702	2020-05-01	1013	PPP	boyer childrens clinic
3	6223567700	2020-05-01	920	PPP	kirtley construction inc
4	9662437702	2020-05-01	101	PPP	aero box llc
...	...	...	...	...	...
968527	4395967002	2020-04-03	897	PPP	roy e paulson jr pc
968528	6985647108	2020-04-14	897	PPP	sweetwater county child developmental center inc
968529	7996438405	2021-02-12	897	PPS	electrical systems of wyoming inc
968530	9054647103	2020-04-15	897	PPP	eden life care
968531	9184687004	2020-04-09	897	PPP	s s johnson enterprises inc

968532 rows × 66 columns

In [80]:
```
fraud_df = pd.read_csv("C:/Users/caleb/Downloads/ppploan/ppp_fraud_cases.csv")
fraud_df
```

Out[80]:

	Company	State	Source	Suspects
0	apollo hp inc	CA	https://www.justice.gov/usao-edca/pr/rocklin-b...	[['Kevin Lee Co', 'CA']]
1	power house sports academy	GA	https://www.justice.gov/opa/pr/six-charged-con...	[['Rodericque Thompson', 'GA'], ['Micah K Bais...
2	faithful transport services llc	SC	https://www.justice.gov/opa/pr/six-charged-con...	[['Rodericque Thompson', 'GA'], ['Travis C Cro...
3	kmj transport llc	SC	https://www.justice.gov/opa/pr/six-charged-con...	[['Rodericque Thompson', 'GA'], ['Keith A Malo...
4	market yourself inc	GA	https://www.justice.gov/opa/pr/six-charged-con...	[['Rodericque Thompson', 'GA'], ['Tabronx W Sm...
...	...	...	...	...
95	pink lady line	TX	https://www.justice.gov/usao-sdtx/pr/local-wom...	[['LaDonna Wiggins', 'TX']]
96	guardian of humanity inc	AZ	https://www.justice.gov/usao-wdny/pr/north-chi...	[['Michael Rech', 'NY']]
97	skilled trade investments lp	TX	https://www.justice.gov/opa/pr/texas-man-arres...	[['Scott Jackson Davis', 'TX']]
98	skilled trade staffing llc	TX	https://www.justice.gov/opa/pr/texas-man-arres...	[['Scott Jackson Davis', 'TX']]
99	skilled trade investments gp llc	TX	https://www.justice.gov/opa/pr/texas-man-arres...	[['Scott Jackson Davis', 'TX']]

100 rows × 4 columns

In [81]:
```
# Label the fraudulent rows based on company names in the label dataset
df['Fraud'] = df['BorrowerName'].isin(fraud_df['Company']).astype(int)
```

In [82]:
```
# Sanity check the join
len(fraud_df['Company'].unique())
```

Out[82]: 100

In [83]:
```
# We only see 83 of the 100 fraudulent companies... there may be invalid company na
len(set(df[df['Fraud'] == 1]['BorrowerName'].unique()))
```

Out[83]: 83

In [84]:
```
not_found_fraudulent_companies = set(
    fraud_df['Company'].unique()
).difference(set(
    df[df['Fraud'] == 1]['BorrowerName'].unique()
))
not_found_fraudulent_companies
```

Out[84]:
```
{'aboveall business',
 'all star room and board services of michigan inc',
 'blue flight logistics llc',
 'dituri construction llc',
 'eni operation co',
 'eric r shibley md pllc',
 'es1 llc',
 'maturino enterprises',
 'nkb enterprise llc',
 'pinnacle industry inc',
 'seattles finest cannabis llc',
 'sfc llc',
 'showtime trucking xpo inc',
 'ss1 llc',
 'velocity innovations',
 'visla captial llc',
 'wingate funeral service group'}
```

In [85]:
```
# Transforming the company names to make the search less stringent. We may be able
companies = [
    'aboveall',
    'all star room',
    'blue flight',
    'dituri',
    'eni operation',
    'shibley',
    'es1',
    'maturino',
    'nkb enterprise',
    'pinnacle industry',
    'seattles finest cannabis',
    'sfc',
    'showtime trucking ',
    'ss1',
    'velocity',
    'visla captial ',
    'wingate funeral'
]
for c in companies:
    print(f"{c}: {df['BorrowerName'].str.contains(c).any()}")
    print(df[df['BorrowerName'].str.contains(c, na=False)]['BorrowerName'])
    print('\n')
```

```
aboveall: False
Series([], Name: BorrowerName, dtype: object)


all star room: True
454316    all star room  board services of michigan inc
Name: BorrowerName, dtype: object


blue flight: False
Series([], Name: BorrowerName, dtype: object)


dituri: True
588733    drs gardiner  dituri pasba small 7a term
Name: BorrowerName, dtype: object


eni operation: False
Series([], Name: BorrowerName, dtype: object)


shibley: True
371855                  shibleyville llc
373694                  shibleyville llc
623978          shibley operating co llc
623979          shibley operating co llc
680901            shibley management inc
684010            shibley management inc
692598    spangenberg shibley  liber llp
692680    spangenberg shibley  liber llp
Name: BorrowerName, dtype: object


es1: True
25546     connie a hottmann dba gampg enterprises1
85495                                pawsibilities1
107014                          pawsibilities1 inc
697585                   gc aviation services1 llc
962841                                 naples15llc
Name: BorrowerName, dtype: object


maturino: False
Series([], Name: BorrowerName, dtype: object)


nkb enterprise: False
Series([], Name: BorrowerName, dtype: object)


pinnacle industry: True
717516    pinnacle industry llc
Name: BorrowerName, dtype: object
```

```
seattles finest cannabis: False
Series([], Name: BorrowerName, dtype: object)


sfc: True
48382                      sfc communications inc
142657                                  dasfc inc
144540                                   sfcj inc
155501                          sfcs van ness llc
157550                          sfcs van ness llc
270256        sfc contract services of georgia llc
270257        sfc contract services of georgia llc
385224                   sfc contract services inc
559203             sfc engineering partnership inc
646209                                   rsfc llc
679289                                   rsfc llc
697190                                    sfc ltd
697322                                    sfc ltd
766923                                   lsfc inc
768870                   sfc delivery services inc
777362    sfc capillas exequiales y funerarias llc
845500                                   qsfc ltd
849726                         sfc management llc
855813                                   qsfc llc
879229                                   sfcc inc
896933                                   sfcs inc
Name: BorrowerName, dtype: object


showtime trucking : True
572055    showtime trucking llc
572361    showtime trucking llc
Name: BorrowerName, dtype: object


ss1: True
98072     rss1 walnut creek llc
137829    rss1 walnut creek llc
218370     fitness1 kendall llc
243950    mattress1holding corp
706107                hrss1 llc
708289                hrss1 llc
Name: BorrowerName, dtype: object


velocity: True
2806                 velocity ventures inc
25422                    velocityscape llc
41744     infinite velocity automotive inc
41745     infinite velocity automotive inc
43018                  velocity motors llc
                         ...
929025       velocity consulting solutions
929415       velocity consulting solutions
947116                velocity machine inc
951280              velocity transport inc
```

```
959232            velocity water works llc
Name: BorrowerName, Length: 106, dtype: object


visla captial : False
Series([], Name: BorrowerName, dtype: object)


wingate funeral: False
Series([], Name: BorrowerName, dtype: object)
```

Summarize Data

In [86]:
```
df.describe()
```

```
c:\Users\caleb\anaconda3\envs\iris-fisher\lib\site-packages\pandas\core\nanops.py:10
16: RuntimeWarning: invalid value encountered in subtract
  sqr = _ensure_numeric((avg - values) ** 2)
```

Out[86]:

	LoanNumber	SBAOfficeCode	Term	SBAGuarantyPercentage	InitialApprov
count	9.685320e+05	968532.000000	968532.000000	968532.0	9.68
mean	5.427133e+09	571.520066	36.297555	100.0	5.32
std	2.551320e+09	263.025314	17.214728	0.0	7.44
min	1.000007e+09	101.000000	0.000000	100.0	0.00
25%	3.271096e+09	373.000000	24.000000	100.0	2.00
50%	5.400677e+09	515.000000	24.000000	100.0	2.95
75%	7.546310e+09	811.000000	60.000000	100.0	5.40
max	9.999007e+09	1094.000000	120.000000	100.0	1.00

8 rows × 33 columns

In [87]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 968532 entries, 0 to 968531
Data columns (total 67 columns):
 #   Column                        Non-Null Count   Dtype
---  ------                        --------------   -----
 0   LoanNumber                    968532 non-null  int64
 1   DateApproved                  968532 non-null  object
 2   SBAOfficeCode                 968532 non-null  int64
 3   ProcessingMethod              968532 non-null  object
 4   BorrowerName                  968525 non-null  object
 5   BorrowerAddress               968532 non-null  object
 6   BorrowerCity                  968532 non-null  object
 7   BorrowerState                 968532 non-null  object
 8   BorrowerZip                   968532 non-null  object
 9   LoanStatusDate                968532 non-null  object
 10  LoanStatus                    968532 non-null  object
 11  Term                          968532 non-null  int64
 12  SBAGuarantyPercentage         968532 non-null  int64
 13  InitialApprovalAmount         968532 non-null  float64
 14  CurrentApprovalAmount         968532 non-null  float64
 15  UndisbursedAmount             968532 non-null  float64
 16  FranchiseName                 968532 non-null  object
 17  ServicingLenderLocationID     968532 non-null  int64
 18  ServicingLenderName           968532 non-null  object
 19  ServicingLenderAddress        968532 non-null  object
 20  ServicingLenderCity           968532 non-null  object
 21  ServicingLenderState          968532 non-null  object
 22  ServicingLenderZip            968532 non-null  object
 23  RuralUrbanIndicator           968532 non-null  object
 24  HubzoneIndicator              968532 non-null  object
 25  LMIIndicator                  968532 non-null  object
 26  BusinessAgeDescription        968532 non-null  object
 27  ProjectCity                   968532 non-null  object
 28  ProjectCountyName             968532 non-null  object
 29  ProjectState                  968532 non-null  object
 30  ProjectZip                    968532 non-null  object
 31  CD                            968532 non-null  object
 32  JobsReported                  968532 non-null  float64
 33  NAICSCode                     968532 non-null  int64
 34  Race                          968532 non-null  object
 35  Ethnicity                     968532 non-null  object
 36  UTILITIES_PROCEED             968532 non-null  float64
 37  PAYROLL_PROCEED               968532 non-null  float64
 38  MORTGAGE_INTEREST_PROCEED     968532 non-null  float64
 39  RENT_PROCEED                  968532 non-null  float64
 40  REFINANCE_EIDL_PROCEED        968532 non-null  float64
 41  HEALTH_CARE_PROCEED           968532 non-null  float64
 42  DEBT_INTEREST_PROCEED         968532 non-null  float64
 43  BusinessType                  968532 non-null  object
 44  OriginatingLenderLocationID   968532 non-null  int64
 45  OriginatingLender             968532 non-null  object
 46  OriginatingLenderCity         968532 non-null  object
 47  OriginatingLenderState        968532 non-null  object
 48  Gender                        968532 non-null  object
 49  Veteran                       968532 non-null  object
 50  NonProfit                     968532 non-null  object
```

```
 51  ForgivenessAmount              968532 non-null  float64
 52  ForgivenessDate                968532 non-null  object
 53  ApprovalDiff                   968532 non-null  float64
 54  NotForgivenAmount              968532 non-null  float64
 55  ForgivenPercentage             968532 non-null  float64
 56  TOTAL_PROCEED                  968532 non-null  float64
 57  PROCEED_Diff                   968532 non-null  float64
 58  UTILITIES_PROCEED_pct          968532 non-null  float64
 59  PAYROLL_PROCEED_pct            968532 non-null  float64
 60  MORTGAGE_INTEREST_PROCEED_pct  968532 non-null  float64
 61  RENT_PROCEED_pct               968532 non-null  float64
 62  REFINANCE_EIDL_PROCEED_pct     968532 non-null  float64
 63  HEALTH_CARE_PROCEED_pct        968532 non-null  float64
 64  DEBT_INTEREST_PROCEED_pct      968532 non-null  float64
 65  PROCEED_Per_Job                968532 non-null  float64
 66  Fraud                          968532 non-null  int64
dtypes: float64(25), int64(8), object(34)
memory usage: 495.1+ MB
```

There are no missing values

Check for Infinite Numeric Values and Duplicates

In [88]:
```
numeric_df = df.select_dtypes(include=[np.number])

# Find rows where ANY column has an infinite value
rows_with_inf = df[np.isinf(numeric_df).any(axis=1)]
rows_with_inf
```

Out[88]:

	LoanNumber	DateApproved	SBAOfficeCode	ProcessingMethod	BorrowerName
123812	6557167306	2020-04-30	914	PPP	st margaret mary school
550932	5780187005	2020-04-06	766	PPP	reliant transportation inc
763783	4864227203	2020-04-27	303	PPP	vincera rehab llc
967206	4563247009	2020-04-03	897	PPP	weeden construction llc

4 rows × 67 columns

In [89]:
```
# We will replaced these inf values with NaN
df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

In [90]:
```
# There are no duplicates
duplicates = df[df.duplicated(keep=False)]
duplicates
```

Out[90]:

LoanNumber	DateApproved	SBAOfficeCode	ProcessingMethod	BorrowerName	Borrow

0 rows × 67 columns

Check for Inconsistent Data

In [91]:

```
# Validate amounts
amount_cols = [c for c in df.columns if "Amount" in c]
for a in amount_cols:
    print(f"Column: {a}")
    print(f"Min Val: {df[a].min()}, Max Val: {df[a].max()}")
    print(f"Number of occurences of Min Val: {len(df[df[a] == df[a].min()])}, numbe
    print("\n")
```

```
Column: InitialApprovalAmount
Min Val: 0.0, Max Val: 10000000.0
Number of occurences of Min Val: 2, number of occurences of Max Val: 747


Column: CurrentApprovalAmount
Min Val: 150000.0, Max Val: 10000000.0
Number of occurences of Min Val: 6690, number of occurences of Max Val: 707


Column: UndisbursedAmount
Min Val: 0.0, Max Val: 3397100.0
Number of occurences of Min Val: 968515, number of occurences of Max Val: 1


Column: ForgivenessAmount
Min Val: 0.0, Max Val: 10196272.22
Number of occurences of Min Val: 93710, number of occurences of Max Val: 1


Column: NotForgivenAmount
Min Val: -196272.22000000067, Max Val: 10000000.0
Number of occurences of Min Val: 1, number of occurences of Max Val: 118
```

For InitialApprovalAmount, CurrentApprovalAmount, and NotForgivenAmount, the maximum value of 10000000.0 being common makes sense since the maximum allowed amount was $10,000,000 for an individual borrower (https://www.jw.com/news/insights-2021-paycheck-protection-program-loans/)

For CurrentApprovalAmount, the 150,000 minimum looks to be clipped. It is likely 150k loans or lower

The rest of the amounts do not seem to be inconsistent

In [92]:
```
# Validate proceeds
proceed_cols = [c for c in df.columns if "PROCEED" in c]
for p in proceed_cols:
    print(f"Column: {p}")
    print(f"Min Val: {df[p].min()}, Max Val: {df[p].max()}")
    print(f"Number of occurences of Min Val: {len(df[df[p] == df[p].min()])}, numbe
    print("\n")
```

```
Column: UTILITIES_PROCEED
Min Val: 0.0, Max Val: 10000000.0
Number of occurences of Min Val: 635755, number of occurences of Max Val: 1


Column: PAYROLL_PROCEED
Min Val: 0.0, Max Val: 10000000.0
Number of occurences of Min Val: 1931, number of occurences of Max Val: 512


Column: MORTGAGE_INTEREST_PROCEED
Min Val: 0.0, Max Val: 10000000.0
Number of occurences of Min Val: 939898, number of occurences of Max Val: 1


Column: RENT_PROCEED
Min Val: 0.0, Max Val: 5518277.93
Number of occurences of Min Val: 875030, number of occurences of Max Val: 1


Column: REFINANCE_EIDL_PROCEED
Min Val: 0.0, Max Val: 2951590.0
Number of occurences of Min Val: 966784, number of occurences of Max Val: 1


Column: HEALTH_CARE_PROCEED
Min Val: 0.0, Max Val: 3880000.0
Number of occurences of Min Val: 916822, number of occurences of Max Val: 1


Column: DEBT_INTEREST_PROCEED
Min Val: 0.0, Max Val: 2497617.0
Number of occurences of Min Val: 952173, number of occurences of Max Val: 1


Column: TOTAL_PROCEED
Min Val: 94800.0, Max Val: 10000000.0
Number of occurences of Min Val: 1, number of occurences of Max Val: 639


Column: PROCEED_Diff
Min Val: -14997.0, Max Val: 323707.0
Number of occurences of Min Val: 1, number of occurences of Max Val: 1


Column: UTILITIES_PROCEED_pct
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 860335, number of occurences of Max Val: 1014


Column: PAYROLL_PROCEED_pct
Min Val: 0.0, Max Val: 1.041
Number of occurences of Min Val: 1935, number of occurences of Max Val: 1


Column: MORTGAGE_INTEREST_PROCEED_pct
```

```
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 940120, number of occurences of Max Val: 213


Column: RENT_PROCEED_pct
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 875213, number of occurences of Max Val: 76


Column: REFINANCE_EIDL_PROCEED_pct
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 966847, number of occurences of Max Val: 122


Column: HEALTH_CARE_PROCEED_pct
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 916883, number of occurences of Max Val: 41


Column: DEBT_INTEREST_PROCEED_pct
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 952318, number of occurences of Max Val: 5


Column: PROCEED_Per_Job
Min Val: 0.0, Max Val: 6512997.5
Number of occurences of Min Val: 1931, number of occurences of Max Val: 1
```

The proceed columns tell a similar story. Some max out at $10,000,000. The minimums are mostly 0$. No inconsistent data looks to be present

Feature Observations

The following numeric columns should not be used for modeling:

- Loan Number (ID)
- SBAOfficeCode (ID, though maybe it matters if certain locations are approving all fraudulent loans)
- SBAGuarantyPercentage(Always value 100)
- UndisbursedAmount (Always value 0)
- ServicingLenderLocationID (ID)
- NAICSCode (ID)
- OriginatingLenderLocationID (ID)

The following column have numeric dtype but are categorical

- Term

The following non-numeric columns should not be used for modeling:

- BorrowerAddress (Random string)
- BorrowerName (Random string)
- Ethnicity (Bias)
- FranchiseName (random str)
- Gender (Bias)
- OriginatingLender (Random str)
- Race (bias)
- ServicingLenderAddress (random str)
- ServicingLenderName (random str)

There are also a few non-numeric columns that are categorical but have many values. I will skip them for initial analysis but may revisit them if they end up being useful

- BorrowerCity (categorical but not specific enough to be useful)
- BorrowerZip (categorical but not specific enough to be useful)
- CD (categorical but not specific enough to be useful)
- OriginatingLenderCity (categorical but not specific enough to be useful)
- ProjectCity (categorical but not specific enough to be useful)
- ProjectCountyName (categorical but not specific enough to be useful)
- ProjectZip (categorical but not specific enough to be useful)
- ServicingLenderCity (categorical but not specific enough to be useful)
- ServicingLenderZip (categorical but not specific enough to be useful)

Non numeric cols which can be used for modeling (one hot/ordinal encoding or datetime to numeric conversion)

- BusinessType (categorical)
- BusinessAgeDescription (categorical)
- DateApproved (datetime, maybe convert to int)
- ForgivenessDate (datetime, maybe convert to int)
- HubzoneIndicator (categorical)
- LMIndicator (categorical)
- LoanStatus (categorical)
- LoanStatusDate (datetime, maybe convert to int)
- NonProfit (categorical)
- ProcessingMethod (categorical)
- RuralUrbanIndicator (categorical)
- Veteran (categorical)

In [93]:
```
numeric_cols_to_drop = [
    'LoanNumber',
    'SBAOfficeCode',
    'SBAGuarantyPercentage',
    'UndisbursedAmount',
    'ServicingLenderLocationID',
```

```
    'NAICSCode',
    'OriginatingLenderLocationID',
]
```

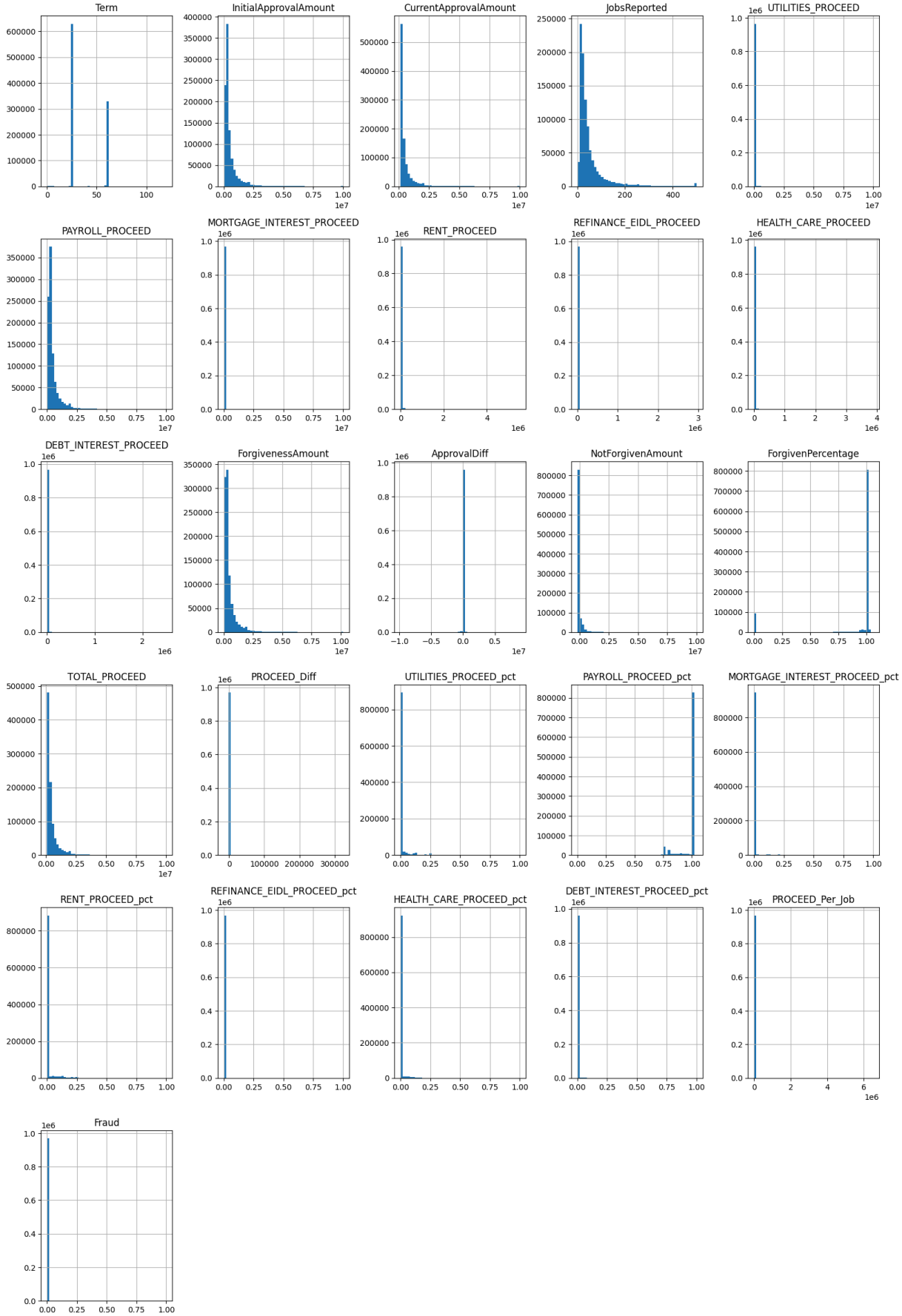
In [94]:
```
non_numeric_cols_to_drop = [
    'BorrowerCity',
    'BorrowerZip',
    'CD',
    'OriginatingLenderCity',
    'ProjectCity',
    'ProjectCountyName',
    'ProjectZip',
    'ServicingLenderCity',
    'ServicingLenderZip',
    'BorrowerAddress',
    'BorrowerName',
    'Ethnicity',
    'FranchiseName',
    'Gender',
    'OriginatingLender',
    'Race',
    'ServicingLenderAddress',
    'ServicingLenderName',
]
```

In [95]:
```
columns_to_drop = set(numeric_cols_to_drop).union(set(non_numeric_cols_to_drop))
df = df.drop(columns_to_drop, axis=1)
```

In [96]:
```
numeric_cols = df.select_dtypes(include='number').columns
non_numeric_cols = df.columns.difference(numeric_cols)
```

Histograms

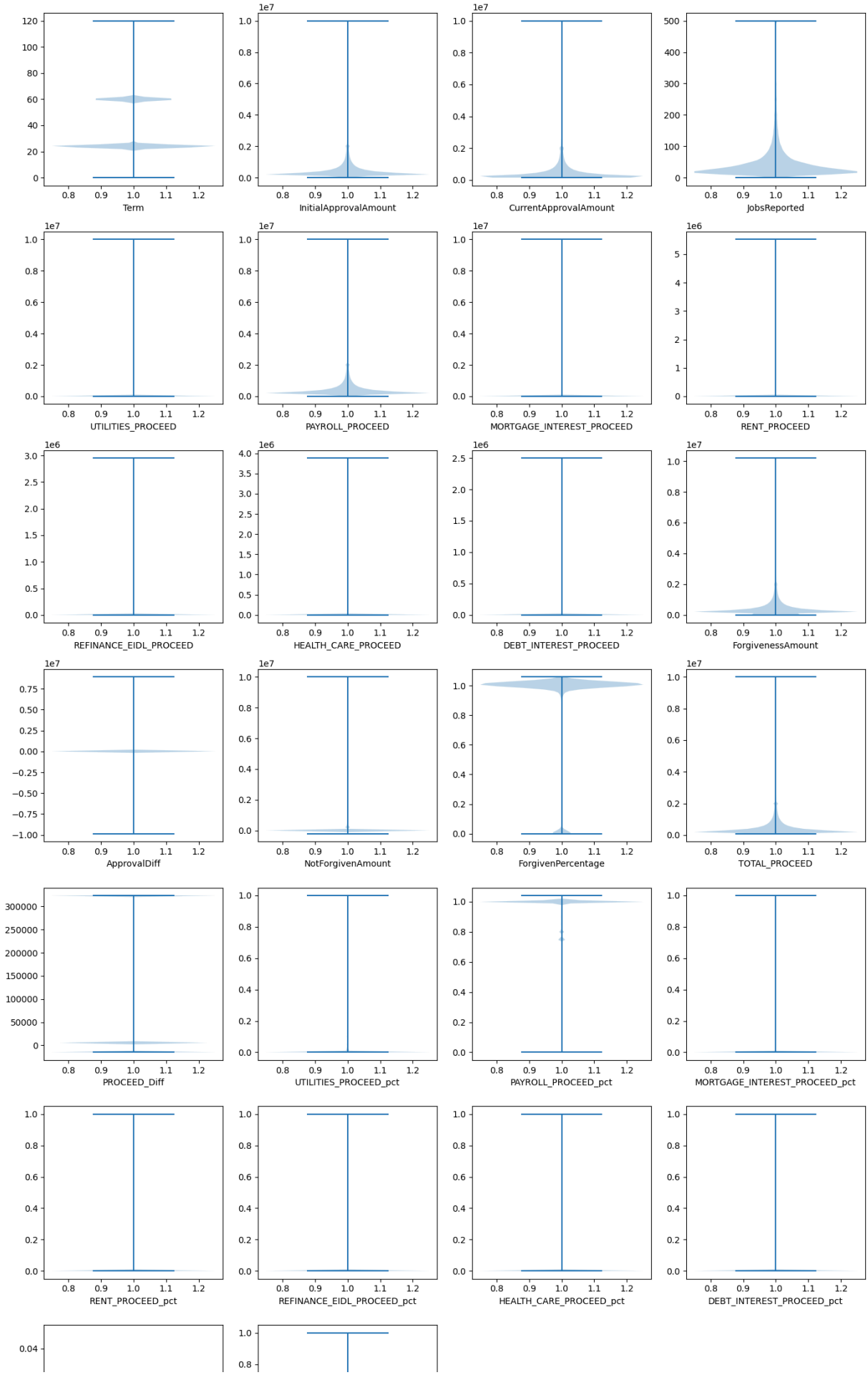
In [97]:
```
df[numeric_cols].hist(column=numeric_cols, figsize=(16, 24), bins=50)
plt.tight_layout()
None
```

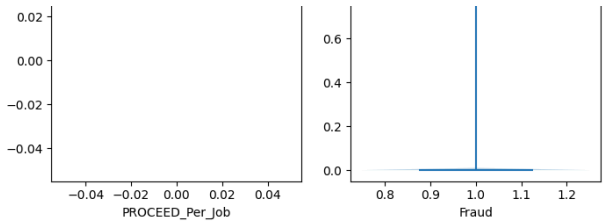


Violin Plots

In [98]:
```
num_cols = 4
num_rows = (len(numeric_cols) + num_cols - 1) // num_cols
plt.figure(figsize=(num_cols * 3.5, num_rows * 3.5))
for (i,c) in enumerate(numeric_cols):
    plt.subplot(num_rows, num_cols, i+1)
    plt.violinplot(df[c])
    plt.xlabel(c)
plt.tight_layout()
None
```

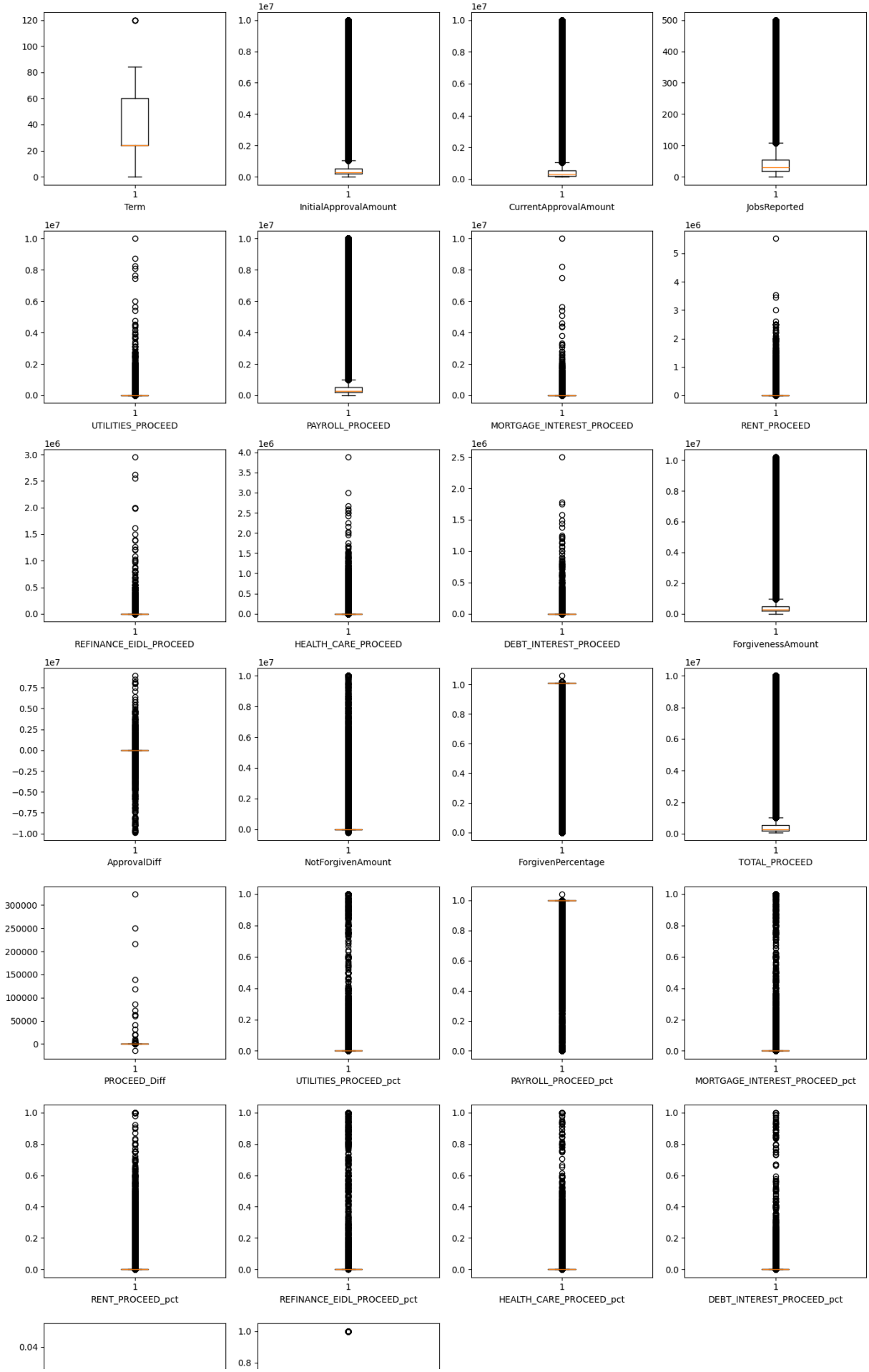
```
c:\Users\caleb\anaconda3\envs\iris-fisher\lib\site-packages\numpy\linalg\_linalg.py:
2383: RuntimeWarning: invalid value encountered in det
  r = _umath_linalg.det(a, signature=signature)
```

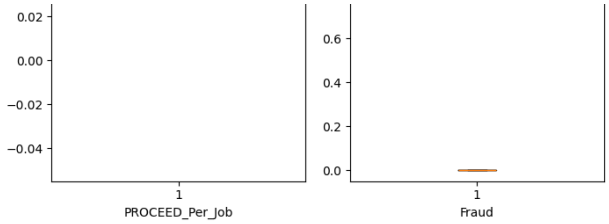

Term
InitialApprovalAmount
CurrentApprovalAmount
JobsReported
UTILITIES_PROCEED
PAYROLL_PROCEED
MORTGAGE_INTEREST_PROCEED
RENT_PROCEED
REFINANCE_EIDL_PROCEED
HEALTH_CARE_PROCEED
DEBT_INTEREST_PROCEED
ForgivenessAmount
ApprovalDiff
NotForgivenAmount
ForgivenPercentage
TOTAL_PROCEED
PROCEED_Diff
UTILITIES_PROCEED_pct
PAYROLL_PROCEED_pct
MORTGAGE_INTEREST_PROCEED_pct
RENT_PROCEED_pct
REFINANCE_EIDL_PROCEED_pct
HEALTH_CARE_PROCEED_pct
DEBT_INTEREST_PROCEED_pct



Boxplots

In [99]:
```
num_cols = 4
num_rows = (len(numeric_cols) + num_cols - 1) // num_cols
plt.figure(figsize=(num_cols * 3.5, num_rows * 3.5))
for (i,c) in enumerate(numeric_cols):
    plt.subplot(num_rows, num_cols, i+1)
    plt.boxplot(df[c])
    plt.xlabel(c)
plt.tight_layout()
None
```


Term
InitialApprovalAmount
CurrentApprovalAmount
JobsReported
UTILITIES_PROCEED
PAYROLL_PROCEED
MORTGAGE_INTEREST_PROCEED
RENT_PROCEED
REFINANCE_EIDL_PROCEED
HEALTH_CARE_PROCEED
DEBT_INTEREST_PROCEED
ForgivenessAmount
ApprovalDiff
NotForgivenAmount
ForgivenPercentage
TOTAL_PROCEED
PROCEED_Diff
UTILITIES_PROCEED_pct
PAYROLL_PROCEED_pct
MORTGAGE_INTEREST_PROCEED_pct
RENT_PROCEED_pct
REFINANCE_EIDL_PROCEED_pct
HEALTH_CARE_PROCEED_pct
DEBT_INTEREST_PROCEED_pct



Plotting Categorical Columns

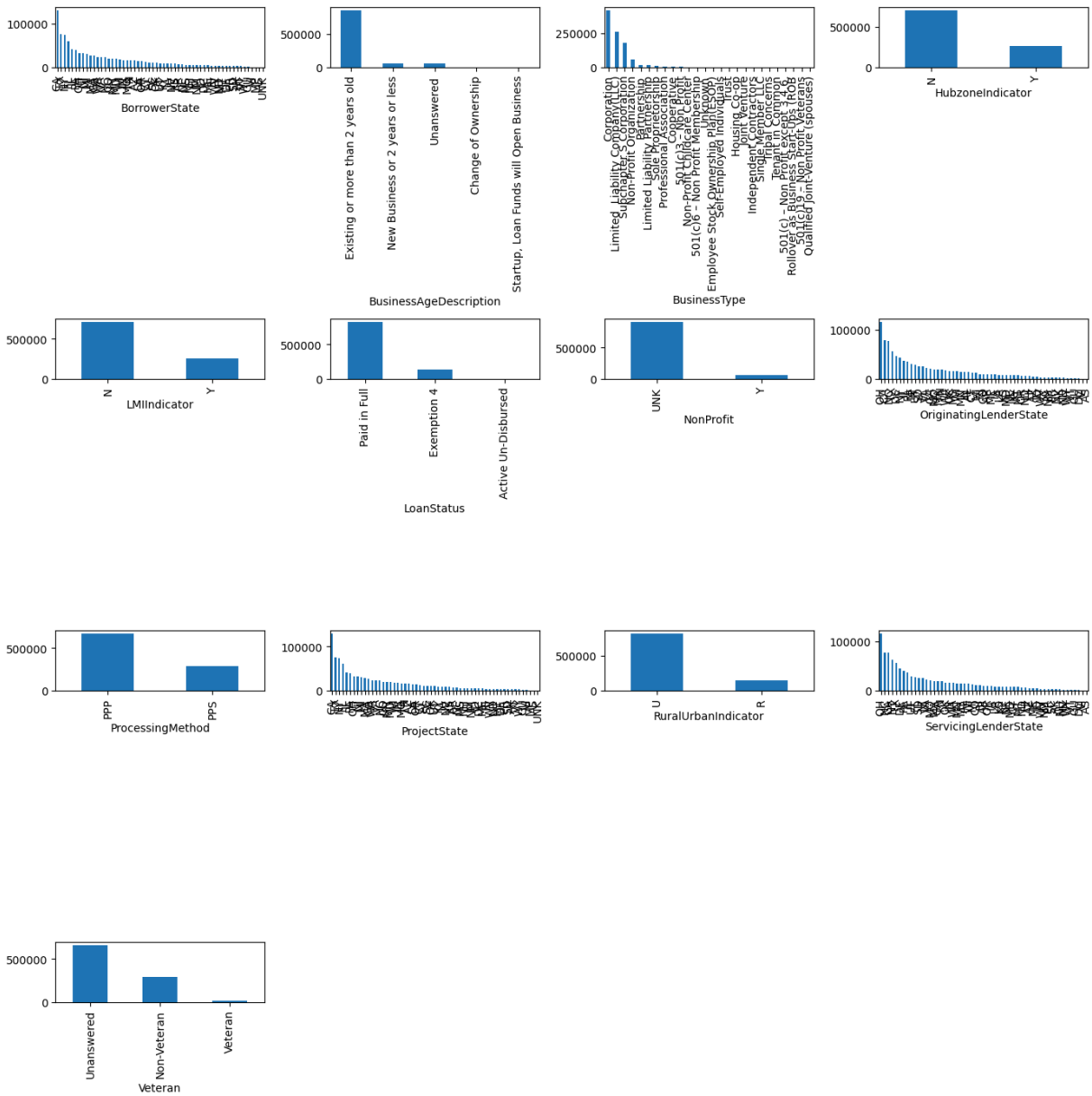
In [100…
```
categorical_cols = [c for c in non_numeric_cols if df[c].nunique() <= 100]
```

In [101…
```
set(non_numeric_cols) - set(categorical_cols)
```

Out[101…
```
{'DateApproved', 'ForgivenessDate', 'LoanStatusDate'}
```

In [104…
```
date_columns = [
    'DateApproved',
    'ForgivenessDate',
    'LoanStatusDate',
]
```

In [102…
```
num_cols = 4
num_rows = (len(categorical_cols) + num_cols - 1) // num_cols
plt.figure(figsize=(num_cols * 3.5, num_rows * 3.5))
for (i,c) in enumerate(categorical_cols):
    plt.subplot(num_rows, num_cols, i+1)
    df[c].value_counts().plot(kind='bar')
    plt.xlabel(c)
plt.tight_layout()
None
```



Encoding Date Features

In [107…

```
for c in date_columns:
    print(df[c].head(5))
    print()
```

```
0    2020-05-01
1    2020-05-01
2    2020-05-01
3    2020-05-01
4    2020-05-01
Name: DateApproved, dtype: object

0    2020-11-20
1    2021-08-12
2    2021-02-10
3    2021-09-10
4    2021-04-08
Name: ForgivenessDate, dtype: object

0    2020-12-18
1    2021-09-28
2    2021-03-17
3    2021-10-16
4    2021-08-17
Name: LoanStatusDate, dtype: object
```

In [109…

```
for c in date_columns:
    df[f"{c}_datetime"] = pd.to_datetime(df[c], format='%Y-%m-%d')
    print(df[f"{c}_datetime"].head(5))
    df[f"{c}_int"] = df[f"{c}_datetime"].astype(np.int64) // 10**9
    print(df[f"{c}_int"].head(5))
    df.drop([c, f"{c}_datetime"], axis=1, inplace=True)
```

```
0   2020-05-01
1   2020-05-01
2   2020-05-01
3   2020-05-01
4   2020-05-01
Name: DateApproved_datetime, dtype: datetime64[ns]
0    1588291200
1    1588291200
2    1588291200
3    1588291200
4    1588291200
Name: DateApproved_int, dtype: int64
0   2020-11-20
1   2021-08-12
2   2021-02-10
3   2021-09-10
4   2021-04-08
Name: ForgivenessDate_datetime, dtype: datetime64[ns]
0    1605830400
1    1628726400
2    1612915200
3    1631232000
4    1617840000
Name: ForgivenessDate_int, dtype: int64
0   2020-12-18
1   2021-09-28
2   2021-03-17
3   2021-10-16
4   2021-08-17
Name: LoanStatusDate_datetime, dtype: datetime64[ns]
0    1608249600
1    1632787200
2    1615939200
3    1634342400
4    1629158400
Name: LoanStatusDate_int, dtype: int64
```

Encoding Categorical Features

In [114…

```
categorical_cols
```

Out[114…

```
['BorrowerState',
 'BusinessAgeDescription',
 'BusinessType',
 'HubzoneIndicator',
 'LMIIndicator',
 'LoanStatus',
 'NonProfit',
 'OriginatingLenderState',
 'ProcessingMethod',
 'ProjectState',
 'RuralUrbanIndicator',
 'ServicingLenderState',
 'Veteran']
```

I am going to Ordinal Encode each of these due to the size of the data. We can later attempt One-Hot encoding.

In [115…

```
from sklearn.preprocessing import OrdinalEncoder

enc = OrdinalEncoder()
enc.fit(df[categorical_cols])
enc.categories_
```

Out[115…

```
[array(['AK', 'AL', 'AR', 'AS', 'AZ', 'CA', 'CO', 'CT', 'DC', 'DE', 'FL',
        'GA', 'GU', 'HI', 'IA', 'ID', 'IL', 'IN', 'KS', 'KY', 'LA', 'MA',
        'MD', 'ME', 'MI', 'MN', 'MO', 'MP', 'MS', 'MT', 'NC', 'ND', 'NE',
        'NH', 'NJ', 'NM', 'NV', 'NY', 'OH', 'OK', 'OR', 'PA', 'PR', 'RI',
        'SC', 'SD', 'TN', 'TX', 'UNK', 'UT', 'VA', 'VI', 'VT', 'WA', 'WI',
        'WV', 'WY'], dtype=object),
 array(['Change of Ownership', 'Existing or more than 2 years old',
        'New Business or 2 years or less',
        'Startup, Loan Funds will Open Business', 'Unanswered'],
       dtype=object),
 array(['501(c) – Non Profit except 3,4,6,',
        '501(c)19 – Non Profit Veterans', '501(c)3 – Non Profit',
        '501(c)6 – Non Profit Membership', 'Cooperative', 'Corporation',
        'Employee Stock Ownership Plan(ESOP)', 'Housing Co-op',
        'Independent Contractors', 'Joint Venture',
        'Limited  Liability Company(LLC)', 'Limited Liability Partnership',
        'Non-Profit Childcare Center', 'Non-Profit Organization',
        'Partnership', 'Professional Association',
        'Qualified Joint-Venture (spouses)',
        'Rollover as Business Start-Ups (ROB', 'Self-Employed Individuals',
        'Single Member LLC', 'Sole Proprietorship',
        'Subchapter S Corporation', 'Tenant in Common', 'Tribal Concerns',
        'Trust', 'Unknown'], dtype=object),
 array(['N', 'Y'], dtype=object),
 array(['N', 'Y'], dtype=object),
 array(['Active Un-Disbursed', 'Exemption 4', 'Paid in Full'], dtype=object),
 array(['UNK', 'Y'], dtype=object),
 array(['AK', 'AL', 'AR', 'AS', 'AZ', 'CA', 'CO', 'CT', 'DC', 'DE', 'FL',
        'GA', 'GU', 'HI', 'IA', 'ID', 'IL', 'IN', 'KS', 'KY', 'LA', 'MA',
        'MD', 'ME', 'MI', 'MN', 'MO', 'MS', 'MT', 'NC', 'ND', 'NE', 'NH',
        'NJ', 'NM', 'NV', 'NY', 'OH', 'OK', 'OR', 'PA', 'PR', 'RI', 'SC',
        'SD', 'TN', 'TX', 'UT', 'VA', 'VI', 'VT', 'WA', 'WI', 'WV', 'WY'],
       dtype=object),
 array(['PPP', 'PPS'], dtype=object),
 array(['AK', 'AL', 'AR', 'AS', 'AZ', 'CA', 'CO', 'CT', 'DC', 'DE', 'FL',
        'GA', 'GU', 'HI', 'IA', 'ID', 'IL', 'IN', 'KS', 'KY', 'LA', 'MA',
        'MD', 'ME', 'MI', 'MN', 'MO', 'MP', 'MS', 'MT', 'NC', 'ND', 'NE',
        'NH', 'NJ', 'NM', 'NV', 'NY', 'OH', 'OK', 'OR', 'PA', 'PR', 'RI',
        'SC', 'SD', 'TN', 'TX', 'UNK', 'UT', 'VA', 'VI', 'VT', 'WA', 'WI',
        'WV', 'WY'], dtype=object),
 array(['R', 'U'], dtype=object),
 array(['AK', 'AL', 'AR', 'AS', 'AZ', 'CA', 'CO', 'CT', 'DC', 'DE', 'FL',
        'GA', 'GU', 'HI', 'IA', 'ID', 'IL', 'IN', 'KS', 'KY', 'LA', 'MA',
        'MD', 'ME', 'MI', 'MN', 'MO', 'MS', 'MT', 'NC', 'ND', 'NE', 'NH',
        'NJ', 'NM', 'NV', 'NY', 'OH', 'OK', 'OR', 'PA', 'PR', 'RI', 'SC',
        'SD', 'TN', 'TX', 'UT', 'VA', 'VI', 'VT', 'WA', 'WI', 'WV', 'WY'],
       dtype=object),
 array(['Non-Veteran', 'Unanswered', 'Veteran'], dtype=object)]
```

In [116…

```
df[categorical_cols] = enc.transform(df[categorical_cols])
```

In [118…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 968532 entries, 0 to 968531
Data columns (total 42 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   ProcessingMethod                968532 non-null  float64
 1   BorrowerState                   968532 non-null  float64
 2   LoanStatus                      968532 non-null  float64
 3   Term                            968532 non-null  int64
 4   InitialApprovalAmount           968532 non-null  float64
 5   CurrentApprovalAmount           968532 non-null  float64
 6   ServicingLenderState            968532 non-null  float64
 7   RuralUrbanIndicator             968532 non-null  float64
 8   HubzoneIndicator                968532 non-null  float64
 9   LMIIndicator                    968532 non-null  float64
 10  BusinessAgeDescription          968532 non-null  float64
 11  ProjectState                    968532 non-null  float64
 12  JobsReported                    968532 non-null  float64
 13  UTILITIES_PROCEED               968532 non-null  float64
 14  PAYROLL_PROCEED                 968532 non-null  float64
 15  MORTGAGE_INTEREST_PROCEED       968532 non-null  float64
 16  RENT_PROCEED                    968532 non-null  float64
 17  REFINANCE_EIDL_PROCEED          968532 non-null  float64
 18  HEALTH_CARE_PROCEED             968532 non-null  float64
 19  DEBT_INTEREST_PROCEED           968532 non-null  float64
 20  BusinessType                    968532 non-null  float64
 21  OriginatingLenderState          968532 non-null  float64
 22  Veteran                         968532 non-null  float64
 23  NonProfit                       968532 non-null  float64
 24  ForgivenessAmount               968532 non-null  float64
 25  ApprovalDiff                    968532 non-null  float64
 26  NotForgivenAmount               968532 non-null  float64
 27  ForgivenPercentage              968532 non-null  float64
 28  TOTAL_PROCEED                   968532 non-null  float64
 29  PROCEED_Diff                    968532 non-null  float64
 30  UTILITIES_PROCEED_pct           968532 non-null  float64
 31  PAYROLL_PROCEED_pct             968532 non-null  float64
 32  MORTGAGE_INTEREST_PROCEED_pct   968532 non-null  float64
 33  RENT_PROCEED_pct                968532 non-null  float64
 34  REFINANCE_EIDL_PROCEED_pct      968532 non-null  float64
 35  HEALTH_CARE_PROCEED_pct         968532 non-null  float64
 36  DEBT_INTEREST_PROCEED_pct       968532 non-null  float64
 37  PROCEED_Per_Job                 968528 non-null  float64
 38  Fraud                           968532 non-null  int64
 39  DateApproved_int                968532 non-null  int64
 40  ForgivenessDate_int             968532 non-null  int64
 41  LoanStatusDate_int              968532 non-null  int64
dtypes: float64(37), int64(5)
memory usage: 310.4 MB
```

Conclusions

The dataset has a large number of instances (over 900,000) and features (66). The features are a mix of categorical and numeric. There are also some string features such as names or

street addresses, which will not be useful for modeling. There are also a number of protected features including gender and race, which could produce biased models.

Some numeric features like the amounts and proceeds have a skewed distribution. There are a number of binary numeric features which are bimodal

Based on the boxplots, there are outliers for almost every numeric feature. This is not surprising since there are almost 1,000,000 instances in the dataset

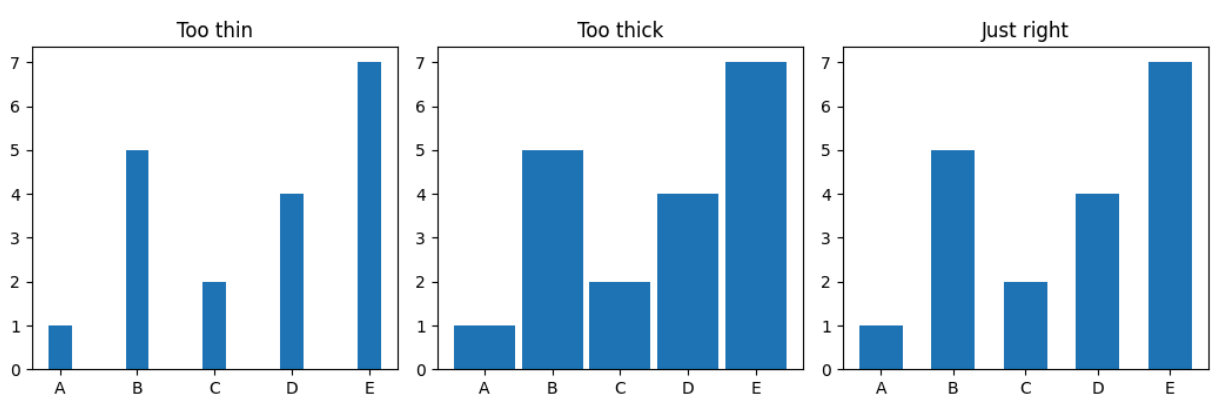
After encoding categorical and datetime features as well as dropping features which cannot be used for modeling, we are 42 usable features.

4. Storytelling With Data graph

Reproduce any graph of your choice in p. 52-68 of the Storytelling With Data book as best you can. (The second half of chapter two). You do not have to get the exact data values right, just the overall look and feel.

In [103…

```
# Figure 2.14, page 53
categories = ['A', 'B', 'C', 'D', 'E']
heights = [1,5,2,4,7]
num_cols = 3
num_rows = 1
plt.figure(figsize=(num_cols * 3.5, num_rows * 3.5))

plt.subplot(1, 3, 1)
plt.bar(x=categories, height=heights, width=.3)
plt.title("Too thin")

plt.subplot(1, 3, 2)
plt.bar(x=categories, height=heights, width=.9)
plt.title("Too thick")

plt.subplot(1, 3, 3)
plt.bar(x=categories, height=heights, width=.6)
plt.title("Just right")

plt.tight_layout()
```



Week 5 - Bivariate Analysis

Please run the cells of the notebook as you get to them while reading

In [ ]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import matplotlib.patches as patches
```

1. Lesson: Bivariate graphs

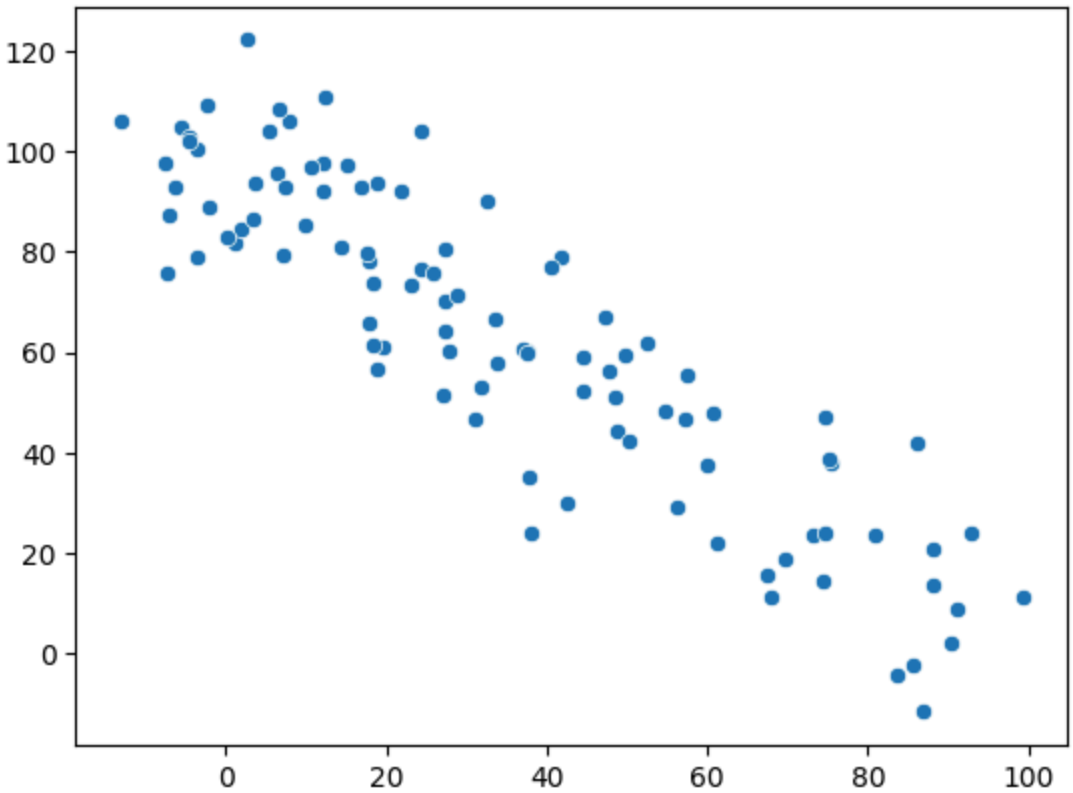
In this lesson, we'll consider some bivariate plots: scatter plots, line plots, waterfall plots, area plots, and pair plots. We'll also look at bivariate statistics such as correlations, ratios, and percentages. First, let's create some data.

In [3]:
```
np.random.seed(0)
numpoints = 100
lesson_series_uniform = np.round(np.random.random(size = numpoints) * 100, 2)
lesson_series_base = np.round(np.random.random(size = numpoints)**2 * 100, 2)
lesson_series_skewed = lesson_series_base + np.random.normal(scale = 10, size = num
lesson_series_reverse = 100 - lesson_series_base + np.random.normal(scale = 10, siz
```

This scatter plot puts one series against another. The two series are anticorrelated - one goes up when the other goes down.

In [4]:
```
sns.scatterplot(x = lesson_series_skewed, y = lesson_series_reverse)
```

Out[4]:
```
<Axes: >
```

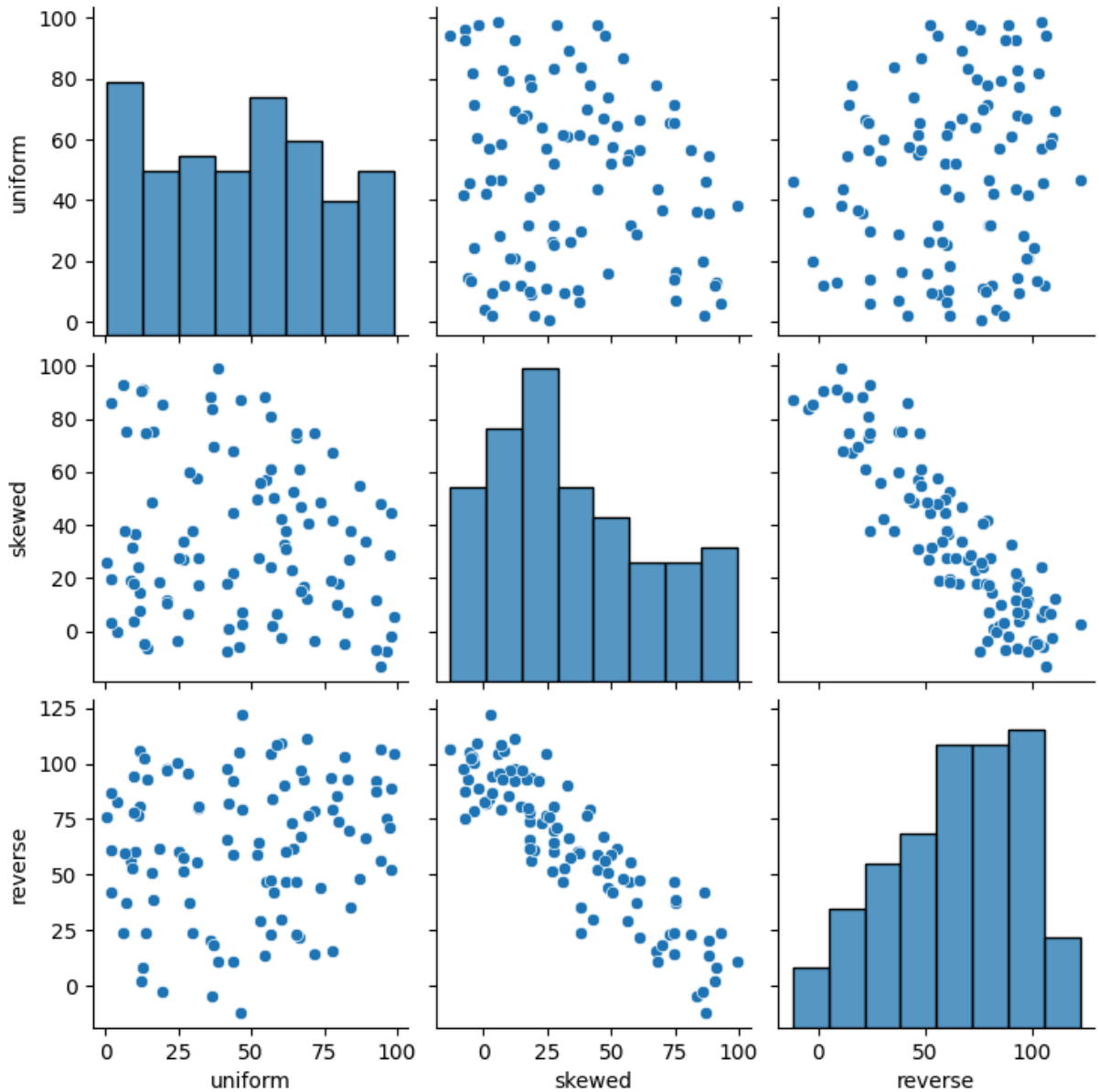


This pair plot compares three series against one another. The diagonal shows the histogram of each series, while the non-diagonal entries plot two different series against each other. The result in column 1, row 2 is a scatter plot that compares series #1 with series #2. In effect, when the logic of the pair plot tells us to plot a series against itself (column 1, row 1), we draw a histogram. (It would be silly to draw a scatter plot of a series against itself). Because series #1 is not based on the "base" series, it is not correlated with series #2 or series #3, so the four scatter plots that compare series #1 to #2 and #3 look patternless. The two scatter plots that compare series #2 with series #3 do appear to be correlated due to the "base" series that they share in common.

In [5]:
```
df = pd.DataFrame(np.array([lesson_series_uniform, lesson_series_skewed, lesson_ser
```

In [6]:
```
sns.pairplot(df)
```

Out[6]:
```
<seaborn.axisgrid.PairGrid at 0x218437a2860>
```



We can compute the correlation coefficience between each pair.

In [7]:
```
print("Correlations:", df.corr())
```

```
Correlations:           uniform    skewed   reverse
uniform  1.000000 -0.145022  0.149771
skewed  -0.145022  1.000000 -0.894549
reverse  0.149771 -0.894549  1.000000
```

This shows that there is a strong negative correlation between the "skewed" and "reverse" datasets (a larger negative number in row 2, column 3 and in row 3, column 2) and almost no correlation between the "uniform" dataset and the others (the much smaller numbers at those locations in the table.)

Q: What is a correlation telling us?

If the correlation is large (close to 1) it tells us that the two variables are usually either both large or both small in value. If we graphed them on a scatter plot, the shape would be close to a straight line, positioned diagonally from the lower left to upper right.

If the correlation is very negative (close to -1), it tells us that when one variable is large, the other is small. If we graphed them on a scatter plot, the shape would be close to a straight line, positioned diagonally from the upper left to lower right.

If the correlation is zero, it tells us that the two variables, plotted on a scatter plot, would form a cloud. The cloud can have different shape - for instance, it could be square or circular - but it cannot look like a diagonal line. Knowing the value of one variable would tell you nothing about the value of the other variable. (At least, it would not tell you whether the other variable is large or small).

The correlation coefficients above are called by the letter R. R can be positive or negative, and lies between -1 and 1. If we square R, we get a value called R^2 which is sometimes used to describe the correlation. Unless R = 0 or 1, R^2 will be smaller in magnitude than R (and always positive), which means that it is very important to know if you are talking about R or R^2. Since the values above can be negative, you can know for sure that they are R values, not R^2.

Q: How do you decide if a correlation is strong?

It depends on the application. Some sources say that a correlation of R = 0.7 or 0.8 is strong, but in other applications (say, the social sciences) a correlation of 0.5 might be considered quite strong. It is not unusual for social scientists to report correlations even as low a 0.2 as being important.

Q: What is the purpose of finding a correlation?

Here are a few:

1. Variables that are correlated with a target variable might be especially useful in predicting that variable. This might also suggest that a linear model could work. On the other hand, just because there are no pairwise correlations doesn't mean that a linear model won't work. For example, if you select X, Y, and Z from this list of four possibilities: [[X = 1, Y = 0, Z = 0], [X = 0, Y = 1, Z = 0], [X = 0, Y = 0, Z = 1], [X = 1, Y = 1, Z = 1]] then any two variables are completely uncorrelated, but the sum of the three is usually 1. So the linear model Z = 1 - X - Y is accurate in three out of four cases even though all pairwise correlations are zero.

2. Variables that are very correlated with each other might be redundant. In the extreme case, two variables that have a correlation of 1.0 are "linearly dependent" - they essentially provide the same information. For example, if one column is a height in inches, and another is a height in centimeters, then they are redundant. Moreover, if you use two highly correlated variables as predictors, the results will be somewhat

unpredictable. For example, if HI is the height in inches and HC is the height in centimeters, then the following formulas are all equivalent: HC, 2.54 * HI, 0.5 * HC + 1.27 * HI, 1001 * HC - 2540 * HI. So there's no knowing which one you'll get if you try to build a model based on these variables. The last one in particular is a little bit surprising, given the large coefficients, but it could very well be the result of a linear regression analysis.

3. Correlation can help you to detect problems with the data. If two variables are correlated but shouldn't be, or if they're not correlated but should be, there might be something wrong with the data. It's important to test the data to see if it makes sense.

4. Correlations can be interesting things to report. If your client or organization is asking for a verbal explanation of the dataset, they might be interested in knowing about any correlations for reasons of their own.

A waterfall chart shows a value after a sequence of inputs and outputs.

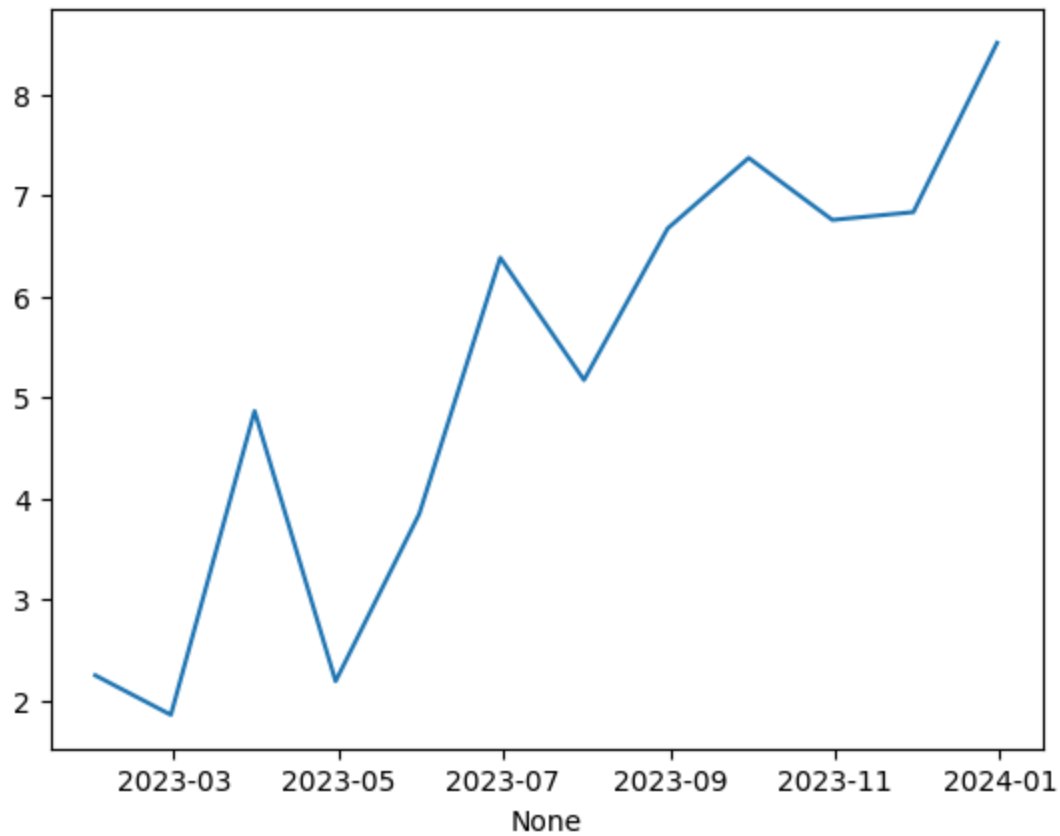
In [8]:
```
import plotly.graph_objects as go

fig = go.Figure(go.Waterfall(
    name='This title is not displayed',
    orientation='v',
    measure=['absolute','relative','relative','relative','relative'],
    x=["2023-Jan", "receipts-in", "interest-in", "expenses-out", "interest-out", "2
    textposition='outside',
    text=['Total','after-receipts','after-interest','after-expenses','after-interes
    y=[123.45,42.08,21.97,-54.31,-50.23],
    connector=dict(line=dict(color='rgb(63, 63, 63)'))
))

fig.update_layout(
    title='Waterfall Plot',
    showlegend=False
)
fig.update_yaxes(range=[-1,230])
fig.show()
```

A line plot is typically associated with a time or date range. It shows how a variable rises and falls over a period of time.

In [9]:
```
numdates = 12
time_series = np.random.random(size = numdates) * 5 + np.arange(numdates) / 3
datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
sns.lineplot(x = datearray, y = time_series)
```

```
C:\Users\caleb\AppData\Local\Temp\ipykernel_14800\2082488306.py:3: FutureWarning:

'M' is deprecated and will be removed in a future version, please use 'ME' instead.
```

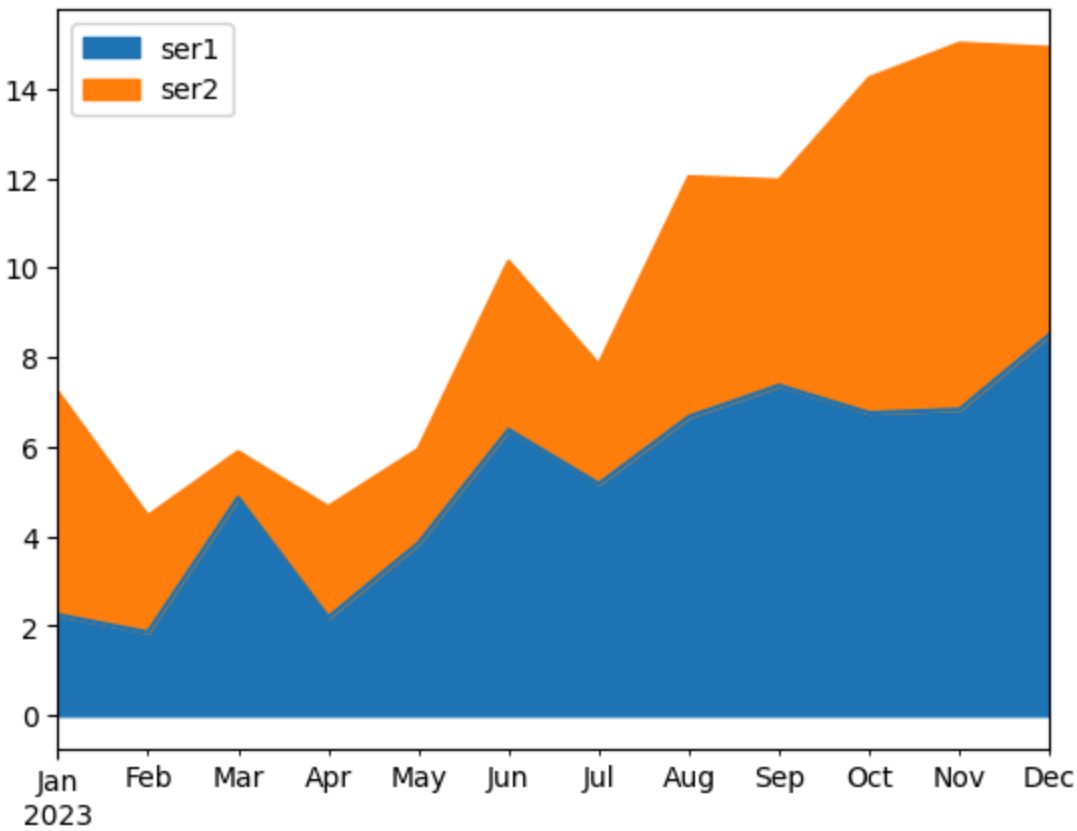
Out[9]:
```
<Axes: xlabel='None'>
```



An area plot stacks one curve on top of another. In the graph below, the top line shows the sum of the two series (ser1 + ser2) while the bottom line just shows ser1.

In [10]:
```
time_series_2 = np.random.random(size = numdates) * 5 + np.arange(numdates) / 3
df = pd.DataFrame({'ser1': time_series, 'ser2': time_series_2}, index = datearray)
df.plot.area()
```

Out[10]: <Axes: >



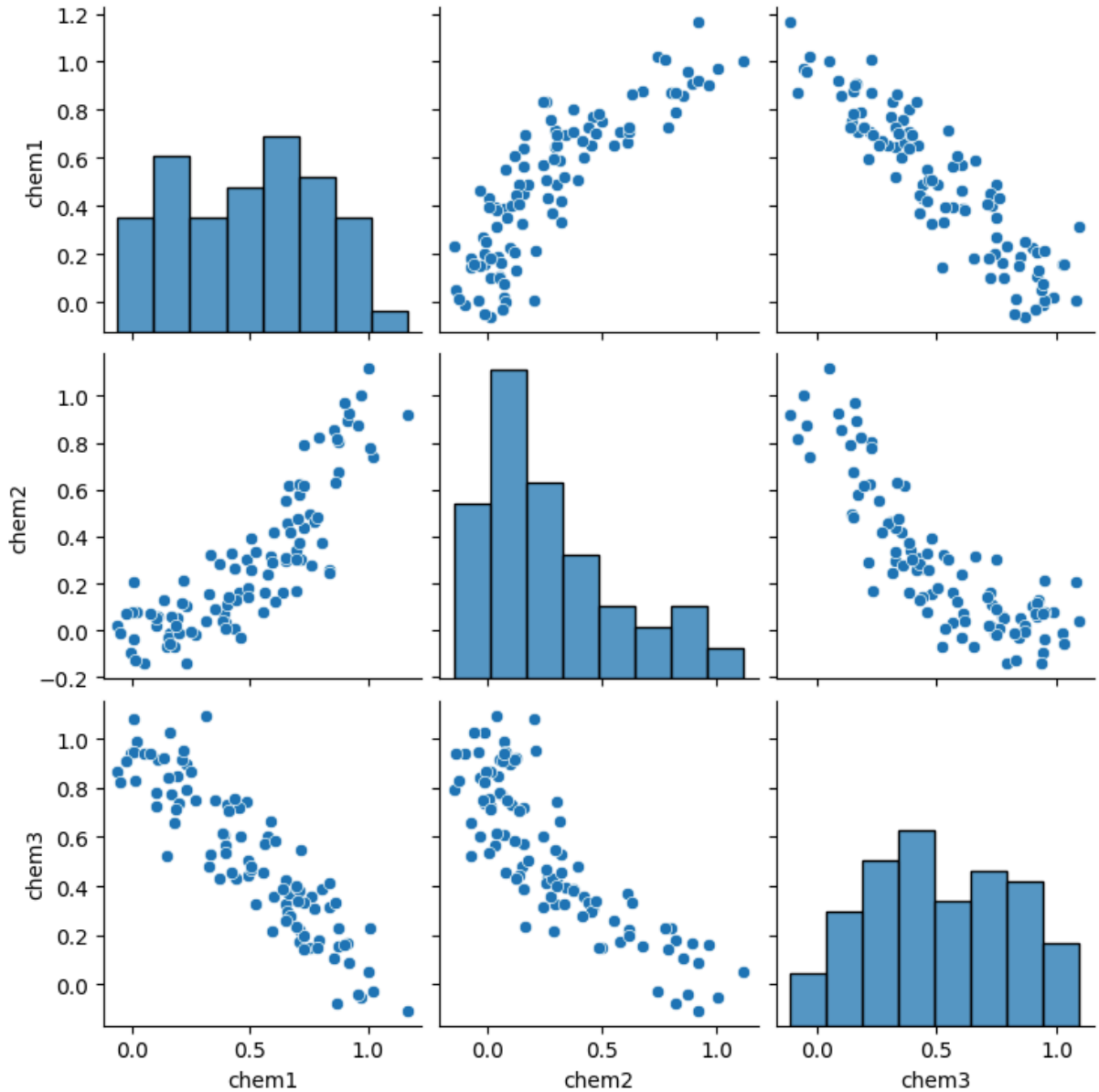
2. Weekly graph question

Below is a pair plot showing the correlation between the levels of chemical A, B, and C in a water sample. If you had to choose just *three* of the nine graphs in the pair plot to display, which would you choose? How would you alter the way these graphs are displayed to make them clearer or more informative? Would you fit any of these graphs to a line or curve? Why or why not? What if you had to choose just *one*? What if you had to choose *six*?

In [11]:
```
import numpy as np
import pandas as pd
import seaborn as sns

np.random.seed(0)
num_data = 100
base_data = np.random.random(size = num_data)
chem1 = base_data + np.random.normal(size = num_data) * 0.1
chem2 = base_data**2 + np.random.normal(size = num_data) * 0.1
chem3 = 1 - base_data + np.random.normal(size = num_data) * 0.1
df_chem = pd.DataFrame({'chem1':chem1,'chem2':chem2,'chem3':chem3})
```

In [12]:
```
sns.pairplot(df_chem)
```

Out[12]:
```
<seaborn.axisgrid.PairGrid at 0x21847fa5150>
```



If I had to chose 3 of the graphs, I would take the pair plot between chem1/chem2, chem1/chem3, and chem2/chem3. These three cover the relationship between each of the features. Although it would be nice to additionally have the histograms of each feature and the opposite pair plots, this gives us enough data to see the correlations.

If I had to chose only 1 of the graphs, I would chose the pair plot between chem1/chem3. This plot shows a strong linear correlation between chem1 and chem3. This is the most relevant pairwise combination since it is an easy to model linear relationship between the two features. Though the other combinations are also strng, they are non-linear.

If I had to chose 6 of the graphs, I would take the three histograms (chem1/chem1, chem2/chem2, chem3/chem3) and the pair plots between chem1/chem2, chem1/chem3, and chem2/chem3. This covers the linear relationships between each feature, as well as the distribution of each feature.

how would you alter the way these graphs are displayed to make them clearer or more informative? To make the graphs more informative, we could color the pair plots by the target variable. This would help to see whether the target can easily be separated using a linear combination of two features.

The combination of chem1/chem3 can be fit to a line. We can see a strong linear relationship between the values. chem2/chem3 can be fit to a curve. It has a strong relationship but has a curve to it.

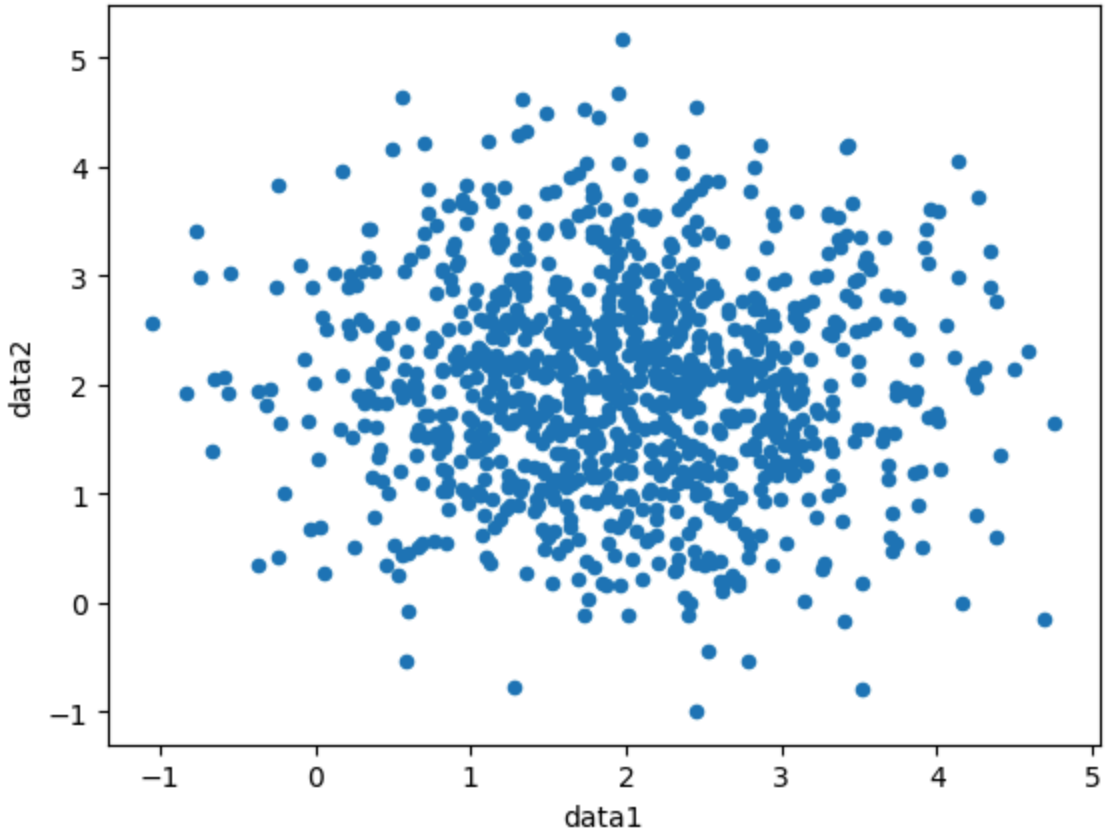
3. Homework - Reproducing Pair Plot Graphs

I'm going to show you some Pair Plot (Scatter and Histogram) graphs. I'd like you to generate some data that will generate those graphs.

Example: this cloud involves a match between two normal distributions.

In [13]:
```
num_points = 1000
np.random.seed(0)
data1 = np.random.normal(size = num_points) + 2
data2 = np.random.normal(size = num_points) + 2
df_twonorm = pd.DataFrame({"data1":data1,"data2":data2})
df_twonorm.plot.scatter(x = "data1", y = "data2")
```

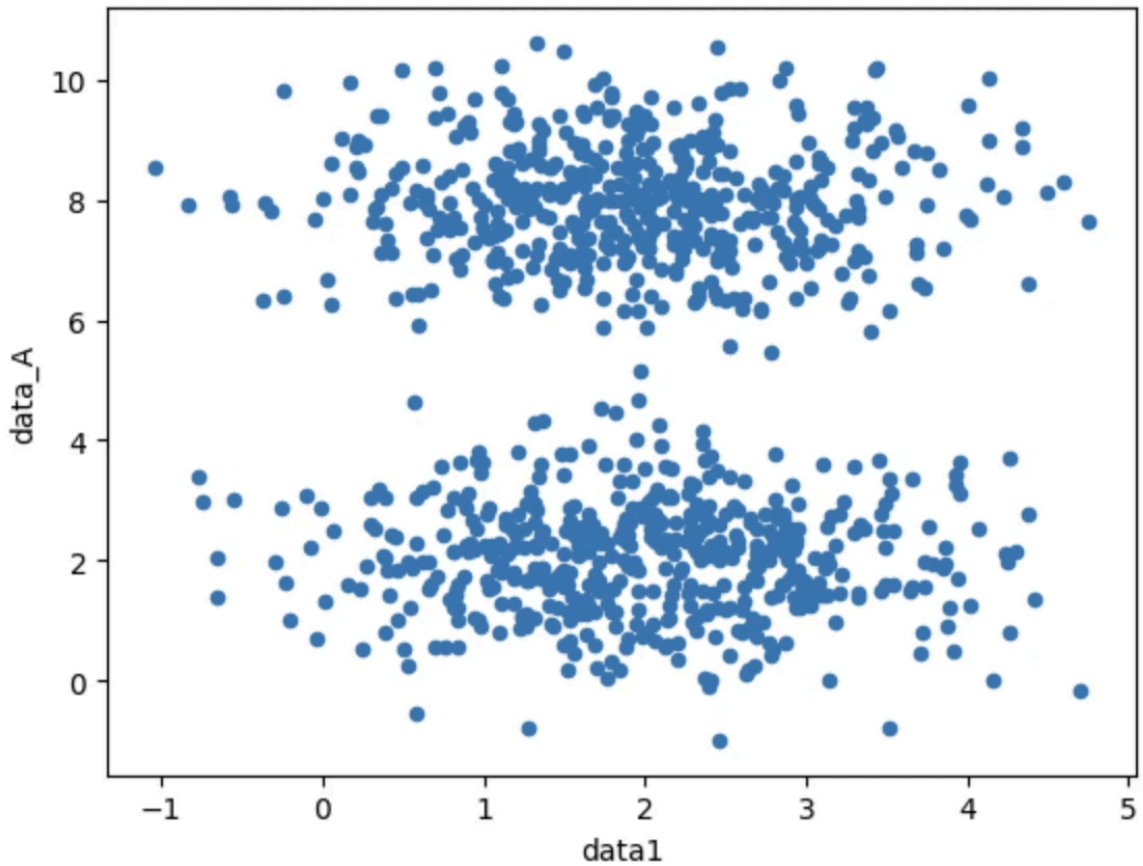
Out[13]:
```
<Axes: xlabel='data1', ylabel='data2'>
```



Problem: Can you design data that will generate the following scatter plot? You may need to join multiple series to make one big series for the y-axis data. That will allow you to create the two clouds. Use Google or ChatGPT to find the function that does that. It is okay if the precise numbers are not re-created; just the general shape: two vertically separated clouds of points.

In [14]:
```
from IPython.display import Image
Image("twoclouds.png")
```

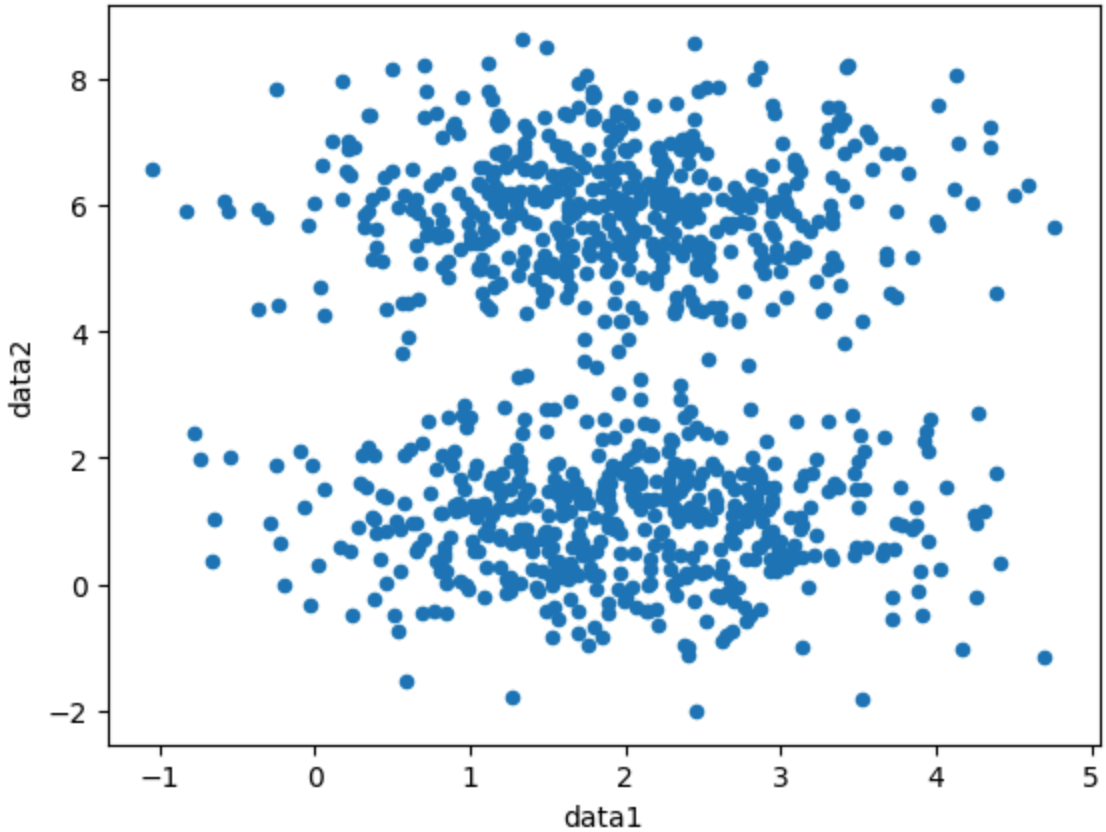
Out[14]:



In [15]:

```
num_points = 1000
np.random.seed(0)
data1 = np.random.normal(size = num_points) + 2
data2_1 = np.random.normal(size = (num_points // 2)) + 1
data2_2 = np.random.normal(size = (num_points // 2)) + 6

data2 = np.concat([data2_1, data2_2])
df_twonorm = pd.DataFrame({"data1":data1,"data2":data2})
df_twonorm.plot.scatter(x = "data1", y = "data2")
```

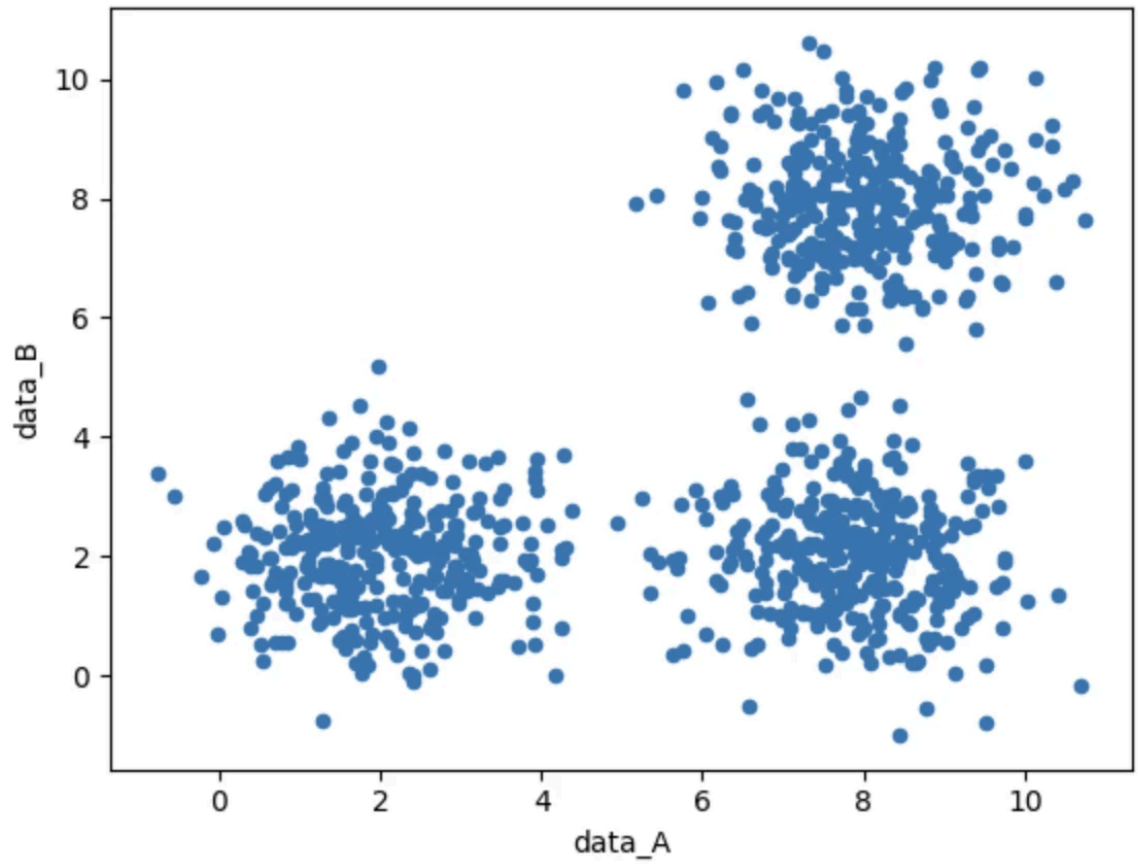
Out[15]: <Axes: xlabel='data1', ylabel='data2'>



Challenge problem! Hint: you will need two series, and three segments of the first series will connect with the three segments of the second series.

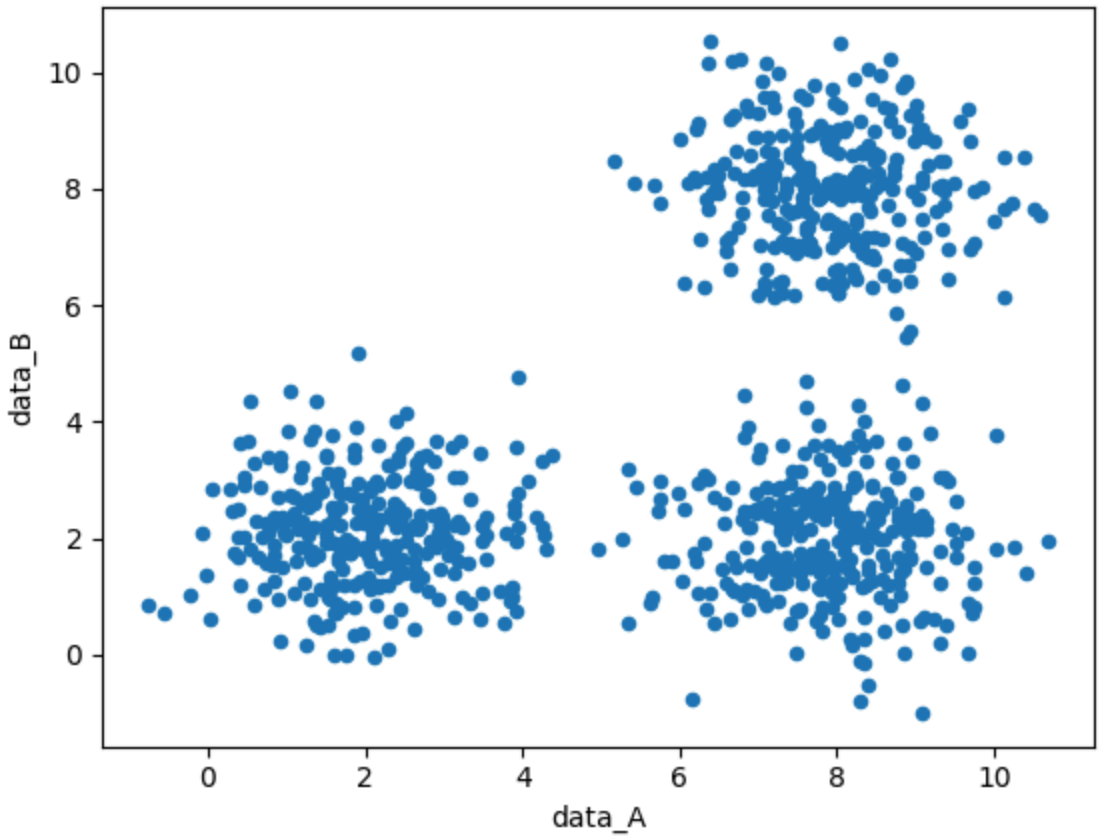
In [16]:
```
from IPython.display import Image
Image("threeclouds.png")
```

Out[16]:



In [17]:

```
num_points = 300
np.random.seed(0)
data1_1 = np.random.normal(size = num_points) + 2
data1_2 = np.random.normal(size = num_points) + 8
data1_3 = np.random.normal(size = num_points) + 8
data2_1 = np.random.normal(size = num_points) + 2
data2_2 = np.random.normal(size = num_points) + 2
data2_3 = np.random.normal(size = num_points) + 8

data1 = np.concat([data1_1, data1_2, data1_3])
data2 = np.concat([data2_1, data2_2, data2_3])
df_twonorm = pd.DataFrame({"data_A":data1,"data_B":data2})
df_twonorm.plot.scatter(x = "data_A", y = "data_B")
None
```



Would you expect to see a correlation for this challenge problem? (Hint: does knowing the value of data_A tell you anything about the value of data_B - in particular, whether it is large or small?) If you managed to reproduce the graph using data, test it yourself using the corr() function.

I would expect to see a corelation between data_A and data_B. For values of data_B between 6-10, we know that the value of data_A will be between 6-10. For lower values of data_B, there is not much of a correlation since there are two distinct clouds.

In [18]:
```
df_twonorm.corr()
```

Out[18]:

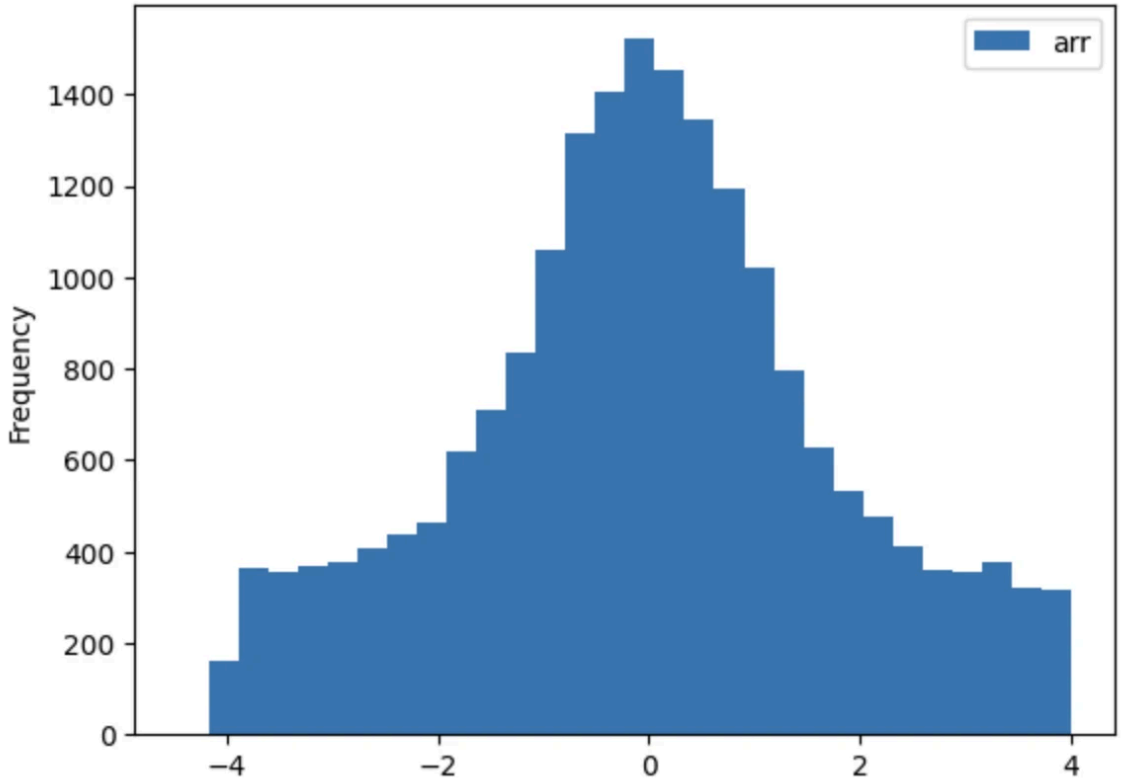
	data_A	data_B
data_A	1.000000	0.433109
data_B	0.433109	1.000000

Here is a histogram. Can you provide data that will approximately re-create this histogram?

It is okay if the precise numbers are not re-created; just the general shape: tailing off on either side, but tailing off to a nonzero number. (In this case, about 300).

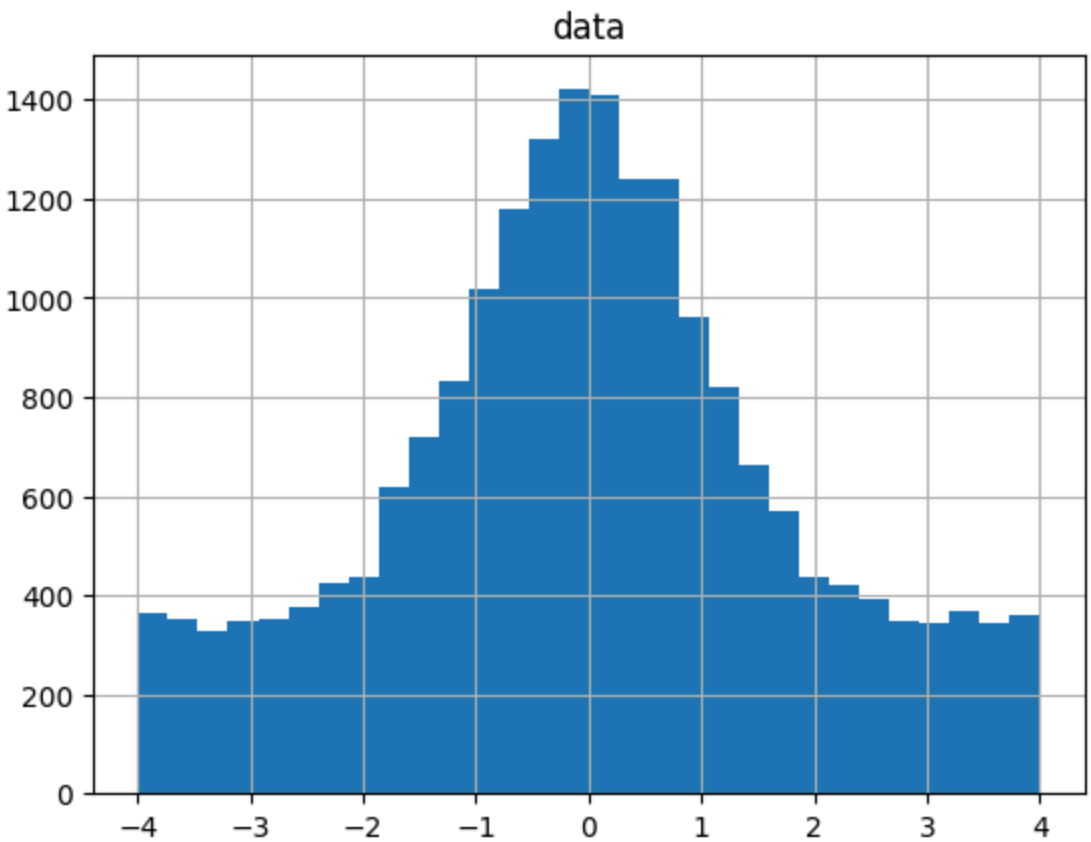
In [19]:
```
from IPython.display import Image
Image("histogram.png")
```

Out[19]:



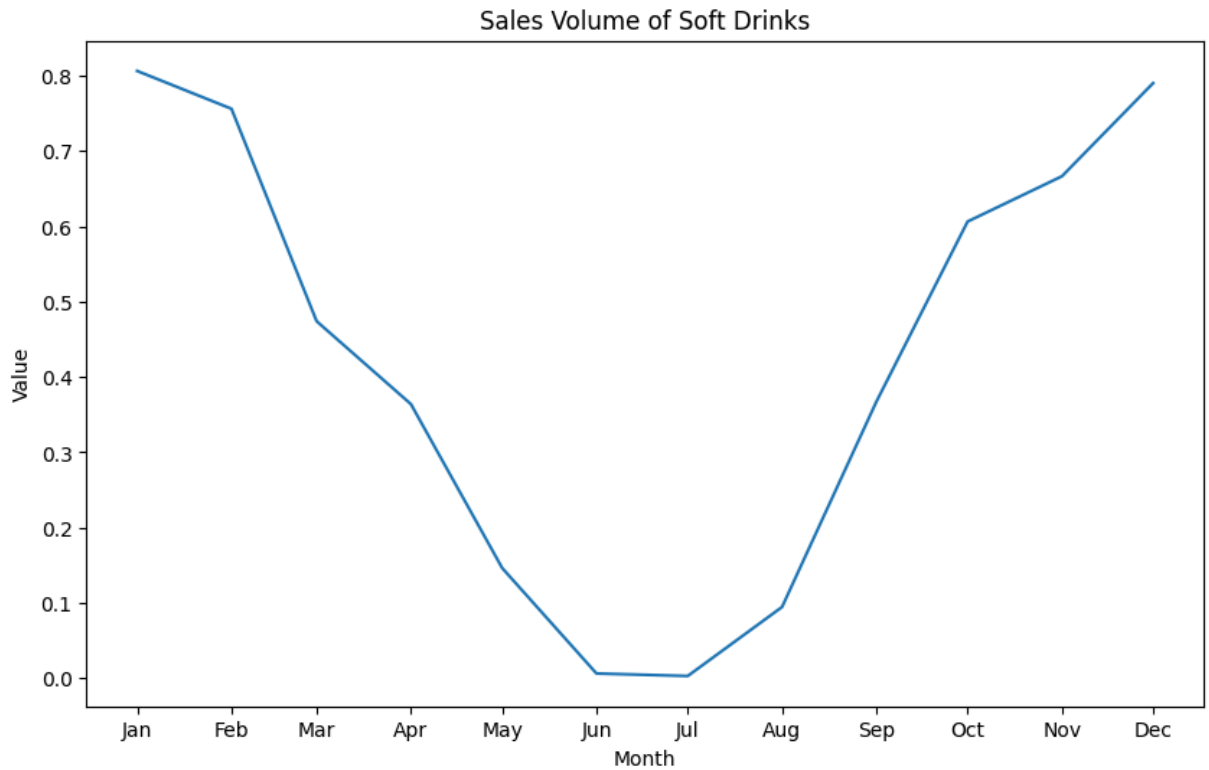
In [20]:

```
num_points = 10000
data = np.random.normal(size = num_points)
uniform = np.random.uniform(low=-4, high=4, size=10000)
data = np.concat([data, uniform])
df = pd.DataFrame({"data":data})
df.hist(bins=30)
None
```



Create a line plot that shows a sales volume of soft drinks that decreases, then increases over the course of a year, in a U-shaped curve, but with some noise.

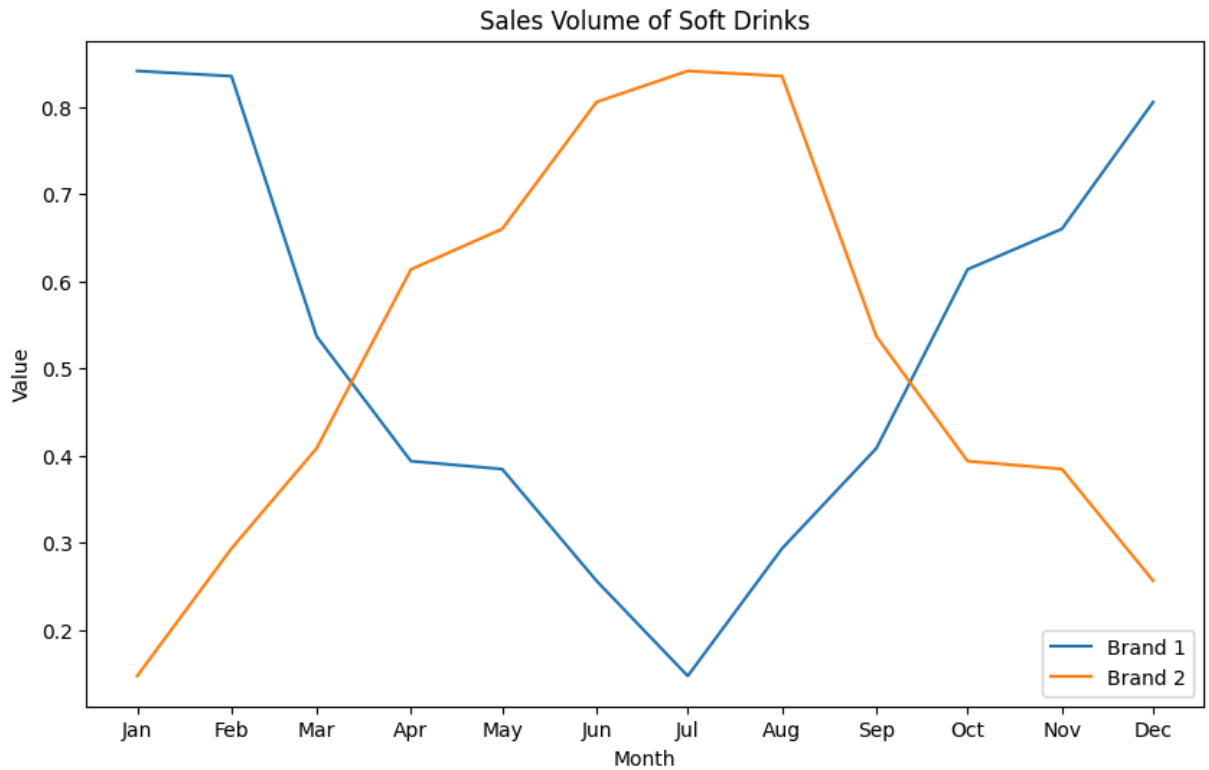
In [21]:
```
dates = pd.date_range(start='2026-01-01', periods=12, freq='MS')

# To get the U shape, we need to start high, decrease, then increase again. Since t
values_asc = np.sort(np.random.rand(6))
values_desc = np.sort(np.random.rand(6))[::-1]
values = np.concat([values_desc, values_asc])

_, ax = plt.subplots(figsize=(10, 6))
ax.plot(dates, values)

# Use months for xticks
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
# Format xticks as abbreviated month
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b'))

plt.xlabel('Month')
plt.ylabel('Value')
plt.title('Sales Volume of Soft Drinks')
None
```



Create an area plot that shows the sales volume for two brands of soft drinks, one of which is increasing then decreasing, the other decreasing then increasing.

In [22]:
```
dates = pd.date_range(start='2026-01-01', periods=12, freq='MS')

# To get the U shape, we need to start high, decrease, then increase again. Since t
values_asc = np.sort(np.random.rand(6))
values_desc = np.sort(np.random.rand(6))[::-1]
values = np.concat([values_desc, values_asc])
values2 = np.concat([values_asc, values_desc])

_, ax = plt.subplots(figsize=(10, 6))
ax.plot(dates, values, label="Brand 1")
ax.plot(dates, values2, label="Brand 2")

# Use months for xticks
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
# Format xticks as abbreviated month
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b'))

plt.xlabel('Month')
plt.ylabel('Value')
plt.legend()
plt.title('Sales Volume of Soft Drinks')
```

Out[22]:
```
Text(0.5, 1.0, 'Sales Volume of Soft Drinks')
```

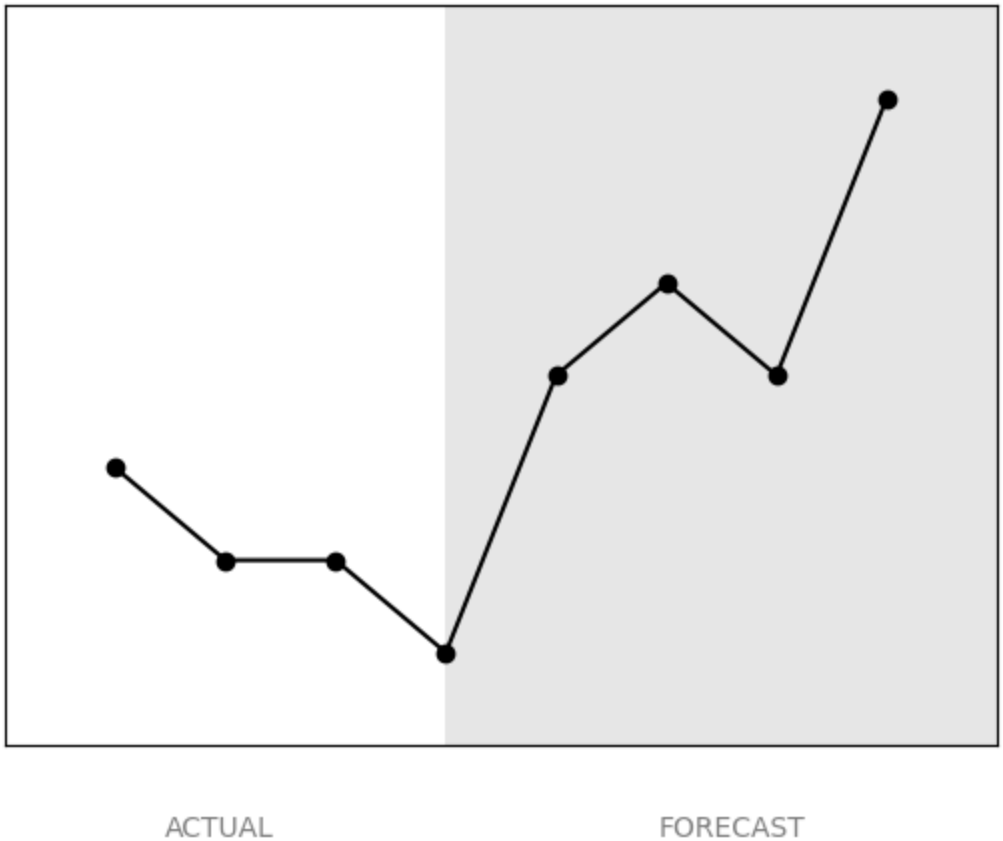


4. Storytelling With Data plot

Reproduce any graph of your choice in p. 71-85 of the Storytelling With Data book as best you can. (The first half of chapter three).

In [ ]:
```
# Figure 3.6 (Page 77)

x = np.arange(1, 9)
y = np.array([
    3,2,2,1,4,5,4,7
])
fig, ax = plt.subplots()
ax.plot(x,y, marker='o', color='black')

plt.xticks([])
plt.yticks([])
plt.ylim(0,8)
plt.xlim(0,9)

rect = patches.Rectangle((4, 0), 5, 8, linewidth=2, facecolor='lightgray', alpha=0.
ax.add_patch(rect)

fig.text(0.25, 0.01, 'ACTUAL', ha='left', va='bottom', fontsize=10, color='gray')
fig.text(0.75, 0.01, 'FORECAST', ha='right', va='bottom', fontsize=10, color='gray'
```

Out[ ]:
```
Text(0.75, 0.01, 'FORECAST')
```


ACTUAL
FORECAST

Week 6 - Bivariate Analysis, part 2

1. Lesson: no lesson this week

2. Weekly graph question

Suppose you wanted to show the nitrate and phosphate level in a water sample. What are the advantages and disadvantages of showing this as an area plot, as opposed to two separate line graphs? How would you adjust the graphs shown to improve the presentation?

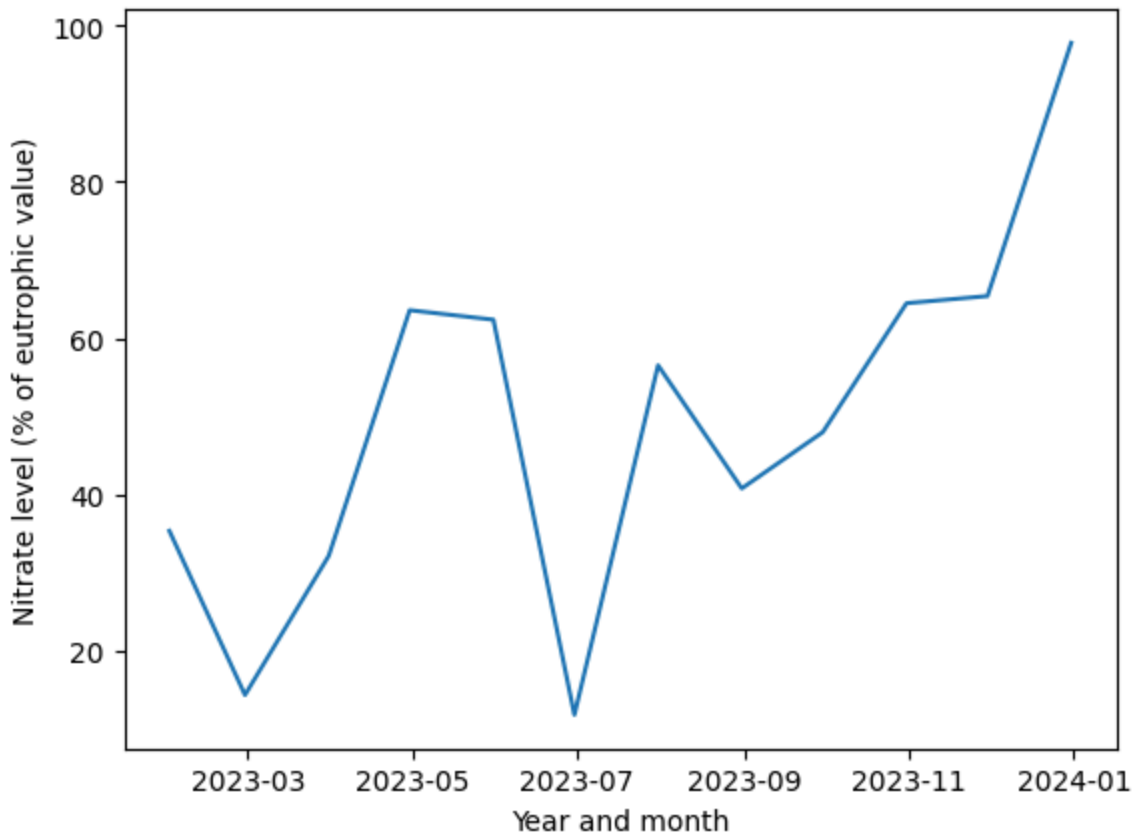
Although the area plot perserves the first plot's data, the second plot's visuals are not as clear. The y-axis, which represented percents previously, is now showing values over 100% for the second plot since it considers 0 to be where the first plot is. This makes it difficult to visualize the percentages of the second plot. One improvement would be to add labels to each individual point on each line.

In [1]:
```
import numpy as np
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

In [3]:
```
numdates = 12
np.random.seed(0)
time_series = 100 * (np.random.normal(size = numdates) / 5 + np.arange(numdates) /
datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
ax = sns.lineplot(x = datearray, y = time_series)
ax.set_xlabel("Year and month")
ax.set_ylabel("Nitrate level (% of eutrophic value)")
```

```
C:\Users\caleb\AppData\Local\Temp\ipykernel_5200\2149468501.py:4: FutureWarning: 'M'
is deprecated and will be removed in a future version, please use 'ME' instead.
  datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
```

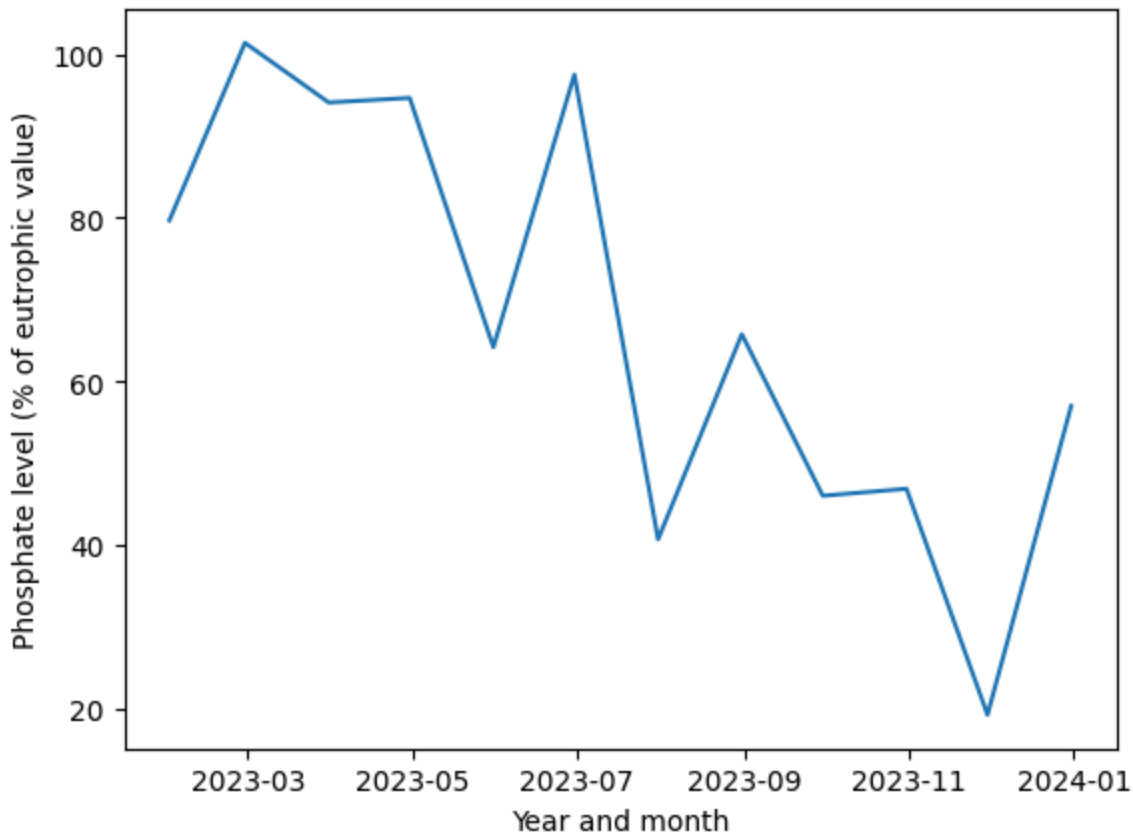
Out[3]:
```
Text(0, 0.5, 'Nitrate level (% of eutrophic value)')
```



In [4]:
```
np.random.seed(1)
time_series_2 =  100 * (1 - (np.random.normal(size = numdates) / 8 + np.arange(numd
datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
ax = sns.lineplot(x = datearray, y = time_series_2)
ax.set_xlabel("Year and month")
ax.set_ylabel("Phosphate level (% of eutrophic value)")
```

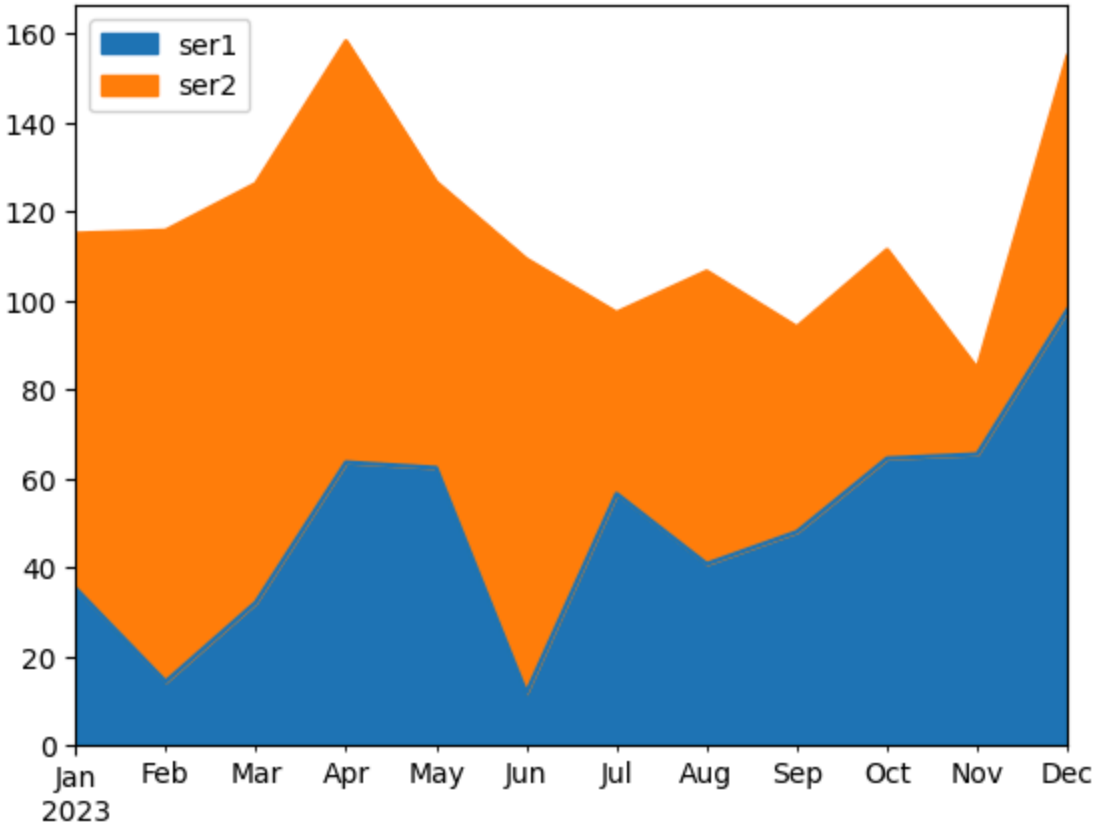
```
C:\Users\caleb\AppData\Local\Temp\ipykernel_5200\1726504607.py:3: FutureWarning: 'M'
is deprecated and will be removed in a future version, please use 'ME' instead.
  datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
```

Out[4]:
```
Text(0, 0.5, 'Phosphate level (% of eutrophic value)')
```



In [5]:
```
df = pd.DataFrame({'ser1': time_series, 'ser2': time_series_2}, index = datearray)
df.plot.area()
```

Out[5]: <Axes: >



3. Working on your datasets

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do:

- Find correlations between pairs of variables.
- Draw scatterplots, especially when the correlation is large.
- Draw pairplots.
- Draw line graphs and/or area graphs when there is date or time data together with numerical data.

Conclusions:

- Explain what conclusions you would draw from this analysis: are the data what you expect? Are the data likely to be usable? If the data are not useable, find some new data!
- Do you see any outliers? (Data points that are far from the rest of the data).

- Are any data items highly correlated with each other, suggesting that they are redundant?
- For the line plots, do you see a trend or pattern over time? Does this suggest that the data are changing over time (drifting) in such a way as to invalidate comparisons?
- Can you think of any confounding variables? (Third variables that could explain any correlations between other variables. These third variables may or may not be reported in the dataset.)

Read Data

In [6]:
```
# https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laund
li_small_accounts_df = pd.read_csv('C:/Users/caleb/Downloads/ibm_aml/LI-Small_accou
li_small_accounts_df
```

Out[6]:

	Bank Name	Bank ID	Account Number	Entity ID	Entity Name
0	China Bank #2820	314693	81B86A280	800D8CCF0	Corporation #41344
1	France Bank #4585	311253	8187FEA80	800B505E0	Corporation #54497
2	China Bank #2242	39996	803961E00	800D03F60	Partnership #36904
3	National Bank of Newport	331440	81B075800	801567C10	Corporation #16224
4	UK Bank #33	135417	80CF87C80	801085E00	Partnership #72930
...	...	...	...	...	...
712683	China Bank #42	692	80346A5F0	800D080A0	Corporation #40025
712684	First Bank of Watertown	118699	80B802A70	8005319C0	Partnership #14803
712685	Bank of Lincoln	213123	80847BE70	800453480	Partnership #11893
712686	Hilltop Credit Union	18747	803A36270	8003B5120	Partnership #6726
712687	Savings Bank of Helena	25242	80B66B2F0	8003122E0	Corporation #15281

712688 rows × 5 columns

In [7]:
```
li_small_trans_df = pd.read_csv('C:/Users/caleb/Downloads/ibm_aml/LI-Small_trans.cs
li_small_trans_df
```

Out[7]:

	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	A
0	2022/09/01 00:08	11	8000ECA90	11	8000ECA90	3.195403e+06	US Dollar	3
1	2022/09/01 00:21	3402	80021DAD0	3402	80021DAD0	1.858960e+03	US Dollar	1
2	2022/09/01 00:00	11	8000ECA90	1120	8006AA910	5.925710e+05	US Dollar	5
3	2022/09/01 00:16	3814	8006AD080	3814	8006AD080	1.232000e+01	US Dollar	1
4	2022/09/01 00:00	20	8006AD530	20	8006AD530	2.941560e+03	US Dollar	2
...	...	...	...	...	...	...	...	
6924044	2022/09/10 23:39	71696	81B2518F1	71528	81C0482E1	3.346900e-02	Bitcoin	
6924045	2022/09/10 23:48	271241	81B567481	173457	81C0DA751	1.313000e-03	Bitcoin	
6924046	2022/09/10 23:50	271241	81B567481	173457	81C0DA751	1.305800e-02	Bitcoin	
6924047	2022/09/10 23:57	170558	81A2206B1	275798	81C1D5CA1	4.145370e-01	Bitcoin	
6924048	2022/09/10 23:31	170558	81A2206B1	275798	81C1D5CA1	3.427700e-02	Bitcoin	

6924049 rows × 11 columns

In [8]:
```
hi_small_accounts_df = pd.read_csv('C:/Users/caleb/Downloads/ibm_aml/HI-Small_accou
hi_small_accounts_df
```

Out[8]:

	Bank Name	Bank ID	Account Number	Entity ID	Entity Name
0	Portugal Bank #4507	331579	80B779D80	80062E240	Sole Proprietorship #50438
1	Canada Bank #27	210	809D86900	800C998A0	Corporation #33520
2	UK Bank #33	21884	80812BE00	800C47F50	Partnership #35397
3	Germany Bank #4815	32742	81047F300	80096F0B0	Corporation #48813
4	National Bank of Harrisburg	127390	80BD8CF00	800FB8760	Corporation #889
...	...	...	...	...	...
518576	France Bank #33	3881	807886B70	80062D160	Sole Proprietorship #22257
518577	National Bank of Topeka	333423	81314C870	800F40230	Sole Proprietorship #4995
518578	Plandor Trust Bank	1467	804ED2270	800CAC4C0	Sole Proprietorship #35326
518579	Sappo Bancorp	2843	801727270	800EFCB40	Sole Proprietorship #3522
518580	Savings Bank of Montpelier	213580	804F6D9F0	800406D60	Partnership #49800

518581 rows × 5 columns

In [9]:
```
hi_small_trans_df = pd.read_csv('C:/Users/caleb/Downloads/ibm_aml/HI-Small_trans.cs
hi_small_trans_df
```

Out[9]:

	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	Ar
0	2022/09/01 00:20	10	8000EBD30	10	8000EBD30	3697.340000	US Dollar	3
1	2022/09/01 00:20	3208	8000F4580	1	8000F5340	0.010000	US Dollar	
2	2022/09/01 00:00	3209	8000F4670	3209	8000F4670	14675.570000	US Dollar	14
3	2022/09/01 00:02	12	8000F5030	12	8000F5030	2806.970000	US Dollar	2
4	2022/09/01 00:06	10	8000F5200	10	8000F5200	36682.970000	US Dollar	36
...	...	...	...	...	...	...	...	
5078340	2022/09/10 23:57	54219	8148A6631	256398	8148A8711	0.154978	Bitcoin	
5078341	2022/09/10 23:35	15	8148A8671	256398	8148A8711	0.108128	Bitcoin	
5078342	2022/09/10 23:52	154365	8148A6771	256398	8148A8711	0.004988	Bitcoin	
5078343	2022/09/10 23:46	256398	8148A6311	256398	8148A8711	0.038417	Bitcoin	
5078344	2022/09/10 23:37	154518	8148A6091	256398	8148A8711	0.281983	Bitcoin	

5078345 rows × 11 columns

Summarize Data

In [10]:
```
li_small_accounts_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 712688 entries, 0 to 712687
Data columns (total 5 columns):
 #   Column          Non-Null Count   Dtype 
---  ------          --------------   ----- 
 0   Bank Name       712688 non-null  object
 1   Bank ID         712688 non-null  int64 
 2   Account Number  712688 non-null  object
 3   Entity ID       712688 non-null  object
 4   Entity Name     712688 non-null  object
dtypes: int64(1), object(4)
memory usage: 27.2+ MB
```

In [11]:
```
li_small_trans_df.info(show_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6924049 entries, 0 to 6924048
Data columns (total 11 columns):
 #   Column              Non-Null Count    Dtype
---  ------              --------------    -----
 0   Timestamp           6924049 non-null  object
 1   From Bank           6924049 non-null  int64
 2   Account             6924049 non-null  object
 3   To Bank             6924049 non-null  int64
 4   Account.1           6924049 non-null  object
 5   Amount Received     6924049 non-null  float64
 6   Receiving Currency  6924049 non-null  object
 7   Amount Paid         6924049 non-null  float64
 8   Payment Currency    6924049 non-null  object
 9   Payment Format      6924049 non-null  object
 10  Is Laundering       6924049 non-null  int64
dtypes: float64(2), int64(3), object(6)
memory usage: 581.1+ MB
```

In [12]:
```
hi_small_accounts_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 518581 entries, 0 to 518580
Data columns (total 5 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   Bank Name       518581 non-null  object
 1   Bank ID         518581 non-null  int64
 2   Account Number  518581 non-null  object
 3   Entity ID       518581 non-null  object
 4   Entity Name     518581 non-null  object
dtypes: int64(1), object(4)
memory usage: 19.8+ MB
```

In [13]:
```
hi_small_trans_df.info(show_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5078345 entries, 0 to 5078344
Data columns (total 11 columns):
 #   Column              Non-Null Count    Dtype
---  ------              --------------    -----
 0   Timestamp           5078345 non-null  object
 1   From Bank           5078345 non-null  int64
 2   Account             5078345 non-null  object
 3   To Bank             5078345 non-null  int64
 4   Account.1           5078345 non-null  object
 5   Amount Received     5078345 non-null  float64
 6   Receiving Currency  5078345 non-null  object
 7   Amount Paid         5078345 non-null  float64
 8   Payment Currency    5078345 non-null  object
 9   Payment Format      5078345 non-null  object
 10  Is Laundering       5078345 non-null  int64
dtypes: float64(2), int64(3), object(6)
memory usage: 426.2+ MB
```

There are no null values in any of the datasets

The account dataframes show details about accounts, what bank the account belongs to, and the business entity that owns the bank account

The transactions show different types of payments between accounts

The reasoning for the datasets being split into two groups is as follows:

- Group HI has a relatively higher illicit ratio (more laundering).
- Group LI has a relatively lower illicit ratio (less laundering).

Check for Inifinite Values and Duplicates

In [14]:
```
hi_small_accounts_num_df = hi_small_accounts_df.select_dtypes(include=[np.number])

# Find rows where ANY column has an infinite value
rows_with_inf = hi_small_accounts_df[np.isinf(hi_small_accounts_num_df).any(axis=1)
rows_with_inf
```

Out[14]:

Bank Name	Bank ID	Account Number	Entity ID	Entity Name

In [15]:
```
li_small_accounts_num_df = li_small_accounts_df.select_dtypes(include=[np.number])
# Find rows where ANY column has an infinite value
rows_with_inf = li_small_accounts_df[np.isinf(li_small_accounts_num_df).any(axis=1)
rows_with_inf
```

Out[15]:

Bank Name	Bank ID	Account Number	Entity ID	Entity Name

In [16]:
```
hi_small_trans_num_df = hi_small_trans_df.select_dtypes(include=[np.number])

# Find rows where ANY column has an infinite value
rows_with_inf = hi_small_trans_df[np.isinf(hi_small_trans_num_df).any(axis=1)]
rows_with_inf
```

Out[16]:

Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	Amount Paid	Payment Currency	I

In [17]:
```
li_small_trans_num_df = li_small_trans_df.select_dtypes(include=[np.number])

# Find rows where ANY column has an infinite value
rows_with_inf = li_small_trans_df[np.isinf(li_small_trans_num_df).any(axis=1)]
rows_with_inf
```

Out[17]:

Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	Amount Paid	Payment Currency	I

There are no inf values

In [18]:
```
hi_small_accounts_df[hi_small_accounts_df.duplicated(keep=False)]
```

Out[18]:

Bank Name	Bank ID	Account Number	Entity ID	Entity Name

In [19]:
```
li_small_accounts_df[li_small_accounts_df.duplicated(keep=False)]
```

Out[19]:

Bank Name	Bank ID	Account Number	Entity ID	Entity Name

In [20]:
```
hi_small_trans_df[hi_small_trans_df.duplicated(keep=False)]
```

Out[20]:

	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	Amou Pa
863293	2022/09/01 16:20	12004	800C927C1	12004	800C927C0	0.000008	Bitcoin	0.0800
863294	2022/09/01 16:20	12004	800C927C1	220	813D8C1E1	0.000008	Bitcoin	0.0000
863295	2022/09/01 16:20	12004	800C927C1	12004	800C927C0	0.000008	Bitcoin	0.0800
863296	2022/09/01 16:20	12004	800C927C1	220	813D8C1E1	0.000008	Bitcoin	0.0000
3684003	2022/09/07 21:25	29992	8099A29B1	220	813725AE1	0.000003	Bitcoin	0.0000
3684005	2022/09/07 21:25	29992	8099A29B1	220	813725AE1	0.000003	Bitcoin	0.0000
4166784	2022/09/08 21:05	113779	811144AB1	113779	811144AB0	0.000002	Bitcoin	0.0200
4166785	2022/09/08 21:05	113779	811144AB1	53744	813C777F1	0.000002	Bitcoin	0.0000
4166786	2022/09/08 21:05	113779	811144AB1	113779	811144AB0	0.000002	Bitcoin	0.0200
4166787	2022/09/08 21:05	113779	811144AB1	53744	813C777F1	0.000002	Bitcoin	0.0000
4510478	2022/09/09 10:03	6075	80C702911	6075	80C702910	0.000002	Bitcoin	0.0200
4510479	2022/09/09 10:03	6075	80C702911	154653	814389B61	0.000002	Bitcoin	0.0000
4510480	2022/09/09 10:03	6075	80C702911	6075	80C702910	0.000002	Bitcoin	0.0200
4510481	2022/09/09 10:03	6075	80C702911	154653	814389B61	0.000002	Bitcoin	0.0000
4816510	2022/09/09 21:33	14433	80935A891	14433	80935A890	0.000001	Bitcoin	0.0100
4816511	2022/09/09 21:33	14433	80935A891	15	813F7AE61	0.000001	Bitcoin	0.0000
4816512	2022/09/09 21:33	14433	80935A891	14433	80935A890	0.000001	Bitcoin	0.0100
4816513	2022/09/09 21:33	14433	80935A891	15	813F7AE61	0.000001	Bitcoin	0.0000

In [21]:
```
li_small_trans_df[li_small_trans_df.duplicated(keep=False)]
```

Out[21]:

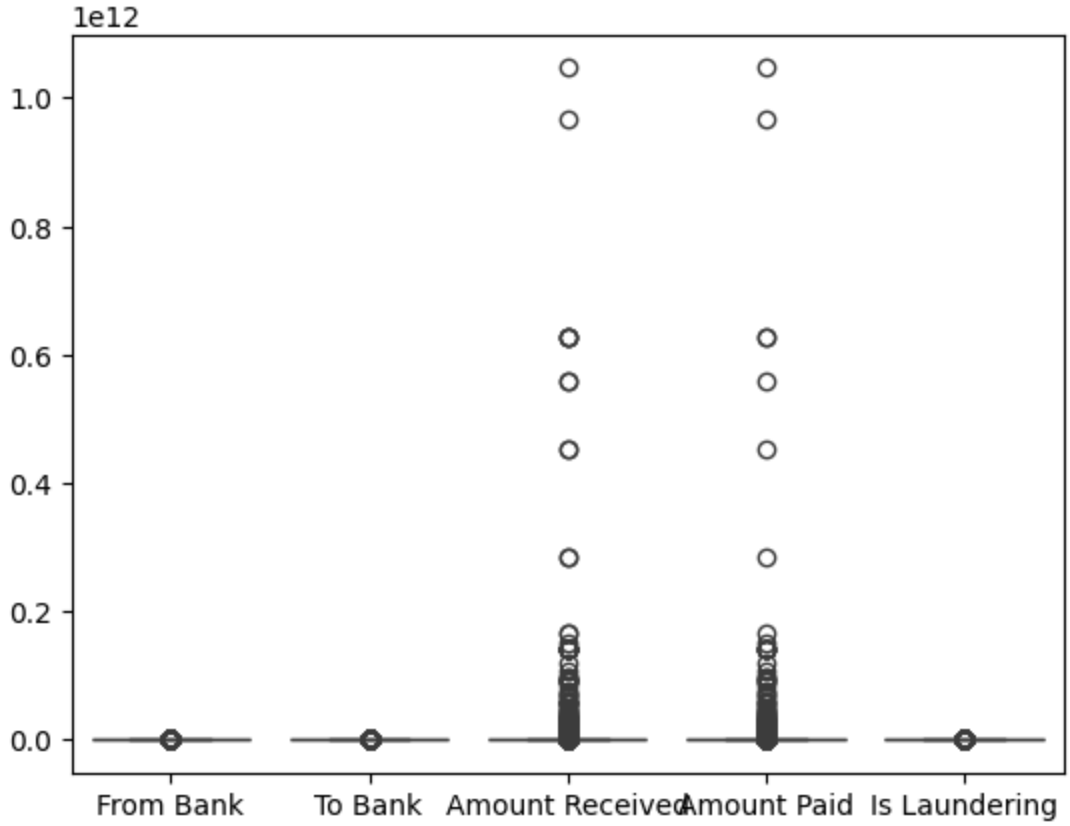
	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	Amou Pa
676694	2022/09/01 05:03	21260	8085A09D1	72146	81AB967E1	0.000001	Bitcoin	0.0000
676696	2022/09/01 05:03	21260	8085A09D1	72146	81AB967E1	0.000001	Bitcoin	0.0000
790828	2022/09/01 07:44	231616	814E45511	171546	81B324AC1	0.000001	Bitcoin	0.0000
790830	2022/09/01 07:44	231616	814E45511	171546	81B324AC1	0.000001	Bitcoin	0.0000
2027614	2022/09/02 11:09	28335	804B468F1	171669	81BBCDAB1	0.000007	Bitcoin	0.0000
2027616	2022/09/02 11:09	28335	804B468F1	171669	81BBCDAB1	0.000007	Bitcoin	0.0000
3482551	2022/09/05 12:44	22661	80D016C81	269294	81A245E81	0.000005	Bitcoin	0.0000
3482553	2022/09/05 12:44	22661	80D016C81	269294	81A245E81	0.000005	Bitcoin	0.0000
4180129	2022/09/06 14:16	19970	804E44F11	169489	8197DC971	0.000001	Bitcoin	0.0000
4180131	2022/09/06 14:16	19970	804E44F11	169489	8197DC971	0.000001	Bitcoin	0.0000
4559108	2022/09/07 03:42	113016	80A4D8561	113016	80A4D8560	0.000001	Bitcoin	0.0100
4559109	2022/09/07 03:42	113016	80A4D8561	169261	81993FDE1	0.000001	Bitcoin	0.0000
4559112	2022/09/07 03:42	113016	80A4D8561	113016	80A4D8560	0.000001	Bitcoin	0.0100
4559113	2022/09/07 03:42	113016	80A4D8561	169261	81993FDE1	0.000001	Bitcoin	0.0000
6422480	2022/09/09 17:56	11701	8034B3501	273006	81B49ADC1	0.000001	Bitcoin	0.0000
6422482	2022/09/09 17:56	11701	8034B3501	273006	81B49ADC1	0.000001	Bitcoin	0.0000

The identified duplicates are duplicate transactions. Since this just means the same amount is sent between the same accounts, I will leave these as valid samples.

Boxplots

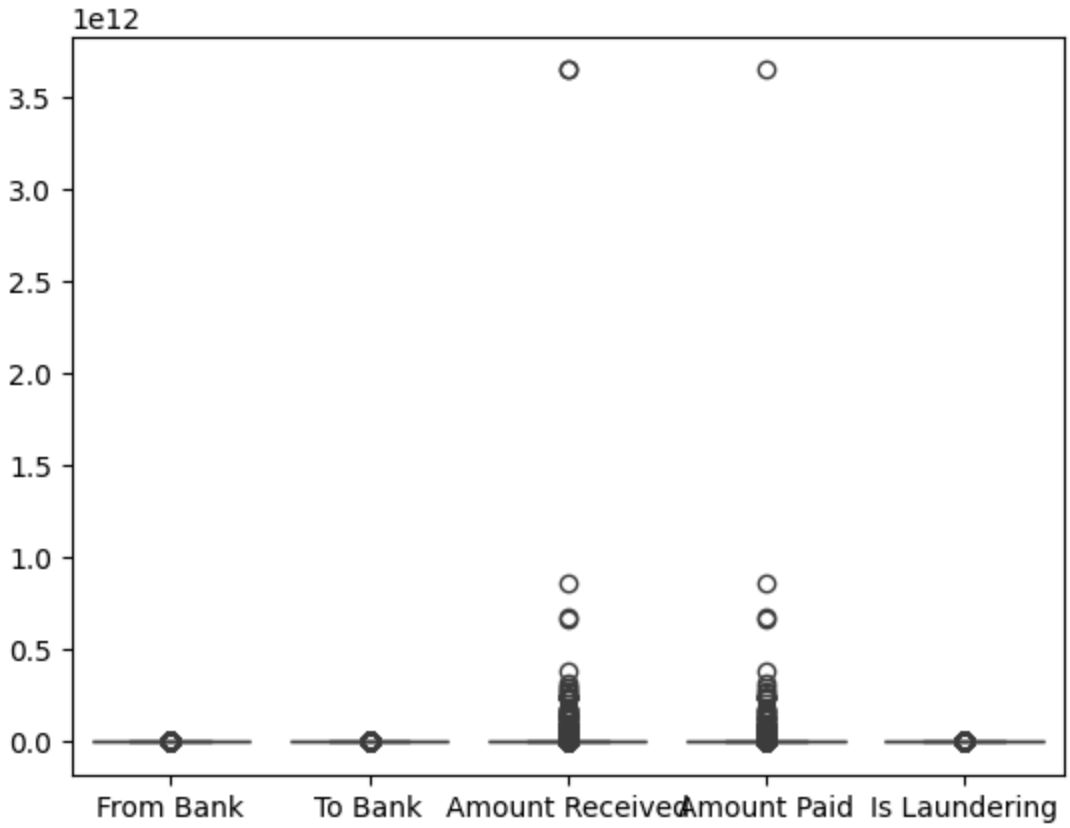
In [23]:
```
sns.boxplot(hi_small_trans_df)
```

Out[23]: <Axes: >



In [24]:
```
sns.boxplot(li_small_trans_df)
```

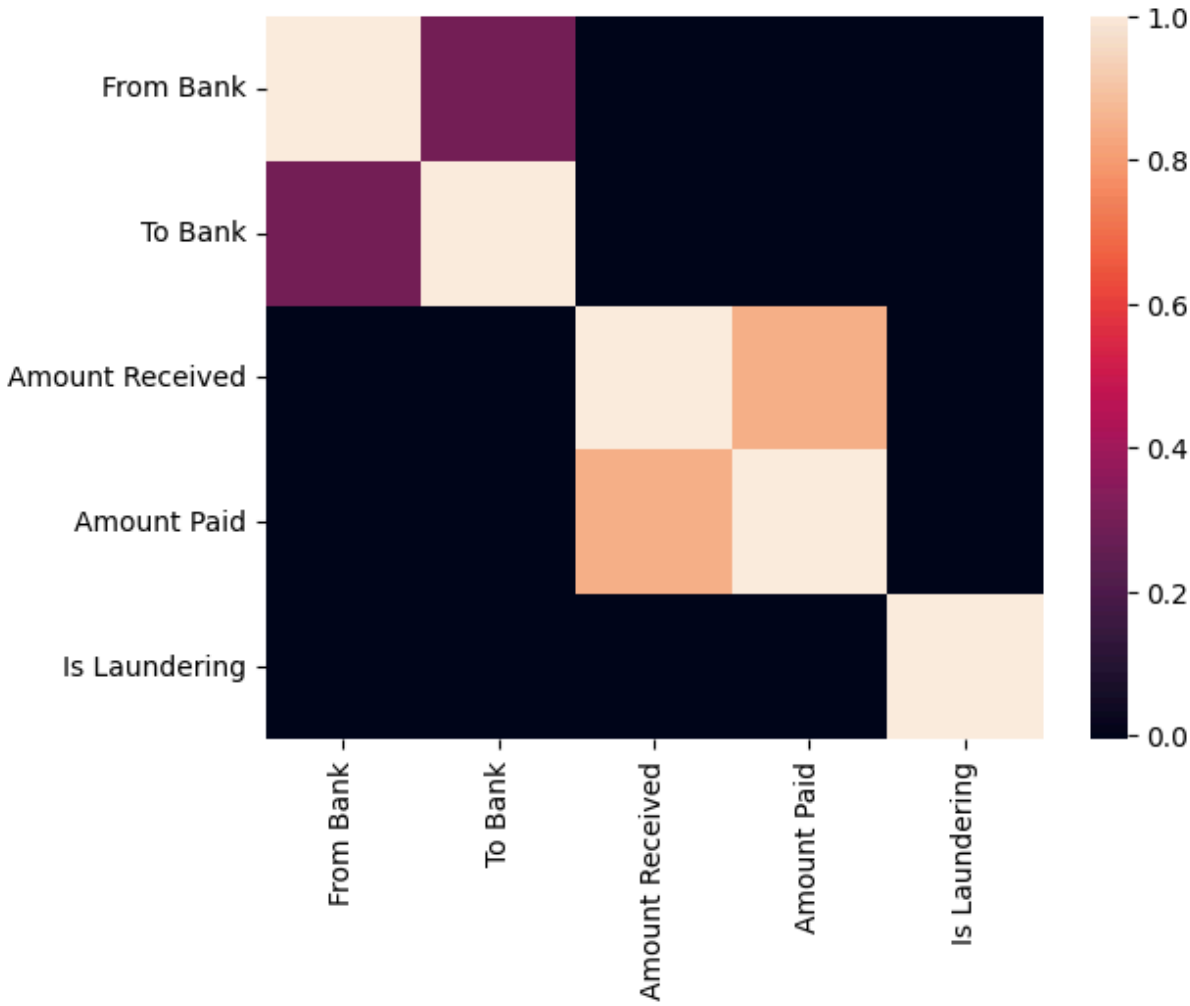
Out[24]: <Axes: >



Correlations

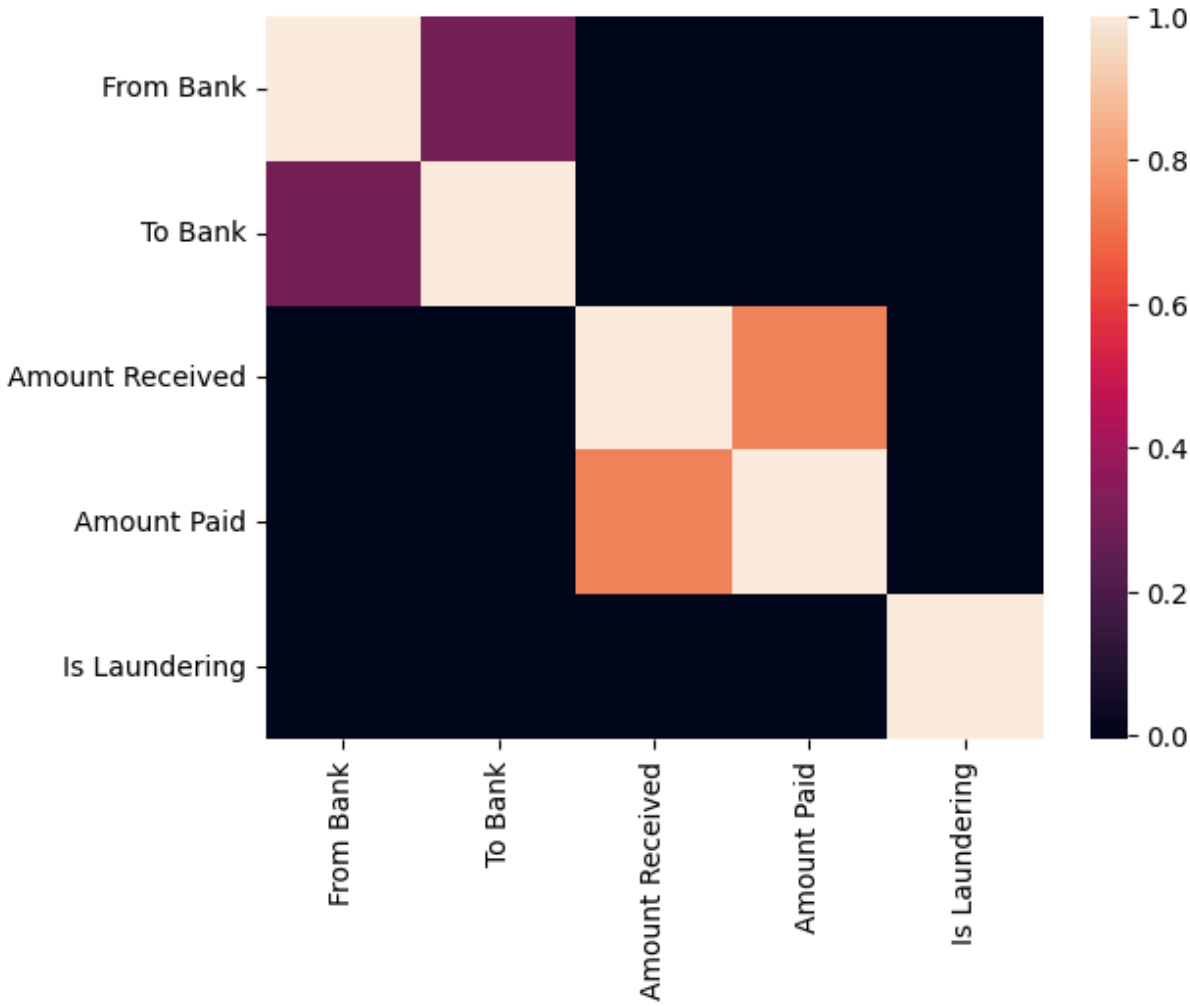
In [25]:
```
sns.heatmap(hi_small_trans_df.corr(numeric_only=True))
```

Out[25]: <Axes: >



In [26]:
```
sns.heatmap(li_small_trans_df.corr(numeric_only=True))
```

Out[26]: <Axes: >

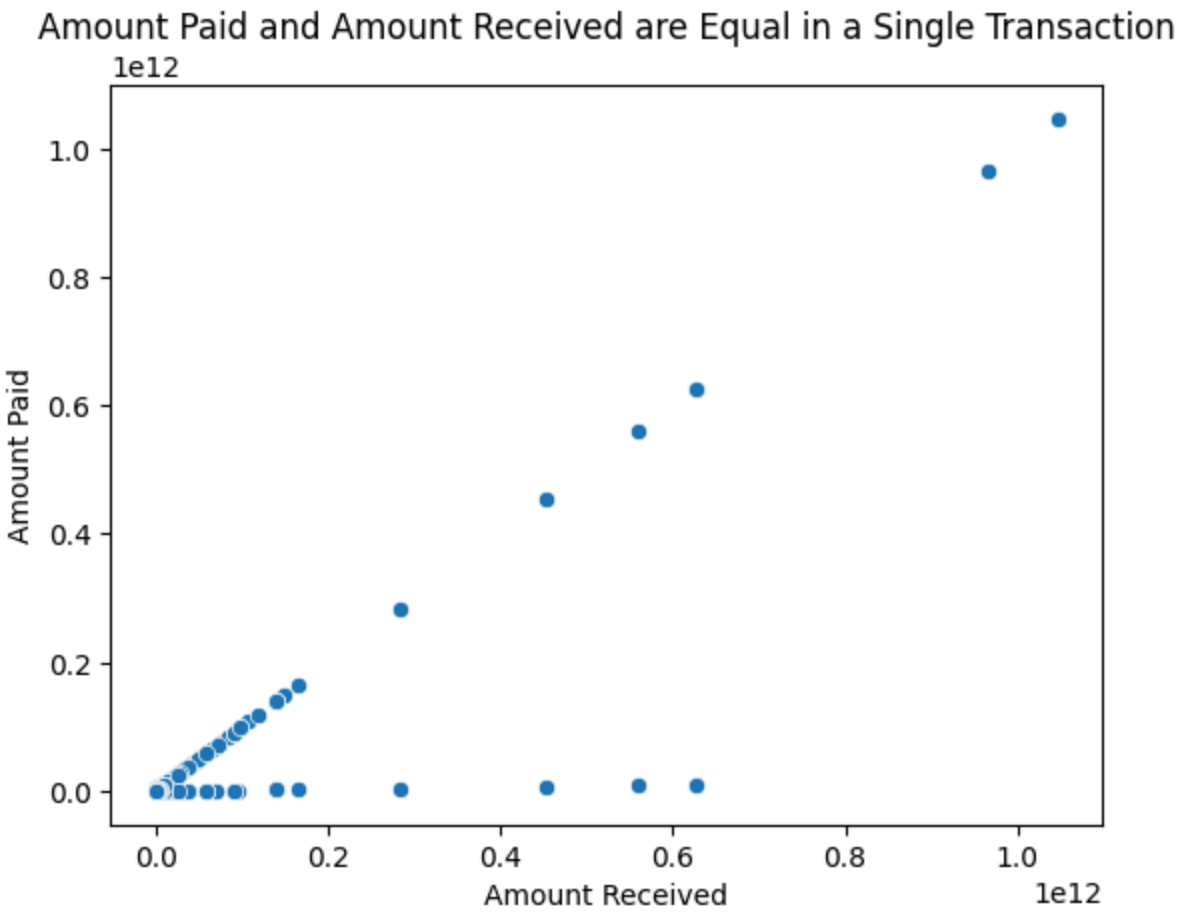


We see a strong correlation between Amount Received/Paid and To/From Bank. The other combinations do not have correlation.

The relationship between the Amounts is expected, since each transaction contains two accounts and the Received/Paid should mirror each other. We can see that in the scatterplot below.

I expect that the relationships in the data will reveal themselves not through just checking correlations, but by building a model of how transactions fit together. We need a full picture of how money flows throughout many transactions.

In [27]:
```
sns.scatterplot(x=hi_small_trans_df['Amount Received'], y=hi_small_trans_df['Amount
plt.title("Amount Paid and Amount Received are Equal in a Single Transaction")
None
```



Timeline of Transactions for a Sample Account

In [28]:
```
# Plotting a timeline of transactions that contains money laundering
hi_small_trans_df[hi_small_trans_df['Is Laundering'] == 1]
```

Out[28]:

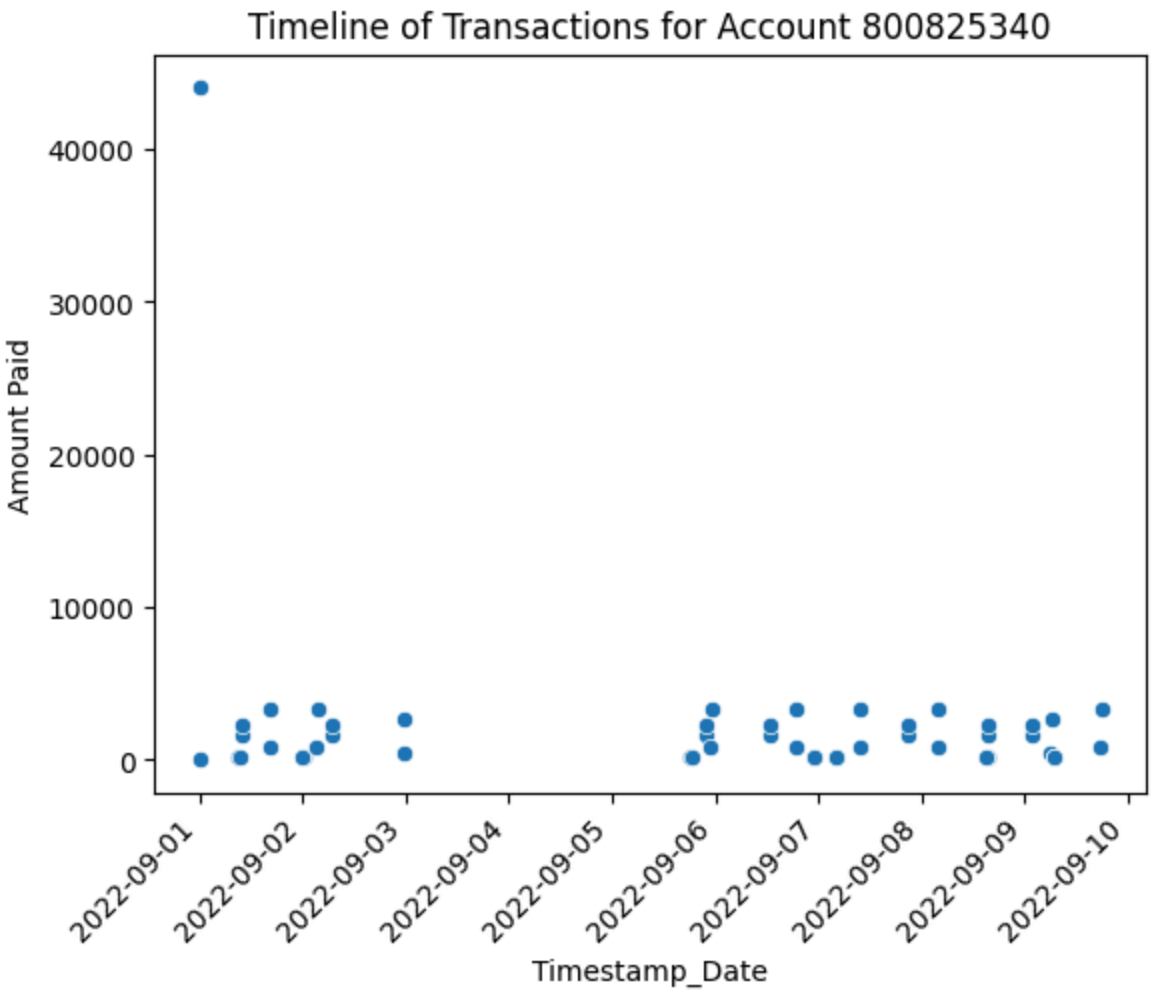
	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	
4742	2022/09/01 00:21	70	100428660	1124	800825340	389769.39	US Dollar	3
40621	2022/09/01 00:03	70	100428660	11474	805B716C0	29024.33	US Dollar	
69152	2022/09/01 00:01	70	100428660	15980	80B39E7B0	792.92	US Dollar	
85763	2022/09/01 00:03	70	100428660	113798	80DC756E0	13171425.53	US Dollar	131
90684	2022/09/01 00:23	70	100428660	32375	80E480620	14288.83	US Dollar	
...	...	...	...	...	...	...	...	
5071168	2022/09/10 23:29	111433	8065C2380	21550	8065C69E0	7773.42	US Dollar	
5073375	2022/09/10 23:07	9	80727F930	219557	8072A28C0	448920.65	Ruble	4
5074248	2022/09/12 03:34	222	812D127D0	50202	812D129C0	20085.71	Saudi Riyal	
5075056	2022/09/10 23:43	17327	8095F6C20	15695	8095F6C70	3314.36	US Dollar	
5077204	2022/09/10 23:53	3	804FAA470	212789	804FAD580	3564699.78	Rupee	35

5177 rows × 11 columns

In [ ]:

```
hi_transactions_sample = hi_small_trans_df[hi_small_trans_df['Account'] == '8008253
hi_transactions_sample['Timestamp_Date'] = pd.to_datetime(hi_transactions_sample['T
sns.scatterplot(hi_transactions_sample, x="Timestamp_Date", y="Amount Paid")
plt.xticks(rotation=45, horizontalalignment='right')
plt.title("Timeline of Transactions for Account 800825340")
None
```

```
C:\Users\caleb\AppData\Local\Temp\ipykernel_13184\1523918599.py:2: SettingWithCopyWa
rning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy
  hi_transactions_sample['Timestamp_Date'] = pd.to_datetime(hi_transactions_sample
['Timestamp'])
```



Conclusions

The IBM synthetic accounts/transactions dataset is high quality. There are no missing values to handle. There are some duplicates instances, but none that we need to drop. Additionally, there are a large number of instances we can use for analysis.

In the boxplots, we can see some outliers for the Amount Recieved/Paid features. The outliers are for large amounts. The majority of transactions are for small sums

The Amount Received/Paid features are highly correlated. This is because they are equal and opposite to each other. Having the duplicate information will be useful for building the full relationship of transactions for a given account

Looking at a sample account which had fraudulent transaction, we see a number of smaller transactions and single large transaction. One strategy that a fraudster might utilize is spending a large sum from an account they have taken over. This does not cover all possible methods though. More in depth analysis will be needed to uncover these patterns.

The largest challenge with this dataset is that feature engineering is required before analysis. There is only a small number of features. The real insights into fraud will come from building

up a full picture of the transactions for each account. From here the model can learn what types of transactions are dissimilar to normal ones. One strategy I will employ is creating an engineered feature to represent the average purchase amount for an account, as well as the amount of times two accounts have sent money to each other.

4. Storytelling With Data plot

Reproduce any graph of your choice in p. 86-98 of the Storytelling With Data book as best you can. (The second half of chapter three). You do not have to get the exact data values right, just the overall look and feel.

In [116…

```
# Figure 3.16 (p. 88)
index = [
    'Selection',
    'Service',
    'Relationship',
    'Convenience',
    'Price'
]
storytelling_with_data_df = pd.DataFrame(
    {
        'Competitor E': [10,9,5,7,7],
        'Competitor D': [6,7,4.5,5,5],
        'Competitor C': [3,2,3.2,4.5,6],
        'Competitor B': [4,3,3,4,2],
        'Competitor A': [2.2,7,1,9,6],
        'Our Business': [2,2.4,3,8,8]
    }, index=index
)
ax = storytelling_with_data_df.plot.barh(color=['grey', 'grey', 'grey', 'grey', 'gr
plt.xticks([])
ax.set_frame_on(False)
plt.title("Performance Overview", loc='left')
plt.text(1, 4.4, 'Weighted performance index |', fontsize=10, color='grey')
plt.text(5.5, 4.4, 'relative rank', fontsize=10, color='royalblue')
plt.text(8.1, 4.15, '1 of 6', fontsize=8, color='royalblue')
plt.text(8.1, 3.2, '2 of 6', fontsize=8, color='royalblue')
plt.text(3.1, 2.2, '4 of 6', fontsize=8, color='royalblue')
plt.text(2.5, 1.2, '6 of 6', fontsize=8, color='royalblue')
plt.text(2.1, 0.18, '6 of 6', fontsize=8, color='royalblue')
plt.tight_layout()
None
```

Performance Overview
Weighted performance index | relative rank
Price
1 of 6
Convenience
2 of 6
Relationship
4 of 6
Service
6 of 6
Selection
6 of 6
Competitor E
Competitor D
Competitor C
Competitor B
Competitor A
Our Business

Week 7 - Milestone One week

1. Lesson: no lesson this week. Milestone One is due!

2. Weekly graph question

Below are the graphs from last week's weekly graph question. Suppose you wanted to cause the viewer to examine the last three months of data first. How could you use preattentive attributes to do that? Suggest several possibilities.

One attribute to use is leveraging color to draw attention. By using a color like blue for the last 3 months of the line, and grey for the rest, it will draw the users eyes to the period of analysis.

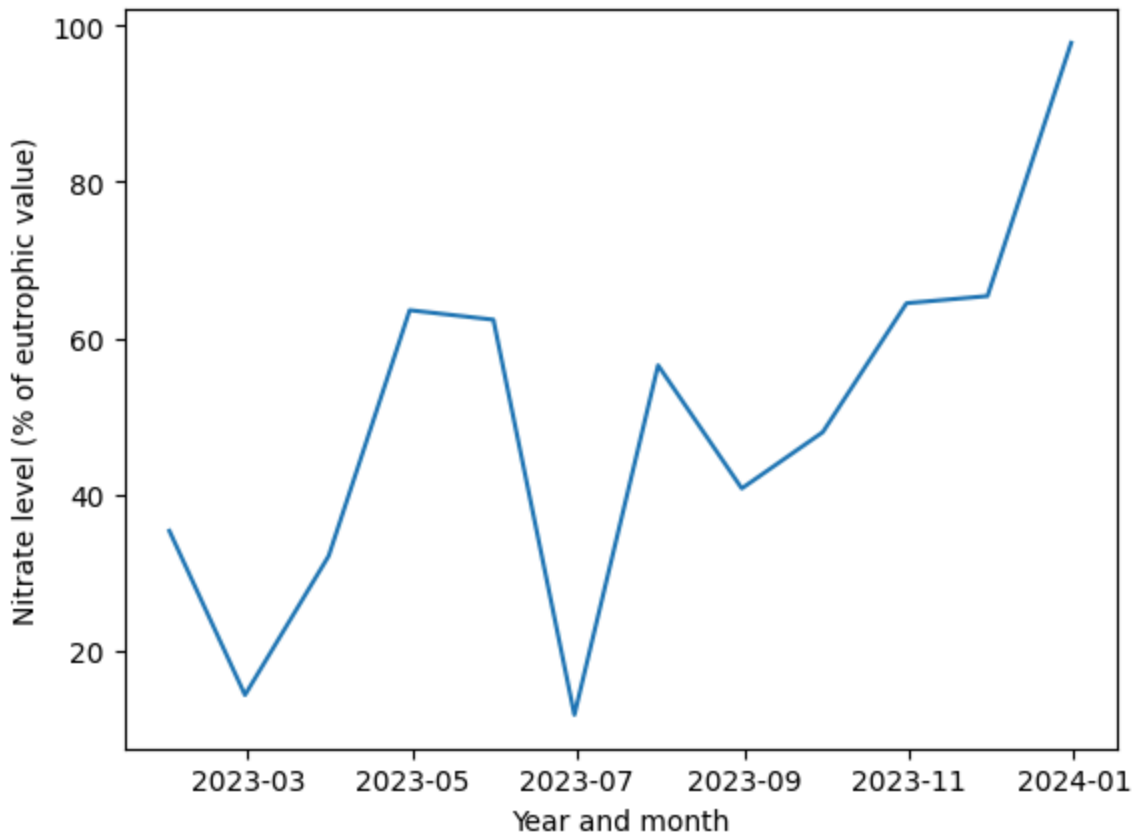
Another option is to add data labels to the plot for the last 3 months. This causes the same effect, the user will focus on the point of the plot which contains the data labels.

Size can also be leveraged. If we add data labels, we can increase the size of the labels to draw attention.

In [1]:
```
import numpy as np
import seaborn as sns
import pandas as pd
```

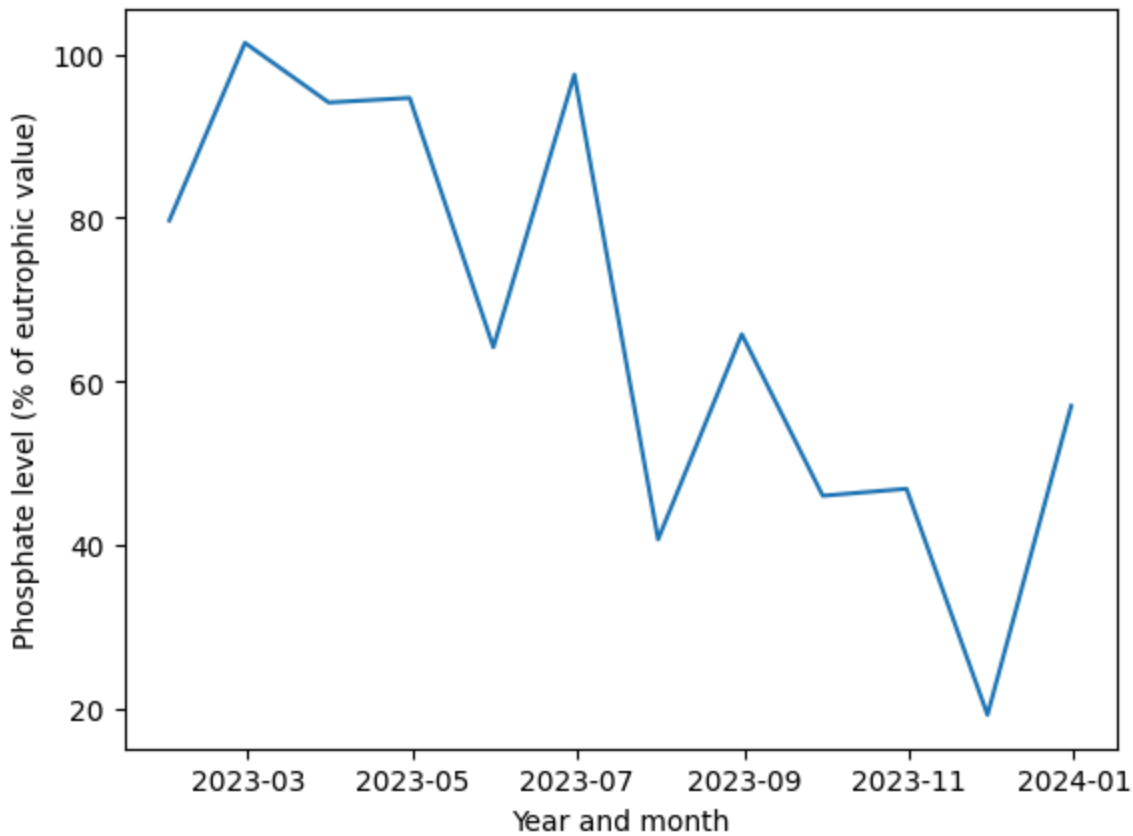
In [4]:
```
numdates = 12
np.random.seed(0)
time_series = 100 * (np.random.normal(size = numdates) / 5 + np.arange(numdates) /
datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
ax = sns.lineplot(x = datearray, y = time_series)
ax.set_xlabel("Year and month")
ax.set_ylabel("Nitrate level (% of eutrophic value)")
```

Out[4]:
```
Text(0, 0.5, 'Nitrate level (% of eutrophic value)')
```



In [5]:
```
np.random.seed(1)
time_series = 100 * (1 - (np.random.normal(size = numdates) / 8 + np.arange(numdate
datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
ax = sns.lineplot(x = datearray, y = time_series)
ax.set_xlabel("Year and month")
ax.set_ylabel("Phosphate level (% of eutrophic value)")
```

Out[5]:
```
Text(0, 0.5, 'Phosphate level (% of eutrophic value)')
```

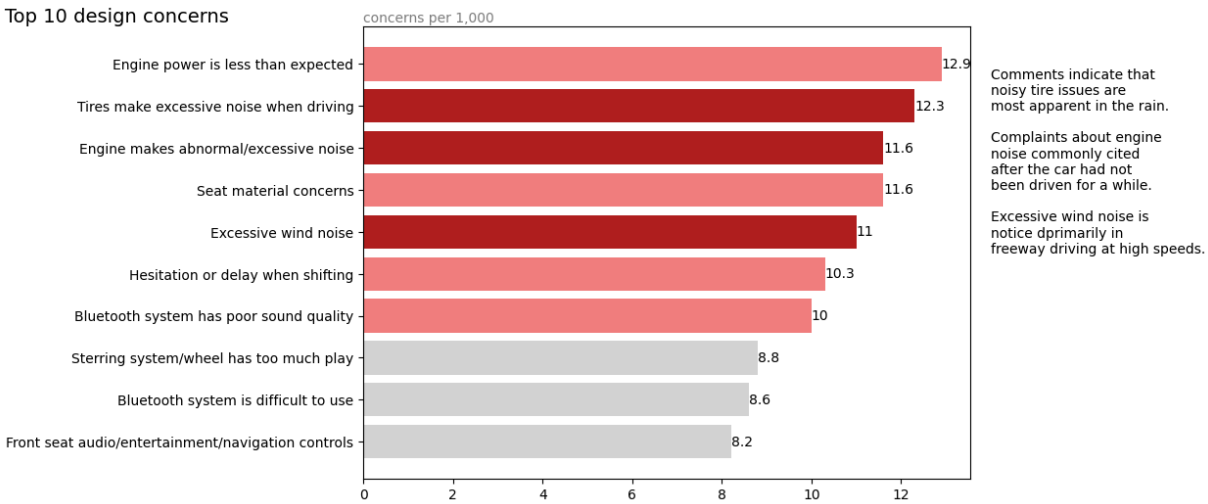


3. Working on your datasets

This week, you can work on Milestone One rather than having a particular exercise about working on your datasets

4. Storytelling With Data plot

Reproduce any graph of your choice in p. 99-119 of the Storytelling With Data book as best you can. (The first half of chapter four). You do not have to get the exact data values right, just the overall look and feel.

In [78]:
```
# Figure 4.9 (p. 111)
import matplotlib.pyplot as plt

labels = [
    'Front seat audio/entertainment/navigation controls',
    'Bluetooth system is difficult to use',
    'Sterring system/wheel has too much play',
    'Bluetooth system has poor sound quality',
    'Hesitation or delay when shifting',
    'Excessive wind noise',
    'Seat material concerns',
    'Engine makes abnormal/excessive noise',
    'Tires make excessive noise when driving',
```

```
    'Engine power is less than expected'
]
y_pos = np.arange(len(labels))
data = [
    8.2, 8.6, 8.8, 10.0, 10.3, 11.0, 11.6, 11.6, 12.3, 12.9
]
colors = ['lightgray','lightgray','lightgray','lightcoral','lightcoral','firebrick'

plt.figure(figsize=(8, 6))
bars = plt.barh(y_pos, data, color=colors)
plt.bar_label(bars)
plt.yticks(
    y_pos,
    labels
)
plt.text(-8, 10.0, 'Top 10 design concerns', fontsize=14)
plt.text(0, 10.0, 'concerns per 1,000', fontsize=10, color='grey')
plt.text(14, 4.5, 'Comments indicate that \nnoisy tire issues are \nmost apparent i
None
```